



ClansCore

Weiterentwicklung Discord-Bot für Vereine

Studien- und Bachelorarbeit im
Herbst-Semester 2025

Ausdruck:

11.12.2025

Team / Autoren:

Vanessa Alves und Joel Thoma

Referent:

Prof. Dr. Frieder Loch

Koreferent:

Julia Czerniak-Wilmes

Gegenleser:

Severin Dellsperger

Abstract

Management Summary

Ausgangslage

Vorgehen und Technologien

Ergebnisse

Danksagung

Inhaltsverzeichnis

I	Produkt Dokumentation	1
1	Einleitung	1
1.1	Problemstellung	1
1.2	Vorarbeiten	1
1.3	Ziel der Arbeit und Abgrenzung	2
1.4	Rahmenbedingungen	2
1.5	Stand der Technik	3
2	Anforderungen	4
2.1	Vorgehen	4
2.2	Priorisierung der Anforderungen	4
2.3	Legende zum Erfüllungsgrad der Anforderungen	5
2.4	User Stories	5
2.5	Functional Requirements	6
2.6	Use Cases	9
2.7	Non-Functional Requirements	13
3	Erweitertes Gamification-Konzept	15
3.1	Self-Determination Theory	15
3.2	Interviews zur Motivation von Vereinsmitgliedern	16
3.2.1	SDT-basierte Analyse der Interviewergebnisse	16
3.2.2	Querschnittliche Implikationen	16
3.3	Octalysis Framework	17
3.4	Erweiterungen, Chancen und Regeln	18
3.4.1	Designprinzipien	18
3.4.2	Geplante Erweiterungen und Regeln	19
3.4.3	Technische Implikationen und Roadmap	20
3.4.4	Zukünftige Erfolgsmessung	20
4	Punktesystem und Punkte-Vorschlag	21
4.1	Erstellung im Dashboard	21
4.2	Aufgabenerstellung	21
5	Erweitertes Sicherheitskonzept	22
5.1	Ziel und Geltungsbereich	22
5.2	Sicherheitsprinzipien	22
5.3	Authentifizierung und Autorisierung	23
5.3.1	Dienstkonto und automatisierte Zugriffe	23
5.3.2	Absicherung des Datenbankzugriffs	23
5.3.2.1	Bewertung und Entscheid	23
5.3.2.2	Übertragbarkeit auf andere Vereine	24
5.3.3	Dashboard-Login	24
5.4	Daten- und Schnittstellensicherheit	24
5.4.1	Eingabevalidierung	24
5.4.2	Synchronisierung und Konfliktlösung	24
5.5	Datenschutz und DSGVO / DSGVO-Umsetzung	25
5.6	Server- und Deployment-Sicherheit	25
5.7	Manipulations- und Integritätsschutz	25
5.8	Risikoanalyse und Massnahmen	25
5.9	Monitoring und Wartung	26
5.10	Abgrenzung des Sicherheitskonzeptes	26
6	Systemanalyse, Architektur und Wireframes	27
6.1	Domain Analyse	27
6.2	C4-Modell	28
6.2.1	System Context Diagramm	28
6.2.2	Container Diagramm	29
6.2.3	Component Diagramm	30

6.3	Wireframes	32
6.3.1	Nicht-Mitglied	32
6.3.2	Gamification-Verwaltung	34
7	Qualitätssicherung	36
7.1	Testkonzept	36
7.1.1	Vorgehensweise	36
7.1.2	Testziele	36
7.1.3	Testumgebung	36
7.1.4	Unit- und Integrationstests	36
7.1.5	Usability-Tests	36
7.2	Ablaufplan	37
7.3	Code-Qualität	38
7.4	Definition of Done	38
8	Implementation	39
8.1	Datenbank	39
8.1.1	Anpassungen der Entitäten	39
8.1.2	Datenbank-Transaktionen	40
8.2	Architekturelles Refactoring	40
8.2.1	Analyse der Ausgangsarchitektur im Vorprojekt	41
8.2.2	Neue Zielarchitektur (ClansCore)	41
8.2.3	Umsetzung und Ergebnis des Refactorings	42
8.3	Erstellung Dashboard	43
8.4	Erweiterung Discord-Bot	43
8.5	Limitierungen	43
8.5.1	Gamification-Erweiterungen	43
8.5.2	Rollenbenennungen	43
8.5.3	Logging	43
8.5.4	Event-Synchronisierung	43
9	Fazit	44
9.1	Resultate	44
9.2	Schlussfolgerung	44

II Projekt Dokumentation 45

10	Projekt Plan	45
10.1	Zusammenarbeit	45
10.2	Meetings	45
10.3	Minimum Viable Product (MVP)	45
10.4	Long-Term Plan (Version 1.0 - Ursprüngliche Planung)	46
10.4.1	Inception (Initiierung)	47
10.4.2	Elaboration (Ausarbeitung)	47
10.4.3	Construction (Konstruktion)	47
10.4.4	Transition (Übergang)	47
10.4.5	Presentation (Präsentation)	47
10.4.6	Meilensteine	48
10.5	Risikomanagement	49
10.5.1	Ausweichplan	52
10.6	Auswirkungen des Refactorings auf die Planung	52
10.6.1	Teamverteilung und Einfluss auf den Projektverlauf	52
10.6.2	Auswirkungen auf Zeitplan und Meilensteine	53
11	Arbeitszeitnachweis	56
12	Tooling und Technologie-Entscheidungen	57
12.1	Jira	57
12.2	Teams	57
12.3	GitHub	57
12.3.1	GitHub Guidelines	58
12.4	Dokumentation	58

12.4.1	Dokumentation Guidelines	58
12.4.2	Dokumentation Deployment	58
12.5	Code und Entwicklung	59
12.5.1	Code Guidelines	59
12.5.2	Setup	59
12.5.3	Hosting	59
12.5.4	Code Deployment	59
12.6	Datenbank	60
12.7	Programmiersprachen und Frameworks	60
12.8	UI-Bibliothek	61
12.9	KI-Tools	61
13	Sitzungsprotokolle	62
14	Persönliche Erfahrungsberichte	70
14.1	Bericht Joel Thoma	70
14.2	Bericht Vanessa Alves	70
III	Glossar und Abkürzungsverzeichnis	71
IV	Literaturverzeichnis	73
V	Abbildungsverzeichnis	75
VI	Tabellenverzeichnis	76
VII	Code-Listings	79
VIII	Anhang	80
15	Interviews zur Vereins-Motivation	80
16	Technische Ergänzungen zum Sicherheitskonzept	80
16.1	Vergleich der DB-MFA-Varianten	80
16.2	Dashboard-Sicherheit und Passwortmanagement	81
16.3	Nebenläufigkeitskontrolle und Transaktionen	82
16.4	Container- und Deployment-Details	82
17	Test-Ergebnisse	82
18	Deployment	82
19	Screenshots Discord-Bot	82
20	Screenshots Dashboard	82

Kapitel I

Produkt Dokumentation

1 Einleitung

Digitale Kommunikationsplattformen wie Discord haben sich in den vergangenen Jahren zunehmend als zentrale Werkzeuge für die Organisation und Interaktion innerhalb von Online-Communities und Vereinen etabliert. Insbesondere im Bereich der Freizeit- und Gaming-Vereine bieten sie die Möglichkeit, Kommunikation, Eventplanung und Mitgliederverwaltung zu bündeln. Trotz dieser Vorteile stossen bestehende Plattformfunktionen bei wachsender Mitgliederzahl, zunehmender organisatorischer Komplexität und steigenden Anforderungen an Datensicherheit und Automatisierung an ihre Grenzen.

Vor diesem Hintergrund befasst sich die vorliegende Bachelorarbeit mit der Weiterentwicklung eines im Vorprojekt erstellten Discord-Bots, der vereinsrelevante Prozesse automatisiert und Gamification-Mechanismen zur Steigerung des Engagements nutzt. Ziel der Arbeit ist es, das bestehende System, um ein plattformunabhängiges Admin-Dashboard zu erweitern, welches die sichere, benutzerfreundliche und rollenbasierte Verwaltung von Mitgliederdaten und Gamification-Statistiken ermöglicht. Damit wird ein Beitrag zur Entkopplung der Vereinsverwaltung von der Kommunikationsplattform Discord geleistet und die Grundlage für eine modulare, erweiterbare Systemarchitektur geschaffen.

1.1 Problemstellung

Das im Vorprojekt entwickelte System ermöglicht die Automatisierung einiger zentraler Vereinsprozesse, ist jedoch ausschliesslich innerhalb der Discord-Umgebung funktionsfähig. Dadurch ist die Nutzung stark Discord-zentriert und für Personen ohne Discord-Konto oder mit eingeschränkter technischer Erfahrung nur begrenzt zugänglich.

Die Bearbeitung vereinsrelevanter Daten erfolgt derzeit ausschliesslich über direkten Zugriff auf die MongoDB-Datenbank. Dies erfordert spezifische technische Kenntnisse, birgt Sicherheitsrisiken und verhindert eine kontrollierte, benutzerfreundliche Datenverwaltung. Zudem existiert keine zentrale Oberfläche, die eine visuelle und statistisch fundierte Auswertung des Vereinsengagements ermöglicht.

Darüber hinaus erschwert die aktuelle Architektur die Anbindung zusätzlicher Plattformen oder externer Systeme, etwa zur Erweiterung um Web- oder Mobile-Anwendungen. Diese Einschränkungen verdeutlichen den Bedarf nach einer modularen, skalierbaren und plattformunabhängigen Systemlösung, die sowohl technische als auch organisatorische Herausforderungen adressiert.

1.2 Vorarbeiten

Das Vorprojekt „Discord-Bot für Vereine“ (Alves & Schnider, 2025) bildete die Grundlage für die vorliegende Arbeit. Es verfolgte das Ziel, zentrale Vereinsprozesse wie Mitgliederverwaltung, Eventorganisation und Gamification in einem Discord-Bot zu integrieren. Der entwickelte Prototyp implementierte ein Rollen- und Berechtigungssystem, eine Anbindung an die Google Calendar API sowie ein auf wissenschaftlichen Erkenntnissen basierendes Gamification-Konzept.

Während die Machbarkeit und der Nutzen eines solchen Systems bestätigt werden konnten, blieb dessen Erweiterbarkeit auf andere Plattformen sowie die administrative Steuerung offen. Das Admin-Dashboard wurde im Vorprojekt lediglich als potenzielle Weiterentwicklung beschrieben, jedoch nicht realisiert. Ebenso fehlten Mechanismen zur Messung und Visualisierung des Engagements sowie eine Entkopplung der Discord-spezifischen Logik von der Datenbankstruktur.

Die vorliegende Arbeit schliesst an diese Ergebnisse an und adressiert die genannten Defizite durch eine Neugestaltung der Architektur, eine verbesserte Datenkonsistenz zwischen Discord, Dashboard und Datenbank sowie durch die Integration einer administrativen Benutzeroberfläche. Gleichzeitig werden Erkenntnisse aus Interviews mit Präsidenten anderer Vereine in die konzeptionelle Weiterentwicklung des Gamification-Systems einbezogen, um dessen Übertragbarkeit und Nachhaltigkeit zu erhöhen.

1.3 Ziel der Arbeit und Abgrenzung

Ziel dieser Bachelorarbeit ist die Konzeption und Entwicklung einer plattformunabhängigen, administrativen Erweiterung des bestehenden Discord-Bots, die eine effiziente und sichere Verwaltung vereinsrelevanter Daten ermöglicht. Hierbei liegt der Fokus auf der funktionalen Umsetzung eines Admin-Dashboards sowie auf der architektonischen Modularisierung des Systems, um künftige Integrationen weiterer Plattformen leichter zu ermöglichen. Die neue Softwarelösung wird mit dem Namen ClansCore adressiert.

Im Einzelnen verfolgt die Arbeit folgende Teilziele:

- Entwurf und Implementierung eines funktionsfähigen Admin-Dashboards in ClansCore, zur Verwaltung von Mitgliedern, Rollen, Events und Gamification-Daten
- Refaktorisierung der ClansCore-Backend-Architektur zur Entkopplung von Discord-spezifischer Logik und zur Vorbereitung auf eine plattformübergreifende Erweiterung
- Entwicklung einer sicheren Schnittstellenkommunikation zwischen Bot, Dashboard und Datenbank zur Gewährleistung konsistenter Datenzustände innerhalb des gesamten ClansCore-Systems
- konzeptionelle und technische Erweiterung des Gamification-Systems, insbesondere zur Messung des Mitgliederengagements
- Evaluation von ClansCore durch Usability-Tests mit Vereinsmitgliedern

Diese Arbeit unterscheidet sich vom Vorprojekt durch ihren erweiterten technischen und analytischen Anspruch. Während das Vorprojekt primär die Funktionalität innerhalb von Discord validierte, liegt der Schwerpunkt der Bachelorarbeit auf der systematischen Generalisierung und Professionalisierung der Lösung als Gesamt-System namens ClansCore. Damit wird die Grundlage geschaffen, um das System langfristig als universelles Vereinsmanagement-Tool einsetzen zu können.

1.4 Rahmenbedingungen

Die Arbeit wurde im Rahmen der Bachelor- und Studienarbeit des Studiengangs Informatik an der Ostschweizer Fachhochschule (OST) durchgeführt.

- **Arbeitstyp:** Bachelorarbeit (360 Stunden) + Studienarbeit (240 Stunden)
- **Gesamtarbeitsaufwand:** 600 Stunden
- **ECTS-Punkte:** 12 ECTS (Bachelorarbeit) und 8 ECTS (Studienarbeit)

Der Schwerpunkt liegt auf der Konzeption, Entwicklung und Evaluation eines funktionalen Software-Systems mit wissenschaftlich begründetem Gamification-Ansatz, welches den Namen ClansCore trägt. Die Arbeit vereint methodische Ansätze aus den Bereichen Software-Engineering, Usability und Motivationsforschung und ist als entwicklungsorientierte empirische Arbeit mit prototypischem Charakter einzuordnen.

1.5 Stand der Technik

Der Stand der Technik im Bereich digitaler Vereinsorganisation wurde im Vorprojekt umfassend untersucht und bildet die Grundlage der vorliegenden Arbeit. Die damalige Marktanalyse zeigte, dass bestehende Systeme entweder auf die Interaktion und Gamification innerhalb von Discord oder auf die administrative Vereinsverwaltung über klassische Softwarelösungen fokussieren, jedoch keine integrative Verbindung beider Ansätze bieten.

Seit Abschluss des Vorprojekts im Juli 2025 sind zwar Weiterentwicklungen einzelner Systeme erkennbar, eine Lösung, welche Gamification, Datenverwaltung und plattformübergreifende Nutzung in einem kohärenten Konzept vereint, wurde bisher jedoch nicht öffentlich dokumentiert.

Im Bereich der Discord-Bots zählt MEE6 zu den meistgenutzten Anwendungen und bietet über das Plugin „Statistics Channels“ grundlegende Auswertungen zur Serveraktivität sowie ein Levelsystem zur Förderung der Nutzerinteraktion (MEE6-Contributors, 2025). Auch der Bot Arcane integriert vergleichbare Gamification-Funktionen in Form von Belohnungssystemen (Arcane-Contributors, 2025). Beide Systeme bleiben jedoch auf die Discord-Plattform beschränkt und verfügen über keine dokumentierten Schnittstellen zu externen Administrations- oder Datenverwaltungssystemen.

Demgegenüber adressieren klassische Vereinsmanagement-Lösungen wie ClubDesk oder WildApricot die organisatorischen Anforderungen von Vereinen, insbesondere in den Bereichen Mitgliederverwaltung, Eventplanung und Buchhaltung (ClubDesk-Contributors, 2025; WildApricot-Contributors, 2025). Motivierende oder interaktive Komponenten, wie sie in Gamification-Konzepten beschrieben werden, sind in diesen Systemen kein grosser Bestandteil. Diskussionen über eine mögliche Erweiterung durch Belohnungsmechanismen existieren lediglich in Anwenderforen, was die bislang geringe praktische Umsetzung solcher Konzepte verdeutlicht (WildApricot-Community, 2015).

Die vorliegende Arbeit positioniert sich zwischen diesen beiden Ansätzen, indem sie den organisatorischen Verwaltungsfokus klassischer Systeme mit der motivierenden Wirkung von Gamification-Elementen in ClansCore kombiniert. Durch die laufende Einführung des vorherigen Systems (Discord-Bot) beim Verein EGSwiss über das Teammitglied Vanessa Alves, während der Sommermonate 2025, konnten Rückmeldungen des Vorstandes gesammelt und praxisrelevante Anforderungen sowie Verbesserungsvorschläge identifiziert werden. Diese sind in der bestehenden Softwarelösung bislang unberücksichtigt geblieben sind. Damit leistet die Arbeit einen Beitrag zur Integration von Motivation, Datenmanagement und Vereinsorganisation in einem einheitlichen, plattformunabhängigen Anwendungskonzept ClansCore.

2 Anforderungen

Dieses Kapitel stellt die Anforderungen an ClansCore für den Verein systematisch dar. Die Anforderungen basieren auf klar definierten User Stories, welche die Bedürfnisse der Nutzer abbilden. Aus diesen User Stories werden die Functional Requirements abgeleitet, welche die konkreten Funktionen des Bots und des Dashboards spezifizieren. Darauf aufbauend definieren die Use Cases, wie diese Anforderungen in der geplanten Umsetzung realisiert werden. Neben den Functional Requirements werden auch die Non-Functional Requirements berücksichtigt, um Qualitätsmerkmale wie Sicherheit, Skalierbarkeit und Benutzerfreundlichkeit sicherzustellen. Zudem erfolgt eine Priorisierung der Anforderungen, damit die wichtigsten Funktionen frühzeitig implementiert werden. Im Gegensatz zum Vorprojekt liegt der Schwerpunkt auf der Erweiterung des bestehenden Discord-Bots zu einem modularen und sicheren Verwaltungssystem mit Web-Dashboard (siehe Abschnitt 1.3).

2.1 Vorgehen

Die Anforderungen wurden aus der Aufgabenstellung, dem Vorprojekt, den Praxiserfahrungen von Vanessa Alves in ihrer Funktion als Präsidentin des Vereins EGSwiss sowie einer ergänzenden Analyse der Domäne abgeleitet. Der Erhebungsprozess erfolgte in mehreren Schritten:

1. User Stories erfassen die Bedürfnisse und Erwartungen der verschiedenen Rollen.
2. Functional Requirements (FR) leiten daraus die technologie-neutralen Systemfunktionen ab.
3. Use Cases (UC) konkretisieren die Interaktionen zwischen Nutzern und System und validieren die FR.
4. Non-Functional Requirements (NFR) definieren überprüfbare Qualitäts- und Leistungsanforderungen.

Im Gegensatz zum Vorprojekt liegt der Schwerpunkt dieser Arbeit auf der Erweiterung des bestehenden Discord-Bots zu einem modularen und sicheren Verwaltungssystem mit Web-Dashboard (siehe Abschnitt 1.3). Zwischen User Stories, Functional Requirements und Use Cases besteht kein 1:1-Mapping. Die Beziehungen sind häufig viele-zu-viele.

Die Umsetzung und Evaluierung konzentriert sich auf die Nutzung des Admin-Dashboards auf Desktop-Systemen. Eine Optimierung für mobile Endgeräte und Cross-Device-Design liegt ausserhalb des Projektumfangs.

2.2 Priorisierung der Anforderungen

Damit der Entwicklungsprozess auf die wichtigsten Features fokussiert bleibt, erhalten gewisse Anforderungen eine höhere Priorität als andere. Dadurch kann zügig ein Prototyp entwickelt, getestet und auf Basis von Nutzerfeedback iterativ verbessert werden. Die verpflichtenden Use Cases und Non-Functional Requirements bilden das MVP (Minimum Viable Product) und stellen die funktionale Basis des Projekts dar.

Die nachfolgende Auflistung beschreibt die Prioritäts-Kategorien und deren Bedeutung:

Prioritäts-Kategorie	Beschreibung
Hoch (<u>MVP</u>) - Essenziell für die erste Version	Diese Anforderungen sind direkt mit den Kernanforderungen des MVP verknüpft und müssen für eine funktionale Erstversion vorhanden sein.
Mittel - Wichtige Funktionen nach dem MVP	Diese Anforderungen ergänzen das System sinnvoll, sind jedoch nicht kritisch für die erste Version.
Niedrig - Erweiterungen für spätere Versionen	Diese Anforderungen verbessern die Benutzerfreundlichkeit und Verwaltung, sind aber nicht erforderlich für den ersten MVP-Release.

Tab. 1: Beschreibung der Prioritäts-Kategorien

2.3 Legende zum Erfüllungsgrad der Anforderungen

Die Resultate der Anforderungen werden in den Tabellen dokumentiert. Die farbliche Bewertung des Erfüllungsgrads erfolgt erst im Fazit, am Ende des Projekts, nicht während der Spezifikation.

Farbe	Status (wird am Projektende gesetzt)
 	Ziel der Anforderung wurde erreicht.
 	Ziel der Anforderung wurde zum Teil oder über einen anderen Weg erreicht.
 	Ziel der Anforderung wurde nicht erreicht.

Tab. 2: Beschreibung der Resultats-Kategorien

2.4 User Stories

Die nachfolgenden User Stories beschreiben die Erwartungen und Bedürfnisse der Nutzer an das aus dem Vorprojekt erweiterte System ClansCore. Sie werden aus der Sicht der Endanwender formuliert und bilden die Grundlage für die Ableitung der Functional Requirements. Die Analyse berücksichtigt die Rollen Vorstand, Mitglied, Nicht-Mitglied sowie die Systemperspektive Verein.

Als Verein...

- möchte ich ClansCore plattformunabhängig und modular betreiben, um flexibel auf neue Plattformen reagieren zu können.
- möchte ich eine zentrale Übersicht über Rollen, Events, Aufgaben und Aktivitäten haben, um operative und strategische Entscheide zu treffen.
- möchte ich Verlässlichkeit und Vertrauen durch Datenintegrität, Sicherheit und regelmässige Sicherungen / Überwachung sicherstellen.

Als Vorstandsmitglied...

- möchte ich Mitglieder, Rollen und Events im Admin-Dashboard verwalten, inklusive Rechte und Synchronisation mit Discord.
- möchte ich Aufgaben und Gamification erfassen und steuern sowie Analysen zu Aktivität und Engagement erhalten.
- möchte ich Zahlungserinnerungen und Zahlungsbestätigungen automatisiert erhalten, um administrativen Aufwand zu reduzieren.
- möchte ich Nachvollziehbarkeit durch Änderungs- und Transaktionsprotokolle sowie Datenintegritätsprüfungen.
- möchte ich sicheren Administrationszugriff sowie Monitoring und Alarme bei Konflikten und Sicherheitsereignissen.

Als Mitglied...

- möchte ich Aufgaben, Punkte und Ranglisten plattformunabhängig einsehen, jedoch konsistent zu Discord.
- möchte ich transparente Punkteberechnung verstehen und mich für Aufgaben anmelden.
- möchte ich meine Datenschutzrechte (Auskunft, Löschung, Einwilligung) selbst ausüben.

Als Nicht-Mitglied...

- möchte ich mich ausserhalb von Discord bewerben können.

2.5 Functional Requirements

Die Functional Requirements leiten sich aus den User Stories ab und spezifizieren die erwarteten Funktionen von ClansCore. Sie beschreiben, welche neuen oder erweiterten Features im Rahmen dieser Arbeit implementiert werden müssen, um die in der Einleitung definierten Ziele zu erreichen (siehe Abschnitt 1.3).

Die Kernfunktionen umfassen Gamification, Vereins- und Eventverwaltung. Basis-Funktionen (z. B. Passwortänderung oder -zurücksetzung) sichern die erwarteten Standardprozesse ab.

Im Gegensatz zum Vorprojekt liegt der Schwerpunkt auf Plattformunabhängigkeit, Sicherheit, Datenintegrität und Erweiterbarkeit durch ein webbasiertes Dashboard.

ID	FR1
Name	Login und Authentifizierung
Beschreibung	• Das Admin-Dashboard stellt ein sicheres Loginsystem mit Benutzernamen und Passwort bereit (Passwort-Policy, Sperrmechanismen/Rate-Limiting, sichere Session-Verwaltung). • Das System muss optional eine <u>Multi-Faktor-Authentifizierung</u> (MFA) unterstützen. • Authentifizierungs- und Autorisierungsvorgänge werden nachvollziehbar protokolliert. • Transport der Anmeldedaten erfolgt über verschlüsselte Verbindungen.

Tab. 3: Beschreibung Functional Requirement 1

ID	FR2
Name	Online-Mitgliedsbewerbung
Beschreibung	• Das Dashboard muss ein Bewerbungsformular für Nicht-Mitglieder bereitstellen. • Bewerbungen müssen automatisch in der Datenbank gespeichert und dem Vorstand im Dashboard angezeigt werden.

Tab. 4: Beschreibung Functional Requirement 2

ID	FR3
Name	Verwaltung von Mitgliedern, Rollen und Events
Beschreibung	• Mitgliederprofile und Rollen anzeigen / bearbeiten. • Events erstellen, bearbeiten und löschen. • Änderungen werden mit der Discord-Datenhaltung konsistent synchronisiert.

Tab. 5: Beschreibung Functional Requirement 3

ID	FR4
Name	Steuerung und Rechteverwaltung
Beschreibung	• Das Dashboard muss erlauben, die Zugriffsrechte für Bot-Funktionen und Dashboard-Bereiche zu verwalten. • Es muss konfigurierbar sein, welche Rollen welche Aktionen ausführen dürfen. • Änderungen an Berechtigungen müssen automatisch protokolliert werden.

Tab. 6: Beschreibung Functional Requirement 4

ID	FR5
Name	Gamification-Parameterverwaltung
Beschreibung	• Das Dashboard muss die Erfassung, Anpassung und Gewichtung von Gamification-Parametern ermöglichen. • Bei der Erstellung von Aufgaben muss das System automatisch eine empfohlene Punktezahl vorschlagen.

Tab. 7: Beschreibung Functional Requirement 5

ID	FR6
Name	Gamification-Datenanalyse
Beschreibung	• Detaillierte Statistiken zu Aufgaben, Belohnungen und Aktivitäten. • Near-Real-Time-Auswertung und Visualisierung von Mitgliederaktivitäten (End-to-End-Latenz ≤ 5 Sekunden für Ereignisse unter Normalbetrieb und keine Anforderungen eines Hard-Echtzeitsystems). • Aggregierte, datensparsame Kennzahlen (z. B. Eventteilnahmequote, Helferquote, Trendlinien) ohne personenbezogenes Dauertracking.

Tab. 8: Beschreibung Functional Requirement 6

ID	FR7
Name	Gamification-Ansicht für Mitglieder
Beschreibung	• Mitglieder sehen Punkte, Aufgaben, Belohnungen (Rewards), Ranglisten (Leaderboard), Punkte-Historie sowie die Regeln der Punktevergabe. • Alle Werte stammen aus der zentralen Datenquelle (Konsistenz zu Discord).

Tab. 9: Beschreibung Functional Requirement 7

ID	FR8
Name	Aufgabenverwaltung und Anmeldung
Beschreibung	• Das Dashboard muss die Erstellung und Verwaltung von Aufgaben ermöglichen. • Mitglieder müssen sich über das Dashboard für Aufgaben anmelden können. • Der Fortschritt und Abschlussstatus muss vom Vorstand überprüft und freigegeben werden können.

Tab. 10: Beschreibung Functional Requirement 8

ID	FR9
Name	Zahlungserinnerung und Bestätigung
Beschreibung	• Das System muss periodisch prüfen, ob ein Mitglied seinen Mitgliederbeitrag bezahlt hat. • Bei ausstehenden Zahlungen muss automatisch eine Erinnerung über Discord oder das Dashboard versendet werden. • Nach der Erfassung einer eingegangenen Zahlung muss das System dem Vorstand automatisch eine Bestätigung übermitteln. • Die Erinnerung darf erst nach Ablauf des definierten Vereinsjahres erfolgen. • Die Zahlungsinformationen müssen revisionssicher gespeichert werden.

Tab. 11: Beschreibung Functional Requirement 10

ID	FR10
Name	Event-Reminder-System
Beschreibung	• Das System erinnert Mitglieder automatisch an anstehende Events • Es können mehrere Erinnerungen an ein Event angeknüpft werden, um ein mehrstufiges System zu haben. • Optional werden die Mitglieder per Direktnachricht kontaktiert.

Tab. 12: Beschreibung Functional Requirement 9

ID	FR11
Name	Plattformunabhängigkeit und Erweiterbarkeit
Beschreibung	• Architektur erlaubt Integration weiterer Plattformen neben Discord. • Gemeinsame Funktionen (Mitglieder, Aufgaben, Rollen, Gamification) sind über eine zentrale Logik- / Datenschicht verfügbar. • Kommunikation über klar definierte Schnittstellen (z. B. REST, WebSocket, Event-Queue). • Neue Plattformen sollen ohne tiefgreifende Änderungen am Kernsystem hinzugefügt werden können (Implementierung neuer Interfaces, ohne grosse Anpassungen des bestehenden Backends).

Tab. 13: Beschreibung Functional Requirement 11

2.6 Use Cases

Die Use Cases beschreiben detailliert die Interaktionen zwischen Nutzern und ClansCore. Sie zeigen, wie die **Functional Requirements** umgesetzt werden. Darüber hinaus konkretisieren sie Anforderungen an Sicherheit, Skalierbarkeit und Benutzerfreundlichkeit.

ID	UC1
Name	Login und Authentifizierung (<u>FR1</u>)
Akteur	Mitglied, Vorstand
Beschreibung	Der Zugriff auf das Admin-Dashboard erfordert eine Authentifizierung (optional wird <u>MFA</u> unterstützt).
Voraussetzung	Ein gültiger Dashboard-Account besteht.
Standard-Verlauf	1. Der Benutzer gibt Benutzername und Passwort ein. 2. ClansCore prüft die Zugangsdaten. 3. (Optional) Das System fordert den zweiten Faktor an und validiert ihn. 4. Bei Erfolg wird der Benutzer zur Startseite weitergeleitet.
Alternativ-Verlauf	• Ungültige Zugangsdaten führen zur Fehlermeldung. • Benutzer hat das Passwort vergessen und kann es zurücksetzen (<u>UC3</u>).
Nachbedingung	Der Benutzer ist eingeloggt und der Zugriff wird protokolliert.
Priorität	Hoch
Resultat	

Tab. 14: Beschreibung Fully dressed Use Case 1

ID	UC2
Name	Online-Mitgliedsbewerbung (<u>FR2</u>)
Akteur	Nicht-Mitglied, Vorstand
Beschreibung	Nicht-Mitglieder bewerben sich ausserhalb von Discord. Der Vorstand wird kanalneutral informiert (Dashboard-Hinweis / Discord-Nachricht / E-Mail). Die konkrete Ausprägung ist konfigurierbar.
Voraussetzung	Dashboard ist mit der Vereinsdatenbank verbunden.
Standard-Verlauf	1. Nicht-Mitglied füllt das Bewerbungsformular aus und stimmt der Datenverarbeitung zu. 2. ClansCore speichert die Bewerbung und informiert den Vorstand im Dashboard. 3. Der Vorstand prüft die Angaben und entscheidet (Annahme / Ablehnung). 4. Bei Annahme wird der Mitgliedsstatus gesetzt und die Synchronisation mit Discord ausgelöst. 5. Bewerbende erhalten eine Status-Benachrichtigung über den Entscheid.
Alternativ-Verlauf	• Unvollständige / fehlerhafte Angaben führen zu einem Validierungsfehler und Korrekturaufforderung. • Ablehnung führt zu einer Benachrichtigung mit Begründung (sofern vorgesehen).
Nachbedingung	Es wird im System dokumentiert und Rollen / Synchronisation sind konsistent.
Priorität	Hoch
Resultat	

Tab. 15: Beschreibung Fully dressed Use Case 2

ID	UC3
Name	Verwaltung von Mitgliedern, Rollen und Rechten (<u>FR3</u> , <u>FR4</u>)
Akteur	Vorstand
Beschreibung	Der Vorstand verwaltet Mitglieder, Rollen, Events und Berechtigungen zentral im Dashboard.
Voraussetzung	Dashboard ist mit der Vereinsdatenbank verbunden und das Vorstandsmitglied ist eingeloggt.
Standard-Verlauf	1. Vorstand ruft den Verwaltungsbereich auf. 2. Anzeigen / Bearbeiten von Mitgliedern, Rollen und Events. 3. Konfiguration von Zugriffsrechten und erlaubten Aktionen. 4. Änderungen werden gespeichert und synchronisiert.
Alternativ-Verlauf	• Ein Validierungsfehler führt zur Fehlermeldung und die Änderung wird nicht übernommen.
Nachbedingung	Änderungen sind konsistent persistiert und protokolliert.
Priorität	Hoch
Resultat	

Tab. 16: Beschreibung Fully dressed Use Case 3

ID	UC4
Name	Gamification-Parameter verwalten (<u>FR5</u>)
Akteur	Vorstand
Beschreibung	Der Vorstand pflegt die Gewichtungen / Parameter, welche für Punktevorschläge verwendet werden.
Voraussetzung	Vorstandsmitglied ist eingeloggt.
Standard-Verlauf	1. Öffnen der Parameterverwaltung. 2. Anpassen der Parameter und Speichern.
Alternativ-Verlauf	Unvollständige / fehlerhafte Eingaben ergeben einen Validierungsfehler.
Nachbedingung	Parameter sind aktuell und künftige Punktevorschläge berücksichtigen die Anpassungen.
Priorität	Hoch
Resultat	

Tab. 17: Beschreibung Fully dressed Use Case 4

ID	UC5
Name	Aufgabenverwaltung mit Punktevorschlag (<u>FR5</u> , <u>FR8</u>)
Akteur	Vorstand
Beschreibung	Der Vorstand erstellt und veröffentlicht Aufgaben. ClansCore schlägt basierend auf Parametern eine Punktezahl vor.
Voraussetzung	Dashboard ist verbunden und das Vorstandsmitglied ist eingeloggt. Die Gamification-Parameter sind gepflegt.
Standard-Verlauf	1. Vorstand erfasst Aufgabe (Metadaten, Kriterien). 2. System zeigt einen berechneten Punktevorschlag. 3. Vorstand bestätigt / ändert und veröffentlicht die Aufgabe.
Alternativ-Verlauf	• Wenn keine Parameter vorhanden sind, gibt es keinen Vorschlag. Die Aufgabe kann dennoch mit manueller Punktezahl angelegt werden. • Ein Validierungsfehler führt zur Fehlermeldung.
Nachbedingung	Aufgabe ist veröffentlicht und im System gespeichert.
Priorität	Hoch
Resultat	

Tab. 18: Beschreibung Fully dressed Use Case 5

ID	UC6
Name	Ansicht Aufgaben, Ranglisten und eigene Punkte (FR6 , FR7)
Akteur	Mitglied, Vorstand
Beschreibung	Mitglieder sehen Aufgaben, Leaderboards, eigene Punkte, Rewards, Historie und die Regeln der Punktevergabe. Der Vorstand erhält zusätzliche Analyseansichten.
Voraussetzung	Dashboard ist verbunden und der Benutzer ist eingeloggt.
Standard-Verlauf	1. Mitglied ruft die Gamification-Ansicht auf und sieht Aufgaben, Ranglisten sowie den eigenen Punktestand. 2. Vorstand ruft Analyse-Dashboard auf (Aggregationen / Trends).
Alternativ-Verlauf	• Wenn keine Aufgaben verfügbar sind, werden entsprechende Hinweise angezeigt.
Nachbedingung	Gamification-Daten werden konsistent und aktuell angezeigt.
Priorität	Hoch
Resultat	

Tab. 19: Beschreibung Fully dressed Use Case 6

ID	UC7
Name	Anmeldung zu Aufgaben (FR7 , FR8)
Akteur	Mitglied, Vorstand
Beschreibung	Mitglieder melden sich über das Dashboard für Aufgaben an. Die Freigabe erfolgt durch den Vorstand.
Voraussetzung	Mitglied ist eingeloggt und die Aufgabe veröffentlicht.
Standard-Verlauf	1. Mitglied wählt eine Aufgabe und meldet sich an. 2. System speichert die Anmeldung. 3. Vorstand prüft Abschluss und gibt Punkte frei.
Alternativ-Verlauf	• Ein Fehler bei der Anmeldung führt zu einer Fehlermeldung und erneute Eingabe.
Nachbedingung	Anmeldung / Abschlussstatus sind korrekt gespeichert und Punkte entsprechend gutgeschrieben.
Priorität	Hoch
Resultat	

Tab. 20: Beschreibung Fully dressed Use Case 7

ID	UC8
Name	Benachrichtigungen und Informationen zu Zahlungen (FR9 , FR10)
Akteur	Mitglied, Vorstand, System
Beschreibung	Beiträge werden jährlich fällig. ClansCore automatisiert die Erinnerung und die revisionssichere Rückmeldung zu eingegangenen Zahlungen.
Voraussetzung	Ein Mitglied hat einen ausstehenden Mitgliederbeitrag. Zahlungsdaten sind im System hinterlegt.
Standard-Verlauf	1. Das System prüft periodisch Fälligkeiten und identifiziert ausstehende Zahlungen. 2. Das System versendet eine Zahlungserinnerung (Discord oder Dashboard-Benachrichtigung). 3. Die Zahlung wird erfasst (z. B. manuelle Verbuchung oder Import). 4. Das System speichert Datum und Referenz revisionssicher. 5. Der Vorstand erhält automatisch eine Zahlungsbestätigung im Dashboard.
Alternativ-Verlauf	• Zahlung kann nicht eindeutig zugeordnet werden: der Vorstand wird zur Klärung aufgefordert (offene Position im Dashboard). • Bei erneuter Nichtzahlung nach Erinnerung erscheint der Fall in der Liste „Überfällig“. Der Vorstand entscheidet das weitere Vorgehen.
Nachbedingung	Der Zahlungsstatus des Mitglieds ist aktuell und protokolliert.
Priorität	Mittel
Resultat	

Tab. 21: Beschreibung Fully dressed Use Case 8

ID	UC9
Name	Event-Erinnerung und Benachrichtigungssystem (FR9)
Akteur	Mitglied, Vorstand, System
Beschreibung	ClansCore erinnert Mitglieder automatisch an anstehende Events. Das System unterstützt mehrere Reminder-Stufen und versendet diese plattformunabhängig über Discord, Dashboard-Benachrichtigungen oder andere integrierte Kanäle.
Voraussetzung	Ein Event ist im System erfasst und besitzt konfigurierte Erinnerungstermine.
Standard-Verlauf	1. Das System prüft periodisch alle aktiven Events und identifiziert jene, für die Erinnerungen fällig sind. 2. Das System versendet die konfigurierte Event-Erinnerung (z. B. Discord-DM, Dashboard-Notifikation oder alternative Plattform). 3. Mitglieder erhalten die Benachrichtigung und sehen Details (Datum, Ort, Teilnahmehinweise). 4. Wenn mehrere Reminder konfiguriert sind, sendet das System automatisch weitere Nachrichten bis zum Eventbeginn.
Alternativ-Verlauf	• Ein Event wurde geändert oder abgesagt: Das System sendet eine aktualisierte Event-Mitteilung an alle betroffenen Mitglieder. • Der Erinnerungszeitpunkt ist ungültig oder liegt in der Vergangenheit: Das System protokolliert den Fehler und überspringt den fehlerhaften Reminder.
Nachbedingung	Alle fälligen Erinnerungen wurden zuverlässig ausgelöst und protokolliert. Mitglieder sind über anstehende Events informiert.
Priorität	Mittel
Resultat	

Tab. 22: Beschreibung Fully dressed Use Case 9

ID	UC10
Name	Weitere Plattform-Integration (<u>FR11</u>)
Akteur	Nicht-Mitglied, Mitglied, System
Beschreibung	ClansCore stellt prototypisch Funktionen ausserhalb von Discord oder Dashboard bereit (Bewerbung, Datenabfrage) und synchronisiert zentrale Daten.
Voraussetzung	Neue Plattform ist mit der zentralen Datenbank verbunden.
Standard-Verlauf	1. Nicht-Mitglied reicht Bewerbung über die Plattform ein (<u>UC2</u>). 2. System persistiert Daten und informiert den Vorstand. 3. Vorstand entscheidet. Bei Annahme wird Mitgliedsstatus gesetzt und Daten synchronisiert. 4. Mitglied kann auf der neuen Plattform eigene Personendaten einsehen.
Alternativ-Verlauf	• Ablehnung führt zu keiner Aufnahme und die bewerbende Person wird informiert.
Nachbedingung	Zentrale Daten sind konsistent und Plattformunabhängigkeit ist prototypisch nachgewiesen.
Priorität	Niedrig
Resultat	

Tab. 23: Beschreibung Fully dressed Use Case 10

2.7 Non-Functional Requirements

Die Non-Functional Requirements definieren die Qualitätsmerkmale von ClansCore. Sie betreffen Sicherheit, Wartbarkeit, Zuverlässigkeit, Kompatibilität und Benutzerfreundlichkeit. Jede Anforderung ist überprüfbar und wird im Projektverlauf gemessen oder getestet. Allgemeine Qualitätsziele wie Testabdeckung, CI/CD-Deployments oder Fehlertoleranz werden als etablierte Entwicklungsstandards umgesetzt und sind nicht Teil der expliziten Anforderungen.

ID	NFR1
Beschreibung	Die Codequalität des ClansCore-Systems erfüllt die im Projekt definierten Linting- und Formatierungsrichtlinien. Im CI-Prozess dürfen keine Fehler vom Typ „error“ auftreten.
Anforderungen	Maintainability - Code Quality
Priorität	Hoch
Messung	Automatische Prüfung durch ESLint in der CI-Pipeline. Ziel: 0 Fehler im Main-Branch.
Testen	Bei jedem Commit und Merge Request.
Resultat	

Tab. 24: Beschreibung Non-Functional Requirement 1

ID	NFR2
Beschreibung	Das Admin-Dashboard bietet ein konsistentes, benutzerfreundliches Design mit klarer Navigation und unmittelbarem Feedback bei Interaktionen.
Anforderungen	Usability - Consistency
Priorität	Hoch
Messung	Usability-Test mit mindestens 3 Testpersonen. Erfolgsziel: 80 % der Aufgaben ohne Hilfe innerhalb von ≤ 2 Minuten lösbar.
Testen	Am Ende der Implementierungsphase.
Resultat	

Tab. 25: Beschreibung Non-Functional Requirement 2

ID	NFR3
Beschreibung	Das Dashboard ist mit den aktuellen Versionen von Chrome und Firefox vollständig funktionsfähig. Layout und Interaktionen sind Desktop-first optimiert. Mobile Nutzung ist nicht Teil des Projektumfangs.
Anforderungen	Compatibility - Interoperability
Priorität	Hoch
Messung	Manueller Cross-Browser-Test. Erfolgsziel: 100 % Funktionsumfang und fehlerfreie Darstellung in beiden Browsern.
Testen	Am Ende der Implementierungsphase.
Resultat	

Tab. 26: Beschreibung Non-Functional Requirement 3

ID	NFR4
Beschreibung	Personenbezogene Daten werden nach dem Prinzip „Privacy by Design“ verarbeitet. Speicherung und Übertragung erfolgen ausschliesslich verschlüsselt (TLS, Datenbankverschlüsselung).
Anforderungen	Security - Confidentiality
Priorität	Hoch
Messung	Überprüfung der aktiven HTTPS/TLS-Konfiguration und Verschlüsselungsparameter der Datenbankverbindung.
Testen	Vor Projektabschluss.
Resultat	

Tab. 27: Beschreibung Non-Functional Requirement 4

ID	NFR5
Beschreibung	Änderungen an Mitgliedern, Rollen und Gamification-Parametern müssen revisionssicher nachvollziehbar sein. Jede Änderung ist eindeutig einem Benutzer oder Prozess zuordenbar.
Anforderungen	Security - Accountability
Priorität	Hoch
Messung	Überprüfung von 5 Stichproben im Änderungsprotokoll. Jede Aktion enthält Benutzer-ID, Zeitstempel und Aktionstyp.
Testen	Am Ende der Implementierungsphase.
Resultat	

Tab. 28: Beschreibung Non-Functional Requirement 5

ID	NFR6
Beschreibung	Das Dashboard reagiert auf Benutzerinteraktionen innerhalb von 1 Sekunde und erfüllt damit die empfohlene Reaktionszeitgrenze der Nielsen Norman Group. <u>(Jakob, 1993)</u>
Anforderungen	Performance - Response Time
Priorität	Mittel
Messung	Automatisierte Performance-Tests (z. B. Lighthouse, Playwright) für Login und Seitenwechsel. Ziel: < 1 s TTI (Time to Interaction).
Testen	Am Ende der Implementierungsphase.
Resultat	

Tab. 29: Beschreibung Non-Functional Requirement 6

3 Erweitertes Gamification-Konzept

Bereits im Vorprojekt wurden erste Erkenntnisse zu Motivation, Hindernissen und Gamification im Vereinskontext erhoben. Diese basierten auf einer Nutzergruppenanalyse mit Nicht-Mitgliedern, Mitgliedern und Vorstand. Die Ergebnisse lieferten eine breite quantitative Datengrundlage, die allgemeine Tendenzen sichtbar machte. Beispielsweise, welche Aktivitäten besonders motivierend wirken oder welche Risiken bei der Einführung von Gamification-Systemen bestehen. (vgl. [Vorprojekt, Kap. 4](#))

Während das Vorprojekt primär die Sicht der Mitglieder sowie die Erfahrungen von Vanessa Alves als Präsidentin von [EGSwiss](#) abbildete, verfolgt die vorliegende Arbeit eine andere Perspektive. Sie untersucht qualitativ, wie Vereinsvorstände Motivation aus organisatorischer Sicht verstehen, welche strukturellen Bedingungen sie fördern oder behindern und wie digitale Systeme, wie der Discord-Bot oder das geplante Admin-Dashboard, sie dabei unterstützen können.

Ziel ist es, die im Vorprojekt identifizierten Motivations-Muster kritisch zu hinterfragen und auf ihre langfristige Umsetzbarkeit im System ClansCore zu prüfen. Dafür wurde die Self-Determination Theory (SDT) als theoretische Grundlage gewählt. Sie beschreibt die psychologischen Bedingungen, die Motivation fördern oder hemmen, und diente als Raster zur Entwicklung und Auswertung der Interviews mit Vereinsvorständen.

Die Erkenntnisse aus diesen Interviews werden im Anschluss mithilfe des Octalysis Frameworks nach Yu-kai Chou in konkrete, technisch umsetzbare Gamification-Mechaniken überführt. Das erweiterte Konzept versteht Gamification somit nicht mehr als isoliertes Punktesystem, sondern als integralen Bestandteil eines digitalen Vereinsökosystems, das psychologische, organisatorische und soziale Faktoren verbindet.

3.1 Self-Determination Theory

Die Self-Determination Theory (SDT) nach Deci und Ryan beschreibt Bedingungen, die Motivation entweder unterstützen oder vermindern können. Im Mittelpunkt der Theorie stehen drei zentrale psychologische Grundbedürfnisse:

- **Autonomie:** Das Bedürfnis, selbstbestimmt handeln zu können und Entscheidungen eigenständig zu treffen.
- **Kompetenz:** Das Bedürfnis, sich fähig zu fühlen und Herausforderungen erfolgreich zu bewältigen.
- **Soziale Eingebundenheit:** Das Bedürfnis, sich mit anderen verbunden zu fühlen und Teil einer Gemeinschaft zu sein.

Werden diese Bedürfnisse erfüllt, erhöht sich die Wahrscheinlichkeit für intrinsisch motiviertes Verhalten. Dadurch können Engagement, Lernprozesse und psychisches Wohlbefinden positiv beeinflusst werden. ([Weir, 2025](#))

Auch im Kontext digitaler Technologien lässt sich die SDT anwenden. Sie geht davon aus, dass Nutzer eine Technologie dann langfristig nutzen, wenn die Interaktion mit dem System zur Befriedigung ihrer psychologischen Grundbedürfnisse beiträgt. Das Interface- und Interaktionsdesign wirkt dabei unmittelbar auf das Erleben von Autonomie und Kompetenz. Eine unklare Bedienung oder mangelndes Feedback kann diese Bedürfnisse frustrieren und zur Ablehnung der Technologie führen. Die soziale Eingebundenheit ist besonders relevant, wenn das System Austausch und Anerkennung zwischen Nutzenden fördert. ([Peters Dorian, 2018](#))

Da die SDT sowohl intrinsische als auch extrinsische Motivation berücksichtigt, bietet sie eine geeignete Grundlage, um zu untersuchen, wie Vereinsmitglieder und Vorstände durch digitale Systeme motiviert werden können. Aufbauend auf diesen theoretischen Annahmen wurden qualitative Interviews mit Vereinsvorständen durchgeführt, um die theoretischen Konzepte mit der Praxis abzugleichen.

Diese drei Bedürfnisse dienten als analytische Kategorien in der Auswertung der Interviews. Aussagen der Befragten wurden entlang dieser Dimensionen kodiert und hinsichtlich ihrer Bedeutung für Vereinsmotivation interpretiert.

3.2 Interviews zur Motivation von Vereinsmitgliedern

Zur Überprüfung der theoretischen Annahmen der Self-Determination Theory (SDT) wurden vier leitfadengestützte Interviews mit Präsidenten verschiedener Schweizer Gaming-Vereine durchgeführt. Ziel war es, zu verstehen, welche Faktoren in der Vereinsrealität Motivation fördern oder behindern, wie Vorstände diese gezielt unterstützen und welche Rolle digitale Systeme dabei spielen. Während das Vorprojekt vorwiegend die Perspektive der Mitglieder beleuchtete (vgl. [Vorprojekt, Kap. 4.2](#)), fokussiert diese Untersuchung die organisatorische Sicht der Vereinsleitung.

Die Auswertung erfolgte qualitativ-interpretativ entlang der drei SDT-Grundbedürfnisse Autonomie, Kompetenz und soziale Eingebundenheit. Sie diente dazu, wiederkehrende Muster und Einflussfaktoren auf die Vereinsmotivation zu identifizieren und im Kontext digitaler Vereinsarbeit zu interpretieren (vgl. Anhang [Abschnitt 15](#)).

Die Interviews zeigen, dass soziale Bindungen, klare Kommunikation und freiwillige Beteiligung die stärksten Treiber langfristiger Motivation darstellen. Freundschaften und gemeinsame Erlebnisse fördern die Verbundenheit, während Zwang oder übermässige Kontrolle als demotivierend empfunden werden. Symbolische Anerkennung (z. B. durch Danksagungen oder sichtbare Rollen) wirkt nachhaltiger als materielle Belohnungen. Fairnessmechanismen wie saisonale Resets werden als motivierend und ausgleichend wahrgenommen, da sie Einstiegshürden für neue Mitglieder senken und Gleichheit fördern.

3.2.1 SDT-basierte Analyse der Interviewergebnisse

Die nachfolgende Tabelle fasst die zentralen Ergebnisse zusammen und ordnet sie den drei Grundbedürfnissen der Self-Determination Theory zu:

SDT-Bedürfnis	Interview-Erkenntnis
Autonomie	Freiwilligkeit, Wahlmöglichkeiten und flexible Beteiligungswege wurden als zentral hervorgehoben. Digitale Werkzeuge sollen unterstützen statt steuern. Transparente, optionale Pfade fördern Selbstbestimmung, während starre Vorgaben sie einschränken.
Kompetenz	Motivation steigt, wenn Fortschritt sichtbar ist (Feedback, Meilensteine, Statistiken) und Beiträge im Vereinskontext wertgeschätzt werden. Ranglisten fördern kurzfristige Aktivität, nachhaltiger sind Aufgaben- und Lernpfade mit klarer Rückmeldung.
Soziale Eingebundenheit (Relatedness)	Gemeinschaft entsteht durch gemeinsame Aktivitäten und sichtbare Anerkennung digitaler Beiträge. Online-Kanäle ergänzen, aber ersetzen keine realen Erlebnisse. Digitale Features sollen Beziehungen sichtbar und anschlussfähig machen (z. B. Team-Achievements, öffentliche Danksagungen).

Tab. 30: Mapping der Interviewergebnisse zu den SDT-Grundbedürfnissen

Insgesamt zeigt sich, dass soziale Eingebundenheit und Kompetenz gleichermassen zentrale Einflussfaktoren für langfristige Motivation darstellen. Autonomie spielt ebenfalls eine bedeutende, aber leicht geringere Rolle und wird insbesondere durch Transparenz, Feedback und flexible Beteiligungsmöglichkeiten gestärkt.

3.2.2 Querschnittliche Implikationen

Aus der Analyse ergeben sich drei zentrale Muster für die Gestaltung motivierender Vereinsstrukturen:

- 1) **Kommunikation als Schlüssel:** Mehrstufige Erinnerungen mit klaren Call-to-Actions erhöhen die Teilnahmebereitschaft.
- 2) **Fairnessmechaniken:** Zeitlich begrenzte Punkte und Resets senken Einstiegshürden und vermeiden dauerhafte Ungleichgewichte.
- 3) **Privatsphäre-wahrende Transparenz:** Aggregierte Kennzahlen fördern Orientierung und Motivation, ohne Privatsphäre zu verletzen.

Diese Erkenntnisse bestätigen die im Vorprojekt gewonnenen Einsichten zur Bedeutung sozialer Erlebnisse und liefern praxisnahe Anhaltspunkte für die Weiterentwicklung des Systems im Sinne einer motivierenden, fairen und nachhaltigen Vereinsführung.

3.3 Octalysis Framework

Das Octalysis Framework nach Yu-kai Chou (2015) ergänzt die psychologische Perspektive der Self-Determination-Theory (SDT) um ein praxisorientiertes Modell zur Analyse und Gestaltung von Motivation in digitalen Systemen. Es identifiziert acht sogenannte Core Drives, welche das Verhalten von Nutzern in Gamification-Umgebungen steuern. (Chou, 2015)

Die acht Core Drives sind:

1. **Epic Meaning & Calling:** das Gefühl, Teil von etwas Grösserem zu sein.
2. **Development & Accomplishment:** Motivation durch Fortschritt und Belohnung.
3. **Empowerment of Creativity & Feedback:** Motivation durch kreative Freiheit und Rückmeldung.
4. **Ownership & Possession:** Motivation durch Besitz oder Kontrolle.
5. **Social Influence & Relatedness:** Motivation durch Gemeinschaft, Anerkennung oder Wettbewerb.
6. **Scarcity & Impatience:** Motivation durch Verknappung oder Limitierung.
7. **Unpredictability & Curiosity:** Motivation durch Neugier und Überraschung.
8. **Loss & Avoidance:** Motivation durch die Vermeidung negativer Konsequenzen.

Während die SDT die Qualität der Motivation (intrinsisch vs. extrinsisch) erklärt, zeigt das Octalysis-Framework konkrete Design-Hebel auf, mit denen diese Motivation im System gefördert werden kann. Für ClansCore wurde das Modell genutzt, um die im Vorprojekt umgesetzten Gamification-Elemente mit einer Bewertungs-Skala von 1 (noch gar nicht umgesetzt) bis 10 (vollumfänglich integriert) einzuschätzen und die Balance zwischen intrinsischen und extrinsischen Anreizen zu bestimmen.

Die Analyse der Vorarbeit zeigt eine klare Schwerpunktsetzung auf Development & Accomplishment (CD2) sowie Social Influence & Relatedness (CD5). Diese Drives erzeugen eine messbare Belohnungsstruktur und fördern sozialen Vergleich innerhalb des Vereins. Intrinsische Anteile sind vorhanden, aber weniger stark systemisch verankert.

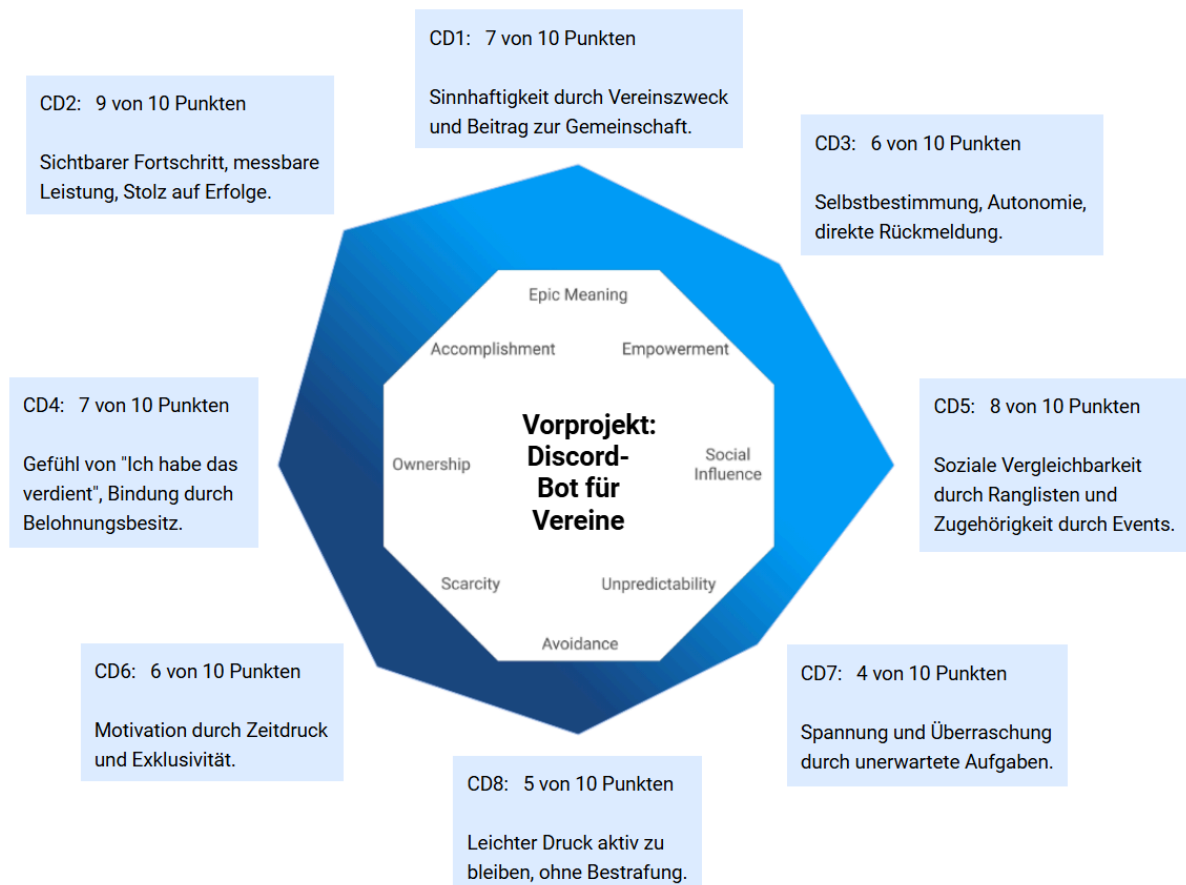


Abb. 1: Die Bewertung des Vorprojektes im Octalysis-Modell

Core Drive		Einschätzung (Stand Vorprojekt)
CD1	Epic Meaning & Calling (7 von 10 Punkten)	Durch Rollen wie Mitglied oder Vorstand sowie den klar kommunizierten Vereinszweck entsteht ein Sinnbezug und Gemeinschaftsgefühl.
CD2	Development & Accomplishment (9 von 10 Punkten)	Das Punktesystem und die Ranglisten (saisonal, jährlich, allzeit) sind stark ausgeprägt. Sie erzeugen sichtbaren Fortschritt und messbaren Erfolge, die Hauptquelle extrinsischer Motivation.
CD3	Empowerment of Creativity & Feedback (6 von 10 Punkten)	Aufgaben können selbstständig übernommen und abgeschlossen werden. Rückmeldungen erfolgen durch Annahme oder Ablehnung seitens des Vorstands. Dies unterstützt Autonomie und Selbstwirksamkeit.
CD4	Ownership & Possession (7 von 10 Punkten)	Punkte und Belohnungen (Merchandise, Rabatte, Gratis-Mitgliedschaft) erzeugen ein Gefühl von Besitz und Zugehörigkeit.
CD5	Social Influence & Relatedness (8 von 10 Punkten)	Öffentliche Ranglisten sowie gemeinsame Events fördern Anerkennung und Verbundenheit in der Community.
CD6	Scarcity & Impatience (6 von 10 Punkten)	Zeitlich begrenzte Saisons und manuell vergebene Belohnungen erzeugen moderate Verknappung und fördern Aktivität innerhalb klarer Zyklen.
CD7	Unpredictability & Curiosity (4 von 10 Punkten)	Neben gelegentlichen, kurzfristig ausgeschriebenen Aufgaben fehlen explizite Zufalls- / Überraschungsmechaniken. Potenzial für Mystery-Belohnungen oder Wochen-Boosts sind vorhanden.
CD8	Loss & Avoidance (5 von 10 Punkten)	Ranglisten-Resets motivieren zur regelmässigen Teilnahme. Die Punkte selbst bleiben bestehen. Der Anreiz entsteht durch periodisch erneuerte Wettbewerbschancen ohne starken Druck.

Tab. 31: Die Bewertung des Vorprojektes im Octalysis-Modell

Das System adressiert insbesondere die extrinsisch motivierenden Core Drives Development & Accomplishment (CD2) und Ownership & Possession (CD4), welche durch das Punktesystem, Ranglisten und Belohnungen stark ausgeprägt sind. In Kombination mit Social Influence & Relatedness (CD5) entsteht eine klare, faire sowie gemeinschaftsorientierte Wettbewerbsdynamik.

Intrinsische Faktoren wie Epic Meaning & Calling (CD1) sind zwar über den Vereinszweck vorhanden, werden im Systemdesign jedoch nur indirekt vermittelt. Empowerment of Creativity & Feedback (CD3) zeigt erste Ansätze durch selbstständige Aufgabenübernahme und Vorstands-Feedback, bleibt aber im Potenzial begrenzt. Explorative und überraschende Elemente (CD7, Curiosity) sowie dynamische Risikofaktoren (CD8, Loss & Avoidance) sind bislang schwächer ausgeprägt.

Insgesamt ist der Discord-Bot in dieser Form primär extrinsisch und kompetenzorientiert motivierend, was Einstieg und Aktivierung stark unterstützt. Für eine nachhaltige, qualitativ hochwertige Motivation im Sinne der SDT sollte die Gewichtung intrinsischer Drives (insbesondere Empowerment, Curiosity, Meaning) erhöht werden, um Selbstbestimmung, Kreativität und persönliche Identifikation mit dem Vereinsleben langfristig zu fördern.

3.4 Erweiterungen, Chancen und Regeln

Auf Basis der Interview-Erkenntnisse (SDT) und der Octalysis-Analyse des Vorprojekts ergeben sich konkrete Erweiterungen, Chancen sowie Gestaltungs- und Fairnessregeln für ClansCore. Ziel ist es, die aktuell dominanten extrinsischen Treiber (CD2, CD4, CD5) um intrinsische Faktoren (CD1, CD3, CD7) zu ergänzen, ohne Fairness, Transparenz und Datenschutz zu gefährden.

3.4.1 Designprinzipien

Aus den theoretischen und empirischen Erkenntnissen wurden zentrale Gestaltungsprinzipien für ClansCore abgeleitet. Sie stellen sicher, dass neue Funktionen Motivation, Fairness und Datenschutz im Sinne der SDT unterstützen.

Prinzip	SDT-Bezug	Octalysis-Hebel
Soziale Verbundenheit vor Wettbewerb	Relatedness: Anerkennung, Zugehörigkeit, Teamgefühl	- CD5: Social Influence - CD1: Meaning
Sichtbarer Fortschritt mit sinnvollem Kontext	Kompetenz: Feedback, Meilensteine, Beiträge „für den Verein“	- CD2: Development - CD4: Ownership
Selbstbestimmte Beteiligung statt Zwang	Autonomie: Wahlfreiheit, optionale Pfade	CD3: Empowerment
Fairness & Datenschutz-by-Design	Alle drei Grundbedürfnisse nicht untergraben	- CD6: Scarcity - CD8: Loss (transparent)

Tab. 32: Leitprinzipien für ClansCore

3.4.2 Geplante Erweiterungen und Regeln

Die folgenden Erweiterungen sind so konzipiert, dass sie Autonomie (SDT) stärken und die Octalysis-Drives CD1/CD3/CD7 gezielt ausbalancieren. Die Wahl der Priorität ergibt sich aus der Wirkung und Umsetzbarkeit.

ID	Erweiterung	Motivationswirkung	Priorität
E1	Mehrstufiges Reminder-System für Events (Save-the-Date, Wochen-, Tages-Reminder, optional Direktnachricht). Optional: Export CSV / ICS im Dashboard (Kalendersubscribe).	- Beseitigt Kommunikations-Engpass - Steigert Teilnahme ohne Zwang (SDT: Kompetenz / Autonomie) - CD2 und CD5	Hoch
E2	Datensparsame Vereins-Analytics im Dashboard (nur Aggregat aktiver Mitglieder, Eventquote, Helferquote und Trendlinien, ohne personenbezogenes Dauertracking).	- Kompetenz der Vorstände (Steuerungsfähigkeit) (SDT: Kompetenz) - Keine Untergrabung der Autonomie / Privacy (SDT: Autonomie) - CD2	Hoch
E3	Punkte haben ein Ablaufdatum (z. B. jährlicher Reset oder schrittweiser Verfall zur neuen Saison).	- Fördert regelmässige Aktivität anstelle von kurzfristigem Punktesammeln - Verhindert „Horten“ und stärkt Fairness zwischen neuen und alten Mitgliedern - Bietet wiederkehrende Chancen auf Erfolg (SDT: Kompetenz / Autonomie) - CD8 in moderater, motivierender Form	Hoch
E4	Anerkennungssysteme in Form von speziellen Hervorhebungen (z. B. „Helfer des Monats“, Rollenfarben, öffentliche Danksagungen).	- Sichtbare Wertschätzung (SDT: Relatedness) - Ownership über sichtbare Symbole - CD1, CD4 und CD5	Mittel
E5	Kooperative Formate wie Team-Quests, Gemeinschaftsziele (z. B. „50 Teilnahmen in 30 Tagen“) mit serverweiter Belohnung (z. B. Punktevergabe).	- Verlagerung von Wettbewerb zu Kooperation - Stärkt Verbundenheit & Sinn (SDT: Relatedness) - CD1, CD3, CD5	Mittel
E5	Event-Recaps im Kanal (kurzer Auto-Post mit Teilnehmerzahl, Dank an Helfer, optional mit Bild).	- Verstärkt Anerkennung und Sinn (SDT: Relatedness) - Erhöht Sichtbarkeit realer Beiträge - CD1, CD2 und CD5	Mittel
E6	Leichte Curiosity-Elemente wie zum Beispiel „Wochen-Boost“ (+20 % für genau eine Aktivität, transparent angekündigt) rotierend per Saison (kein Zufall bei Belohnungen).	- Erhöht Abwechslung ohne Intransparenz - Schützt Fairness - CD6 und CD7	Niedrig

Tab. 33: Erweiterungen und Zielwirkung

Die geplanten Regeln stellen Fairness, Datenschutz und langfristige Motivation sicher. Es übernimmt unter anderem die in der Vorarbeit bereits beschriebenen Regeln (vgl. [Vorprojekt, Kap. 4.4](#)) und formuliert sie mit den neuen Erkenntnissen implementationsnah für Discord-Bot und Dashboard.

Die Regeln lauten wie folgt:

1. **Keine Macht- oder Stimmenvorteile durch Punkte:** Verhindert toxische Hierarchien; erhält demokratische Vereinsprozesse (SDT: Relatedness / Autonomie).
2. **Punkte sind nicht übertragbar:** Verhindert Pay-to-Win / Transferhandel, schützt Fairness und Ownership.
3. **Saisonale Ranglisten-Resets, wobei Punkte erhalten bleiben:** Ergibt niedrige Einstiegshürden für Neumitglieder, da der Wettbewerb periodisch neu geöffnet wird (CD6 und CD8).
4. **Transparente Punktequellen und Limits:** Jede vergütete Aktivität ist dokumentiert. Limits an Aufgaben (täglich / monatlich) vermeiden übertriebenes Sammeln ohne fertigstellen.
5. **Datensparsame Analytics (nur Aggregat):** Steuerung ohne personenbezogenes Dauertracking sowie DSGVO- und Vereinskultur-konform.
6. **Freiwilligkeit:** Gamification ist optional und wird ohne Druck als zusätzliches Vereins-Angebot angesehen (SDT: Autonomie).
7. **Keine Überraschungs-Strafen:** Loss & Avoidance nur über Ranglisten-Resets, keine verdeckten Bestrafungen. Schützt intrinsische Motivation.
8. **Moderierte Curiosity:** Wochen-Boost oder Community-Quests sind angekündigt und fair verteilt. Keine Zufallsbelohnungen mit Ungleichheitseffekt.
9. **Angepasste Ästhetik und Sprache:** Gamification-Elemente sollen die Vereinsidentität widerspiegeln und nicht zu verspielt oder infantil wirken. Dadurch bleibt das System glaubwürdig und akzeptiert.

Bei der Weiterentwicklung von ClansCore ist darauf zu achten, dass Motivation nicht zu stark über Belohnungen gesteuert wird. Um eine Überbetonung extrinsischer Faktoren zu vermeiden, werden kooperative Formate und persönliche Zielpfade gezielt priorisiert. Ungleichheit durch Zufallsmechaniken wird ausgeschlossen, indem Curiosity-Elemente stets angekündigt und fair verteilt bleiben. Datenschutz wird durch strikte Aggregation der Kennzahlen gewährleistet.

3.4.3 Technische Implikationen und Roadmap

Kurzfristig werden die Erweiterungen mehrstufige Reminder-System (E1), ein datensparsames Dashboard mit aggregierten Kennzahlen (E2) und Punkte mit Ablaufdatum (E3) umgesetzt. Diese Massnahmen bilden den Kern der ersten Ausbaustufe und sollen sowohl die Beteiligung als auch die Sichtbarkeit von Engagement verbessern. Anschliessend werden, sofern ausreichend Zeit und Ressourcen vorhanden sind, neue Anerkennungsfunktionen (E4), die kooperativen Formate (E5) sowie automatisierte Event-Recaps (E6) in Angriff genommen. Beide Erweiterungen fördern Selbstbestimmung und Zusammenarbeit und dienen als nächste Entwicklungsstufe zur Stärkung intrinsischer Motivation. Das optionale Feature E7 (Curiosity-Elemente) gilt als Ergänzung, welches nach ersten Praxiserfahrungen schrittweise integriert werden könnte, jedoch keine Priorität besitzt.

3.4.4 Zukünftige Erfolgsmessung

Eine direkte Messung der Wirksamkeit erfolgt im Rahmen dieser Arbeit nicht, kann aber künftig auf Basis anonymisierter Aggregatdaten erfolgen. Als hypothetische Indikatoren eignen sich Kennzahlen wie die Teilnahmequote an Events, der Anteil aktiver Helfer, die Anzahl vergebener Anerkennungen oder die Vielfalt der Aktivitäten pro Mitglied und Saison. Diese Grössen könnten in zukünftigen Auswertungen zeigen, ob sich Beteiligung, Sichtbarkeit und Fairness durch die neuen Mechanismen messbar verbessern. Die Datenerhebung soll dabei ausschliesslich aggregiert und ohne personenbezogene Speicherung erfolgen.

4 Punktesystem und Punkte-Vorschlag

4.1 Erstellung im Dashboard

4.2 Aufgabenerstellung

5 Erweitertes Sicherheitskonzept

Das erweiterte Sicherheitskonzept definiert die konzeptionellen und organisatorischen Massnahmen, mit denen die Vertraulichkeit, Integrität und Verfügbarkeit der in ClansCore verarbeiteten Daten gewährleistet werden. Es baut auf den im Vorprojekt etablierten Grundlagen zu Datensicherheit, Rechteverwaltung und Datenschutz (vgl. Vorprojekt, Kap. 3 Sicherheitskonzept) auf, erweitert diese jedoch in Hinblick auf die neue Systemarchitektur mit einem webbasierten Dashboard und der synchronisierten Datenhaltung zwischen Discord, der Webplattform (Dashboard) und potenziell weiteren Plattformen (UC10, NFR5).

Das Backend, welches bereits im Vorprojekt als eigenständige, modulare Anwendung konzipiert wurde, bildet weiterhin das zentrale Bindeglied zwischen allen Plattformen. Neu hinzugekommen sind die Anforderungen an eine sichere Kommunikation zwischen Backend und Angular-Frontend sowie an eine konsistente und konfliktfreie Synchronisation von Benutzerdaten, Rollen und Gamification-Informationen über mehrere Zugriffspunkte. Die Systemarchitektur ist somit nicht nur funktional, sondern auch sicherheitstechnisch stärker verteilt, was eine Erweiterung des bisherigen Sicherheitsmodells erforderlich macht. (UC3, UC6)

5.1 Ziel und Geltungsbereich

Ziel des erweiterten Sicherheitskonzepts ist es, die personenbezogenen und vereinsinternen Daten vor unbefugtem Zugriff, Manipulation und Verlust zu schützen. Der Fokus liegt auf Vertraulichkeit, Integrität und Verfügbarkeit, welche die drei zentralen Schutzziele der Informationssicherheit bilden (NIST, 2020) (NFR4, NFR5).

Das Konzept erstreckt sich auf alle sicherheitsrelevanten Komponenten des Systems:

- **Backend** (Node.js / Express): zentrale Applikationslogik und API-Kommunikation
- **Discord-Bot** (Discord.js): Interaktionsschnittstelle für Mitgliederaktionen
- **Admin-Dashboard** (Angular): browserbasierte Verwaltungs- und Interaktionsplattform
- **Datenbank** (MongoDB): persistente Speicherung von Benutzer-, Rollen- und Aktivitätsdaten
- **Infrastruktur und Serverumgebung** (Docker, Linux-Host, GitHub Actions): Containerisierung, CI/CD und physische bzw. virtuelle Serverabsicherung
- **Externe Dienste** (Google Calendar API): Synchronisation von Kalendereinträgen

Nicht berücksichtigt werden die nativen Sicherheitsmechanismen externer Plattformen (Discord, Google), da sie ausserhalb der direkten Kontrolle des Systems liegen.

5.2 Sicherheitsprinzipien

Das Sicherheitskonzept orientiert sich an vier grundlegende Prinzipien der Informationssicherheit, welche als Fundament weiterer technischer und organisatorischer Sicherheitsmassnahmen dienen sowie in allen Systemschichten konsistent angewendet werden.

Prinzip	Beschreibung der Umsetzung
Least Privilege	Alle Komponenten und Benutzerrollen besitzen nur jene Rechte, die für ihre Funktion notwendig sind. Vorstandsmitglieder dürfen Daten auf Organisations-Ebene (Mitgliederverwaltung, Events, Punktesystem) bearbeiten, während Mitglieder ausschliesslich ihre eigenen Daten sehen und bearbeiten dürfen. Öffentliche Informationen wie Vereins-Events, Bewerbungsformular, Statuten und Datenschutzrichtlinie sind ohne Login zugänglich. (Bezug: <u>UC3</u>)
Defense in Depth	Sicherheit wird durch mehrere, voneinander unabhängige Schutzebenen gewährleistet, etwa durch Transportverschlüsselung, Container-Isolation, Zugriffsbeschränkungen und rollenbasierte Rechteprüfung. (<u>Murugiah & Karen, 2017</u> ; <u>NIST, 2020</u>)
Accountability	Alle sicherheitsrelevanten Aktionen (Login-Versuche, Rollenänderungen, Punkteänderungen) werden protokolliert und sind für berechnigte Personen nachvollziehbar (Bezug: <u>NFR5</u>).
Privacy by Design	Es werden nur notwendige Daten gespeichert, analog zum im Vorprojekt definierten Datensparsamkeitsprinzip (vgl. <u>Vorprojekt</u> , Kap. 3.3 ff.). Die Datenschutzinformationen sind zusätzlich im Dashboard direkt einsehbar (Bezug: <u>NFR4</u>).

Tab. 34: Beschreibung der vier grundlegenden Prinzipien

5.3 Authentifizierung und Autorisierung

Die Authentifizierung erfolgt über mehrere, voneinander getrennte Ebenen:

- **Discord-OAuth2** für Nutzerinteraktionen innerhalb der Plattform (unverändert aus dem [Vorprojekt](#), vgl. Kap. 3.2.1)
- **Dashboard-Authentifizierung** für den Webzugriff
- **Infrastruktur-Authentifizierung** für administrative Server- und Datenbankzugriffe

Das Backend fungiert dabei als zentrale Autorisierungsinstanz und verwaltet sämtliche Rollen- und Rechteinformationen. Es prüft eingehende Anfragen unabhängig von der Plattform, um konsistente Zugriffsbeschränkungen sicherzustellen.

Die Authentifizierung unterscheidet zwischen menschlichen Benutzer und automatisierten Dienstkonten. Menschliche Zugriffe erfolgen über eine interaktive Anmeldung und werden bei sensiblen Operationen durch eine [Multi-Faktor-Authentifizierung](#) ergänzt. Damit wird insbesondere der Zugriff auf Infrastruktur und Datenbanken gegen unautorisierte Übernahmen geschützt ([UC1](#) , [UC3](#) , [NFR4](#)).

5.3.1 Dienstkonten und automatisierte Zugriffe

Im Gegensatz zu menschlichen Benutzer greifen automatisierte Systemkomponenten (z. B. der Discord-Bot, das Backend oder der CI/CD-Prozess) über sogenannte Dienstkonten auf Systemressourcen zu. Diese Konten sind speziell für maschinelle Abläufe vorgesehen, arbeiten mit sicheren Tokens oder Secrets und unterliegen restriktiven Rechten nach dem Prinzip Least Privilege. Aktionen werden nachvollziehbar protokolliert (Bezug: [NFR5](#)) und Secrets werden sicher in der Pipeline verwaltet.

Bereits im [Vorprojekt](#) war der Discord-Bot als solches Dienstkonto umgesetzt. Er lief in einem eigenen Container, authenticizierte sich über ein Bot-Token aus Umgebungsvariablen (.env) und kommunizierte über die Discord-API mit dem System. Da der Bot umfängliche Berechtigungen benötigt, wird er weiterhin Administratorrechte im Discord-Server erhalten. Diese Ausnahme wird bewusst akzeptiert, da sie für die technische Funktionsfähigkeit des Systems erforderlich ist.

Die Trennung zwischen menschlichen Admins (mit MFA-geschütztem Zugriff) und automatisierten Systemprozessen bleibt dennoch gewahrt. Dienstkonten werden systemweit durch [Role-Based Access Control](#) (nachfolgend RBAC genannt) begrenzt und besitzen nur die für ihren Anwendungsfall notwendigen Berechtigungen. Dadurch wird das Risiko einer Kompromittierung oder eines Missbrauchs erheblich reduziert.

5.3.2 Absicherung des Datenbankzugriffs

Die Datenbank (nachfolgend DB genannt) stellt das sicherheitskritischste Element des Systems dar, da sie sämtliche personenbezogenen und transaktionsbezogenen Daten des Vereins enthält. Entsprechend muss der Zugriff sowohl technisch als auch organisatorisch auf mehreren Ebenen geschützt werden. Die Absicherung umfasst einerseits systeminterne Verbindungen zwischen Backend, Bot und DB, andererseits den administrativen Zugang für menschliche Benutzer mit erweiterten Berechtigungen (Administratoren).

Im Rahmen der Konzeptentwicklung wurden drei Ansätze zur MFA des DB-Zugriffs untersucht. Hintergrund ist, dass MongoDB selbst keinen nativen Zwei-Faktor-Login unterstützt. Der zweite Faktor muss daher vorgelagert, also auf Netzwerkebene oder durch Besitzfaktoren wie Zertifikate, umgesetzt werden.

5.3.2.1 Bewertung und Entscheid

Die Analyse zeigt, dass Netzwerk-MFA den besten Kompromiss zwischen Sicherheit, Wartbarkeit und Implementierbarkeit bietet. Sie schützt den gesamten Zugriffspfad zur Datenbank und kann ohne strukturelle Änderungen an bestehenden Komponenten eingeführt werden. In der bestehenden OST-Serverumgebung (virtuelle Linux-Server mit SSH-Zugang) lässt sich diese Methode schnell, kostengünstig und mit geringem Risiko implementieren. Darüber hinaus ist sie vollständig kompatibel mit der bestehenden Container- und CI/CD-Umgebung (Docker, GitHub Actions).

Die detaillierte Evaluation und der technische Vergleich der einzelnen Varianten (Netzwerk-MFA, X.509-Zertifikate, SSO / OIDC) sind im Anhang dokumentiert. (siehe Anhang [Abschnitt 16.1](#))

5.3.2.2 Übertragbarkeit auf andere Vereine

Das vorgestellte Sicherheitskonzept berücksichtigt, dass ClansCore auch von Vereinen betrieben wird, die ihre Infrastruktur eigenständig hosten. Viele kleinere Organisationen verfügen über keinen dedizierten IT-Betrieb und müssen Sicherheit daher mit möglichst geringen Kosten und administrativem Aufwand gewährleisten. Für solche Vereine stellt das Modell der Netzwerk-MFA eine besonders praktikable Lösung dar. Es erfordert weder teure Identity-Management-Lösungen noch komplexe Zertifikatsverwaltungs-Systeme und kann mit frei verfügbaren Open-Source-Werkzeugen (z. B. OpenVPN¹) umgesetzt werden. Dadurch bleibt das Sicherheitsniveau hoch, ohne dass der Betrieb zusätzliche finanzielle oder organisatorische Hürden verursacht. Diese Skalierbarkeit ist zentral, um den Einsatz von ClansCore auch in kleineren, ehrenamtlich geführten Vereinen langfristig sicherzustellen ([UC10](#) , [NFR4](#)).

5.3.3 Dashboard-Login

Das Admin-Dashboard verfügt über eine eigenständige Authentifizierungsebene, die eine sichere Verwaltung von Vereinsdaten unabhängig von Discord ermöglicht. Die Benutzerverwaltung erfolgt zentral im Backend, welches Authentifizierung und Autorisierung des Web-Frontends übernimmt.

Nach erfolgreicher Anmeldung erhalten Nutzer eine kurzlebige Sitzungsberechtigung, die an ihre Rolle (Admin, Vorstand, Mitglied, Gast) gebunden ist. Rollenänderungen oder sicherheitsrelevante Ereignisse führen zum sofortigen Widerruf aktiver Sitzungen.

Die Berechtigungsprüfung erfolgt ausschliesslich serverseitig über rollenbasierte Zugriffskontrolle ([RBAC](#)). Alle Sitzungen und Aktionen werden im Audit-Log erfasst, um Nachvollziehbarkeit und Integrität sicherzustellen ([NFR5](#)). Für privilegierte Konten (z. B. Vorstand) ist eine optionale MFA vorgesehen ([NFR4](#)). Das Dashboard unterstützt zudem sichere Passwortverwaltung und Wiederherstellung. Technische Details zu den Sicherheitsmechanismen sind im Anhang dokumentiert. (siehe Anhang [Abschnitt 16.2](#))

Diese Umsetzung kombiniert Benutzerfreundlichkeit mit hoher Sicherheit und folgt den Prinzipien Least Privilege, Defense in Depth und Accountability.

5.4 Daten- und Schnittstellensicherheit

Die Kommunikation zwischen den Systemkomponenten erfolgt ausschliesslich über gesicherte Kanäle:

Verbindung	Technologie	Absicherung
Dashboard ↔ Backend	REST-API / HTTPS	TLS 1.3 + Token-basierte Auth (JWT im HttpOnly-Cookie)
Backend ↔ Discord-Bot	WebSocket Event-Bridge	Bot-Token + Signaturprüfung
Backend ↔ MongoDB	TCP / TLS	SCRAM-SHA-256 + RBAC + Netzwerk-MFA
Backend ↔ Google API	OAuth2 Client-Secrets	Zugriffstoken mit Scopes

Tab. 35: Beschreibung der abgesicherten Kanäle in ClansCore

5.4.1 Eingabevalidierung

Alle externen Eingaben (REST, WebSocket, Formular) werden serverseitig validiert und sanitisiert (Injection-Schutz). Fehler werden kontrolliert behandelt und geloggt, ohne sensitive Details preiszugeben (Bezug: [NFR4](#) , [NFR5](#)).

5.4.2 Synchronisierung und Konfliktlösung

Da ClansCore sowohl über den Discord-Bot als auch über das Admin-Dashboard oder anderen potentiellen Plattformen genutzt wird, können Änderungen an denselben Datensätzen parallel erfolgen. Beispielsweise kann ein Vorstandsmitglied ein Mitgliederprofil im Dashboard anpassen, während der Bot gleichzeitig eine Punktebuchung durchführt. Um daraus entstehende Datenkonflikte zu vermeiden, wird im Backend eine optimistische Nebenläufigkeitskontrolle (Optimistic Concurrency Control, OCC) angewendet. Eine detaillierte Beschreibung des Transaktions- und Konfliktlösungsverfahrens findet sich im Anhang. (siehe Anhang [Abschnitt 16.3](#))

¹<https://openvpn.net/>

Dieses Verfahren stellt sicher, dass Datenintegrität und Nachvollziehbarkeit auch bei paralleler Nutzung durch mehrere Plattformen gewährleistet bleiben. Es basiert auf anerkannten Architekturmustern zur Transaktions- und Konsistenzsicherung in verteilten Systemen ([Martin, 2003](#)).

5.5 Datenschutz und DSGVO / DSGVO-Umsetzung

Die im Vorprojekt formulierten Datenschutzrichtlinien (vgl. [Vorprojekt, Kap. 3.3 - 3.5](#)) gelten unverändert.

Neu können Nutzer direkt im Dashboard ihre gespeicherten Informationen einsehen, die Einwilligung zur Datenspeicherung widerrufen oder eine Datenlöschung anfordern. Das Hosting erfolgt auf Servern in der Schweiz ohne US-Rechtsbezug, wodurch die Datenhoheit gewahrt bleibt und der Zugriff durch ausländische Behörden (z. B. über den CLOUD Act) ausgeschlossen ist ([U.S.-Department-of-Justice, 2019](#)). Die Verarbeitung personenbezogener Daten richtet sich nach den Grundsätzen der DSGVO ([Europäische-Union, 2016](#)), welche Privacy by Design and by Default voraussetzen, sowie dem schweizerischen Datenschutzgesetz ([Schweizerische-Eidgenossenschaft, 2020](#)). (Bezug: [NFR4](#))

5.6 Server- und Deployment-Sicherheit

Das Backend wird in einer containerisierten Umgebung betrieben, die nach dem Prinzip Defense in Depth aufgebaut ist. Ziel ist eine mehrschichtige Absicherung, bei der einzelne Schutzebenen unabhängig voneinander wirken. ([NIST, 2020](#))

Weitere technische Details zur Container-Isolation, Namespaces, Capability-Drops und Netzwerksegmentierung sind im Anhang erläutert. (siehe Anhang [Abschnitt 16.4](#))

Durch die Kombination von Container-Isolation, Netzwerksegmentierung und Multi-Faktor-Authentifizierung entsteht eine robuste, mehrschichtige Sicherheitsarchitektur, welche die Vertraulichkeit, Integrität und grundlegende Verfügbarkeit des Systems sicherstellt. Eine umfassende Gewährleistung der Verfügbarkeit würde jedoch zusätzliche Massnahmen erfordern, wie beispielsweise eine Systemverteilung (Distribution), Lastverteilung (Load Balancing) oder automatisierte Failover-Mechanismen. Diese Aspekte gehen über den Rahmen der vorliegenden Arbeit hinaus und werden daher nicht weiter berücksichtigt. (Bezug: [NFR4](#))

5.7 Manipulations- und Integritätsschutz

Wie in der Vorarbeit beschrieben (vgl. [Vorprojekt, Kap. 3.3](#)) werden alle Punkte- und Event-Transaktionen manipulations-sicher in der Datenbank protokolliert. Neu ergänzt sind:

- **Transaktionale Speicherung** mittels atomarer Operationen wie MongoDB Sessions ([MongoDB-Contributors, 2025a](#)).
- **Audit-Log** aller Änderungen, insbesondere von Gamification-Daten, sichtbar für den Vorstand.
- Optionaler **Hash-Check** für Integritätsprüfung einzelner Datensätze.

Damit bleibt jede Änderung rückverfolgbar und das Vertrauen in die Datenbasis langfristig erhalten. (Bezug: [NFR5](#))

5.8 Risikoanalyse und Massnahmen

Die zentrale Sicherheitsrisiken umfassen unautorisierte Zugriffe, Datenlecks, Social-Engineering-Angriffe und Datenverlust. Für jedes Risiko existieren die folgenden präventiven und reaktiven Massnahmen.

Risiko	Auswirkungsgrad	Massnahme
Datenleck durch API	Hoch	HTTPS, Token-Auth, Input-Validation (NFR4 , NFR5)
Unautorisierter DB-Zugriff	Hoch	Netzwerk-MFA + SCRAM-SHA-256 + RBAC (NFR4 , NFR5)
Social Engineering	Mittel	Schulung, Prinzip der minimalen Rechte (UC3)
Datenverlust	Hoch	Regelmässige Backups + Restore-Tests

Tab. 36: Einschätzung der Risiken und deren Massnahmen

5.9 Monitoring und Wartung

Ein konsistentes Logging-Konzept dokumentiert sicherheitsrelevante Ereignisse (Auth-Flows, Rollenänderungen, fehlgeschlagene Zugriffe, Konfliktauflösungen). Für schwerwiegende Vorkommnisse sind Benachrichtigungen an einen Discord-Admin-Kanal und das Admin-Dashboard vorgesehen. Wiederkehrende Sicherheitsüberprüfungen orientieren sich an der OWASP-Top-10-Checkliste ([OWASP-Contributors, 2021](#)) . Konfigurations- und Rechteprüfungen erfolgen regelmässig. (Bezug: [NFR5](#))

Diese Massnahmen schaffen einen kontinuierlichen Überblick über die Systemintegrität und ermöglichen ein rasches Eingreifen bei Sicherheitsvorfällen.

5.10 Abgrenzung des Sicherheitskonzeptes

Das vorliegende Sicherheitskonzept fokussiert auf zentrale sicherheitsrelevante Anforderungen, die im Rahmen dieser Arbeit spezifiziert, implementiert und überprüft werden können. Weitere Massnahmen, wie erweiterte Schutzmechanismen für API-Rate-Limiting, Security-Header, Key-Rotation oder detaillierte Frontend-Härtung, werden als Best Practices betrachtet, jedoch nicht explizit dokumentiert oder umgesetzt, da deren Implementierung stark von der jeweiligen Hosting- und Vereinsumgebung abhängt.

Diese Themen werden im Projekt dann adressiert, wenn sie technisch erforderlich sind oder sich aus zukünftigen Erweiterungen ergeben. Die bewusste Fokussierung auf Authentifizierung, Zugriffskontrolle, Datenintegrität und Datenschutz stellt sicher, dass die sicherheitskritischen Kernanforderungen ([NFR4](#) , [NFR5](#)) erfüllt und überprüfbar bleiben, ohne die Arbeit mit Randaspekten zu überfrachten.

6 Systemanalyse, Architektur und Wireframes

In diesem Kapitel werden die grundlegenden Strukturen und technischen Konzepte des Systems beschrieben. Zunächst wird anhand einer Domänenanalyse modelliert, wie die organisatorischen Abläufe, Rollen und Funktionen eines Vereins digital abgebildet werden können. Darauf aufbauend folgt die Darstellung der Systemarchitektur nach dem C4-Modell. Zum Schluss kommen die Wireframes mit den geplanten Ansichten im Dashboard.

6.1 Domain Analyse

Die Domain-Analyse baut auf dem im Vorprojekt entwickelten Vereins-Bot auf, der bereits zentrale Verwaltungs- und Gamification-Funktionen vereinte. Der Bot unterstützte die Mitgliederaufnahme, Rollenverwaltung sowie die Organisation von Aufgaben und Events. Gleichzeitig förderte er durch ein transparentes Punktesystem Motivation und Engagement im Vereinsalltag. (vgl. [Vorprojekt, Kap. 5.1](#))

Im Zentrum steht weiterhin die Person, die als Mitglied oder Vorstand mit dem System interagiert. Bekannte Entitäten wie Aufgabe, Event, Transaktion, Belohnung und Rangliste bilden den Kern des Vereinsworkflows und ursprünglichen Gamification-Systems. Mitglieder sammeln durch Aktivitäten Punkte, die in Belohnungen eingelöst werden können.

Mit ClansCore wird diese Grundlage zu einem plattformunabhängigen Vereinsmanagement-System weiterentwickelt. Ein neues Account-Konzept ermöglicht die zentrale Verwaltung sämtlicher Benutzer, während Erinnerungsfunktionen für Events und periodische Zahlungen sowie zeitlich begrenzte Punkte saisonale Resets und flexible Belohnungszyklen unterstützen. Somit können Erkenntnisse aus dem erweiterten Gamification-Konzept ([Abschnitt 3](#)) eingeführt werden.

Ein ergänzendes Dashboard bietet konfigurierbare Übersichten zu Rollen, Aktivitäten und Belohnungen sowie eine datensparsame Auswertung von Kennzahlen zur Erfolgsmessung. Diese Analysen erfolgen anonymisiert und dienen der kontinuierlichen Verbesserung von Motivation und Vereinsorganisation.

Insgesamt entwickelt sich ClansCore zu einem integrierten, plattformunabhängigen Vereinsökosystem, das Verwaltung, Gamification und Datenanalyse vereint und die Grundlage für eine transparente und nachhaltige Vereinsführung bildet.

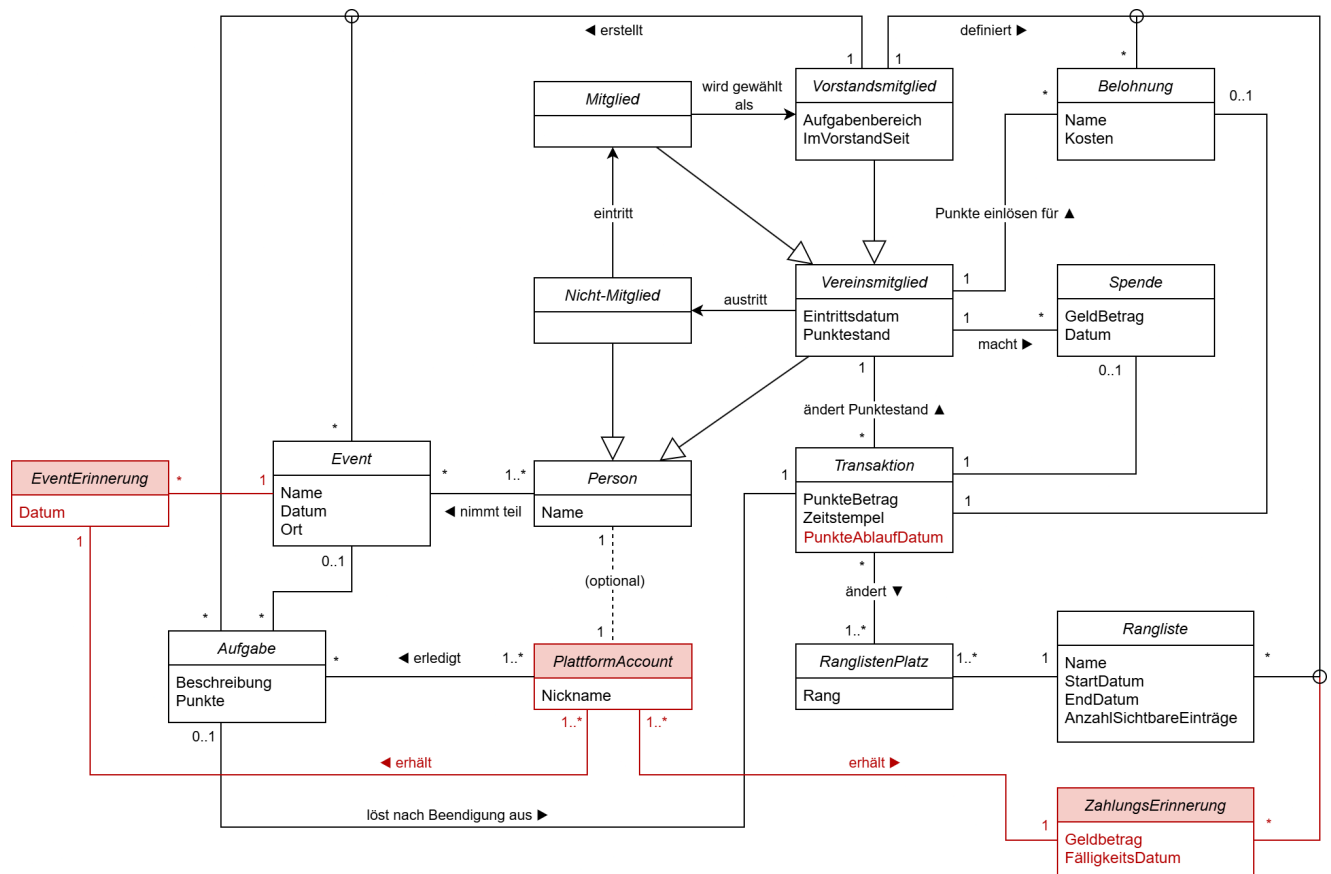


Abb. 2: Domain Model

6.2 C4-Modell

Die Architektur wurde mit dem C4-Modell ausgearbeitet, um eine klare, nachvollziehbare Struktur und eine nachhaltige Erweiterbarkeit zu gewährleisten. Bereits im Vorprojekt wurde der Discord-Bot modular konzipiert, die Entkopplung der Komponenten konnte jedoch noch nicht vollständig umgesetzt werden. Die dafür notwendige Grundstruktur ist im bestehenden Code vorhanden, erfordert aber ein Refactoring, um die Trennung von Verantwortlichkeiten konsequent zu realisieren und zukünftige Erweiterungen zu vereinfachen.

Das nachfolgende Architekturkonzept zeigt den geplanten Zielzustand des Systems. Es wurde so entworfen, dass die Kommunikationskanäle (Discord-Bot und Dashboard) in Zukunft auch durch andere Clients (z. B. Microsoft Teams) ersetzt oder ergänzt werden kann. Ebenso ist der Kalender-Provider abstrahiert, sodass alternative Dienste neben dem derzeit eingesetzten Google Calendar eingebunden werden können.

6.2.1 System Context Diagramm

ClansCore bildet das zentrale System des Vereins und stellt die gemeinsame Basis für alle angeschlossenen Plattformen dar. Über eine einheitliche API können sowohl der Discord-Bot als auch das neu hinzugefügte Web-Dashboard auf dieselbe Logik und Datenbasis zugreifen. Beide Kommunikations- und Verwaltungs-Kanäle interagieren über HTTPS mit der ClansCore-API.

Externe Dienste wie der Google Calendar werden zur Synchronisation von Events eingebunden. Die Architektur ist so ausgelegt, dass weitere Plattformen oder Kalender-Provider künftig ohne strukturelle Änderungen ergänzt werden können.

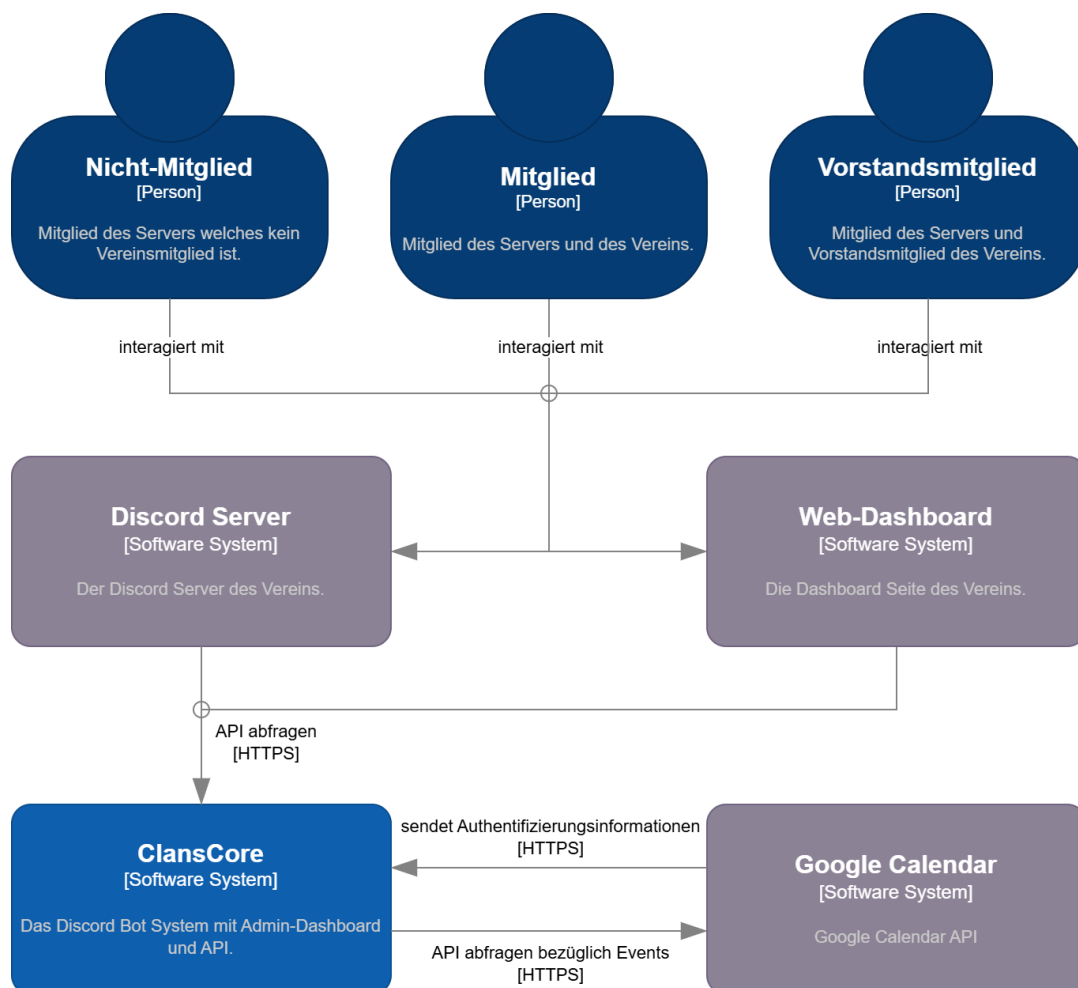


Abb. 3: System Context Diagramm

6.2.2 Container Diagramm

Das ClansCore System besteht aus den zwei zentralen Containern API-Applikation und Datenbank. Die API-Applikation ist in TypeScript auf Basis von Node.js implementiert und dient als Kommunikationsschnittstelle zwischen den Plattformen (Discord und Web-Dashboard) sowie der persistenten Datenhaltung. Sie verwaltet die Geschäftslogik, authentifiziert Anfragen und steuert den Datenaustausch mit externen Diensten wie dem Google Calendar.

Die Datenbank basiert auf MongoDB und speichert alle relevanten Informationen zu Nutzern, Rollen, Punkten, Aufgaben und Events. Über eine klar definierte Repository-Struktur wird eine saubere Trennung zwischen Datenzugriff und Anwendungslogik gewährleistet.

Discord-Server und Web-Dashboard greifen über HTTPS auf die API zu, um Daten zu lesen oder Aktionen auszuführen. Externe Systeme wie der Google Calendar werden ebenfalls über standardisierte API-Endpunkte eingebunden. Dadurch bleibt das System modular, erweiterbar und unabhängig von einzelnen Plattformen oder Providern.

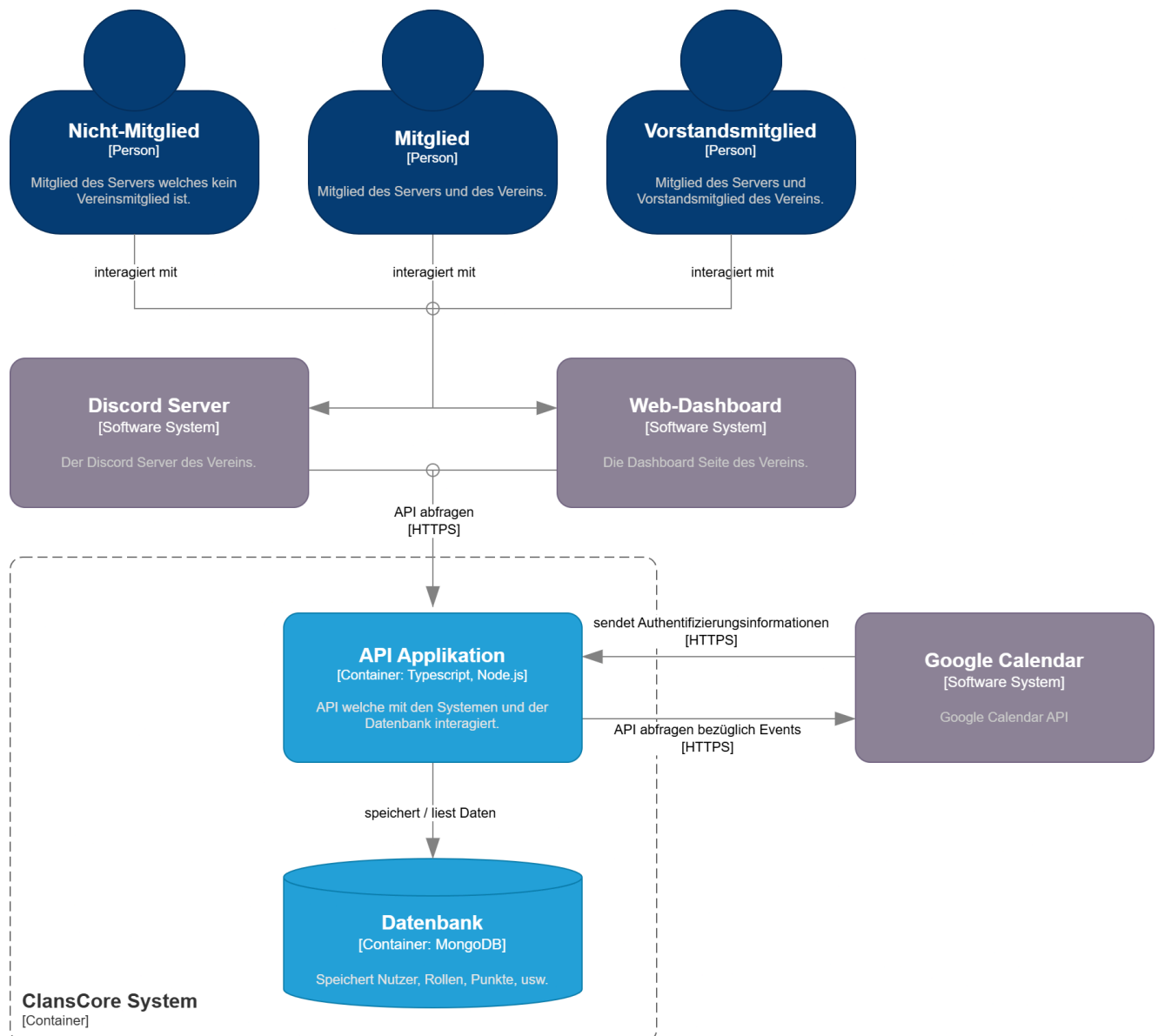


Abb. 4: Container Diagramm

6.2.3 Component Diagramm

Im Vorprojekt waren die Discord-Funktionalitäten direkt in die Backend-Applikation integriert. Der Discord-Client und die Command-Komponente liefen im selben Prozess wie der Webserver und griffen ohne klare Schnittstelle auf die interne Geschäftslogik zu (vgl. [Vorprojekt, Kap. 5.2.5](#)). Diese enge Kopplung vereinfachte die Entwicklung, führte jedoch zu Einschränkungen hinsichtlich Skalierbarkeit, Fehlertoleranz und Wartbarkeit.

In der nun weiterentwickelten Architektur wird dieser Aufbau konsequent entkoppelt. Der Discord-Bot wird als eigenständiger Container ausgelagert, der über eine klar definierte API mit ClansCore kommuniziert. Seine Command-Komponente verarbeitet Discord-spezifische Interaktionen (Slash-Commands, Buttons, Events) und ruft die benötigten Funktionen der ClansCore-API über HTTPS auf. Damit entsteht eine saubere Trennung zwischen der plattformspezifischen Discord-Logik und der zentralen Vereinslogik.

Das Web-Dashboard steht als separates Angular-Frontend ausserhalb der ClansCore-Box und greift über HTTPS auf denselben Webserver zu. Zusätzlich kann es über WebSockets oder Server-Sent-Events (SSE) Echtzeit-Informationen empfangen, etwa zu Punkteständen, Aufgabenfortschritt oder Event-Updates.

Innerhalb der ClansCore-API bildet der Webserver den zentralen Zugangspunkt für Bot und Dashboard. Er verwaltet Authentifizierung, Zugriffskontrolle und leitet Anfragen an die Anwendungslogik der Komponenten User, Event und Gamification weiter. Diese greifen auf persistente Daten über die Database-Komponente zu und werden durch Calendar (für externe Kalender-Synchronisation) und Errors (einheitliches Fehler-Handling) unterstützt.

Mit dieser modularen Struktur wird ClansCore zu einem eigenständigen Vereinsmanagement-System, das sowohl über Discord als auch über das Web-Dashboard genutzt werden kann. Gleichzeitig entsteht durch die Entkopplung eine robuste Grundlage für spätere Skalierung, Service-Isolation und eine langfristig wartbare Architektur.

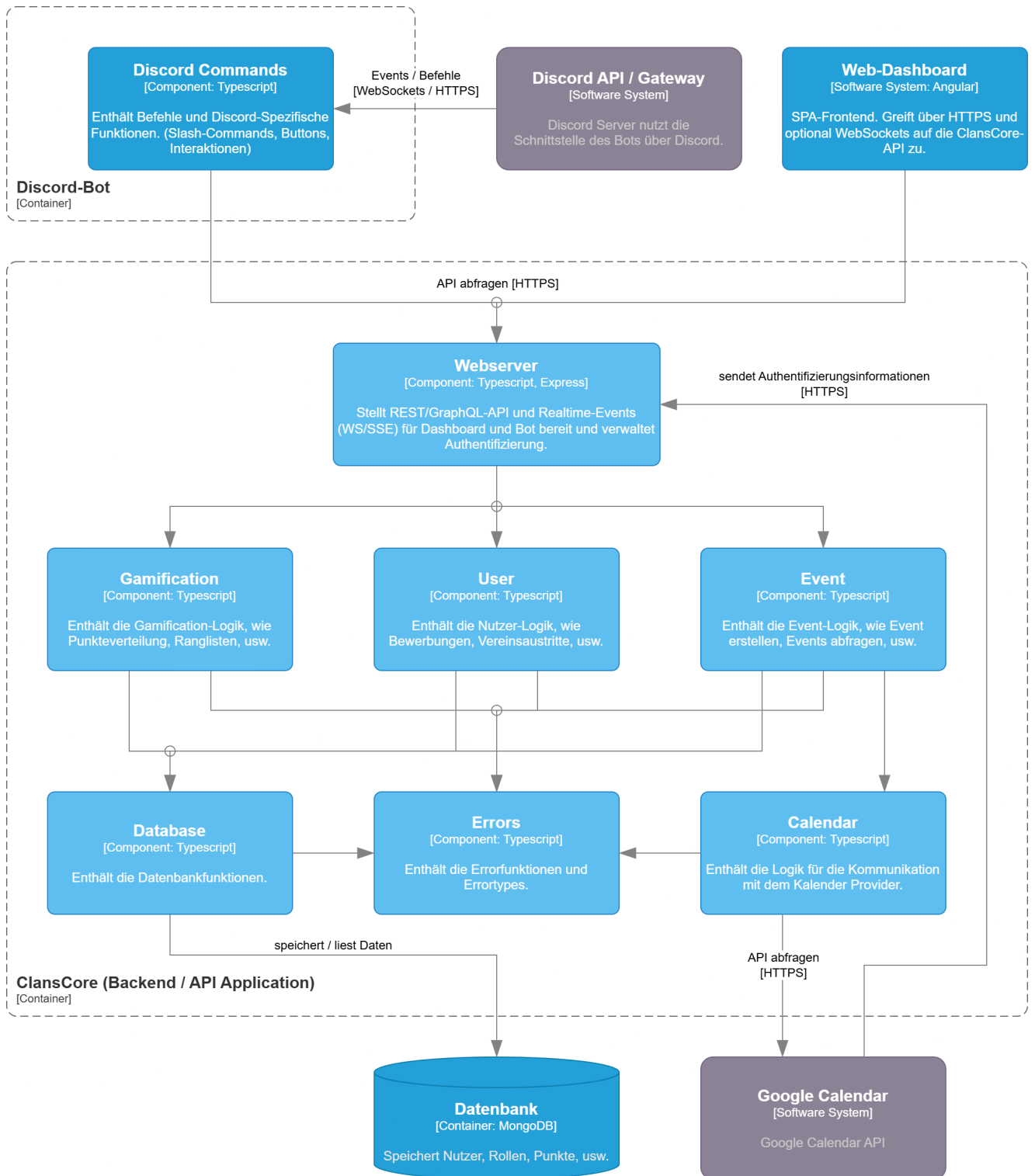


Abb. 5: Container Diagramm

6.3 Wireframes

Zur Skizzierung des Aufbaus und der Struktur der ClansCore-Webseite wurde das Wireframe-Tool Balsamiq verwendet. Die Wireframes wurden unter Bezug auf bestehende Angular-Webseiten entworfen ([madewithangular-Contributors, 2025](#)) und mit Blick auf die Nutzung von Angular-Material-Komponenten konzipiert. Es wurden nicht für alle Seiten Wireframes erstellt, da diese in erster Linie dazu dienen, einen allgemeinen Überblick über die Gestaltung und Funktionalität des Web-Dashboards zu vermitteln.

6.3.1 Nicht-Mitglied

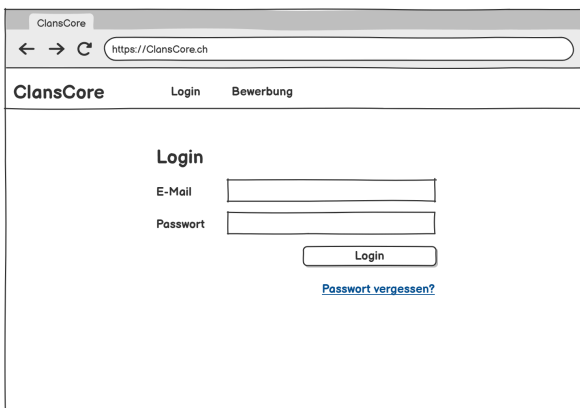


Abb. 6: Login

Login

Um die Kernfunktionen von ClansCore nutzen zu können, ist eine Authentifizierung erforderlich. Diese erfolgt auf der Login-Seite durch die Eingabe der E-Mail-Adresse sowie des zugehörigen Passworts ([UC1](#)). Für den Fall, dass ein Benutzer sein Passwort vergessen hat, besteht über einen entsprechenden Link die Möglichkeit, den Passwort-Zurücksetzungsprozess zu starten und ein neues Passwort anzufordern.

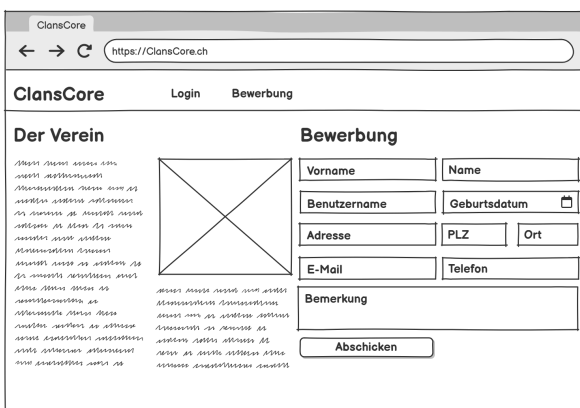


Abb. 7: Bewerbungsformular

Bewerbung

Die Bewerbungsseite ist zweispaltig aufgebaut. Auf der linken Seite werden allgemeine Informationen über den Verein präsentiert, während auf der rechten Seite ein Formular zur Einreichung einer Beitrittsbewerbung zur Verfügung steht. ([UC2](#))

Mitglied

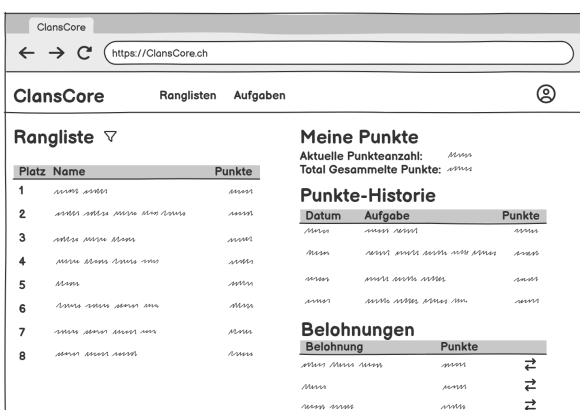


Abb. 8: Ranglisten-Seite

Ranglisten

Auf der Ranglisten-Seite befindet sich auf der linken Seite eine filterbare Rangliste (Saisonal/Gesamt), die einen Überblick über die Platzierungen der Nutzer bietet. Auf der rechten Seite werden die eigenen Punkte, deren Historie und aktuelle Belohnungen angezeigt. ([UC6](#))

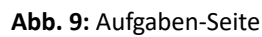


Abb. 10: Mitglieder-Seite

Zugriff auf die Mitglieder-Seite haben ausschliesslich Vorstandsmitglieder. Es können bestehende Mitglieder gefiltert (Name, Vorname, Benutzername), bearbeitet, hinzugefügt oder gelöscht werden. (UC3)

6.3.2 Gamification-Verwaltung

In der Gamification-Verwaltung werden alle Parameter welche für die Punktberechnung von Aufgaben benötigt werden erfasst. Da die Gamification-Verwaltung ein bestehendes Excel-Tool aus dem Vorprojekt (vgl. [Vorprojekt, Kap. 4.8](#)) ablöst, wurde das Design gezielt an dessen Layout und Bedienlogik angelehnt. Dadurch soll der Übergang zur neuen Anwendung für die Benutzer erleichtert werden.

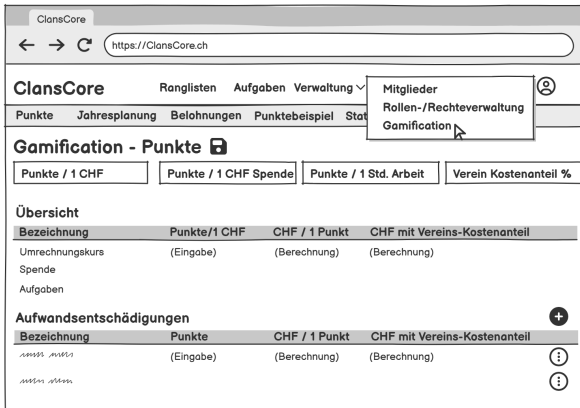
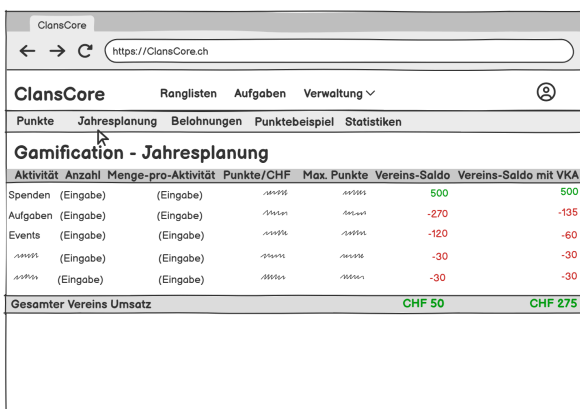


Abb. 11: Punktedefinitions-Seite

Punktedefinition

Auf der Punkte-Seite werden die zentralen Parameter des Gamification-Systems vom Vorstand definiert. Dazu gehören unter anderem der monetäre Wert eines Punktes, die Punktbewertung einer Arbeitsstunde sowie der Punktwert einer Spende. Zusätzlich kann der Kostenanteil des Vereins festgelegt werden. Darüber hinaus besteht die Möglichkeit, neue Aufwandsentschädigungen anzulegen, bestehende zu bearbeiten oder zu löschen. ([UC4](#))



Aktivität	Anzahl	Menge-pro-Aktivität	Punkte/CHF	Max. Punkte	Vereins-Saldo	Vereins-Saldo mit VKA
Spenden (Eingabe)	(Eingabe)				500	500
Aufgaben (Eingabe)	(Eingabe)				-270	-135
Events (Eingabe)	(Eingabe)				-120	-60
Aufwandsentschädigungen (Eingabe)	(Eingabe)				-30	-30
Aufwandsentschädigungen (Eingabe)	(Eingabe)				-30	-30
Gesamter Vereins Umsatz					CHF 50	CHF 275

Abb. 12: Jahresplanung-Seite

Jahresplanung

Auf der Jahresplanung-Seite können die jährlichen Kosten für den Verein simuliert werden. Zur Berechnung werden Parameter von der Punkte-Seite benötigt. Dadurch lässt sich abschätzen, wie sich unterschiedliche Szenarien auf den jährlichen Gesamtumsatz des Vereins auswirken. ([UC4](#))

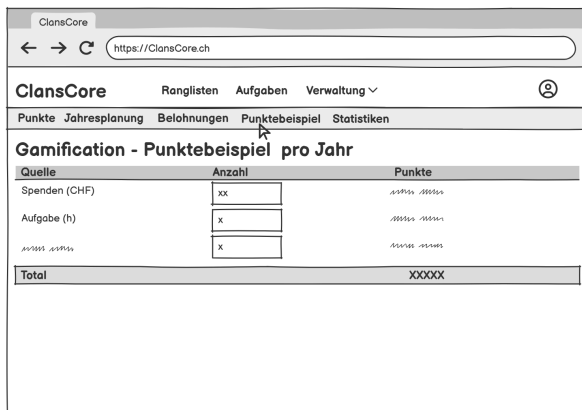


Belohnung	Realwert (CHF)	Ziel-Kostenanteil (%)	Ziel-Kostenanteil (CHF)	Punktwert	Aufwand (h)

Abb. 13: Belohnung-Seite

Belohnungen

Es können Belohnungen bearbeitet, hinzugefügt oder gelöscht werden. Zur Berechnung des Punktwertes und dem Aufwand werden Parameter von der Punkte-Seite benötigt. ([UC4](#))



Quelle	Anzahl	Punkte
Spenden (CHF)	xx	10000
Aufgabe (h)	x	10000
Anzahl	x	10000
Total		XXXXX

Abb. 14: Punktebeispiel-Seite

Punktebeispiel

Auf der Punktebeispiel-Seite kann die Punktevergabe für ein Mitglied simuliert werden. Dabei werden verschiedene Aktivitäten, wie Spenden, Event-Teilnahmen oder Aufgaben, berücksichtigt, um die resultierende Gesamtpunktzahl zu berechnen und deren Auswirkungen im Punktesystem zu veranschaulichen. ([UC4](#))

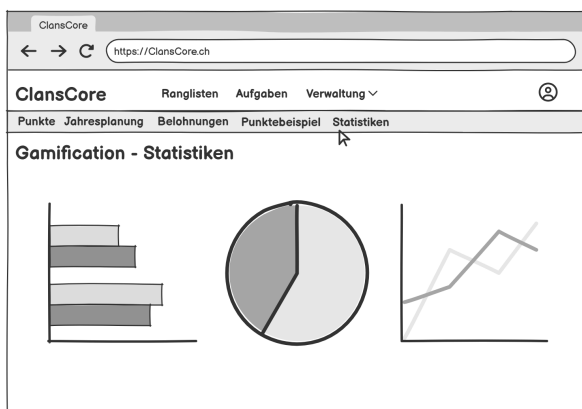


Abb. 15: Statistiken-Seite

Statistiken

Auf der Statistik-Seite können Auswertungen zu Events und Mitgliedern angezeigt werden. Diese bieten einen Überblick über relevante Kennzahlen und zukünftige Trends. Diese unterstützen den Vorstand bei der Analyse der Vereinsaktivitäten und dient als Planungshilfe.

7 Qualitätssicherung

Die Qualitätssicherung spielt eine entscheidende Rolle um eine stabile, sichere und fehlerfreie Anwendung bereitzustellen. In diesem Kapitel werden die Teststrategien beschrieben, sowie erläutert wie die Code-Qualität sichergestellt werden kann. Ziel ist es durch Tests und Überprüfungen eine hohe Softwarequalität zu gewährleisten.

7.1 Testkonzept

Das Testkonzept beschreibt die Vorgehensweise zur Sicherstellung der Software-Qualität für ClansCore. Es legt fest, welche Tests durchgeführt werden, wie diese organisiert und dokumentiert werden und welche Tools und Umgebungen dabei zum Einsatz kommen. Ziel ist es, die Einhaltung der Use-Cases und nicht-funktionalen Anforderungen sicherzustellen und eine fehlerfreie Anwendung zu gewährleisten.

7.1.1 Vorgehensweise

Um den Erfolg bei dem Testen sicherzustellen ist es wichtig strukturiert vorzugehen. Die Vorgehensweise lautet wie folgt:

1. Mittels der Requirements Tests definieren.
2. Entscheiden wie die Tests durchgeführt werden.
3. Tests durchführen.
4. Fehler und Erfolge dokumentieren.
5. Aufgefallene Fehler, die während den Tests auftauchen, beheben.
6. Tests erneut durchführen.

7.1.2 Testziele

Folgende Funktionen und Elemente sollen getestet werden:

- Alle Use Cases
- Alle Non-Functional Requirements
 - Usability
 - Maintainability
 - Security
 - Performance

7.1.3 Testumgebung

Noch nicht definiert.

7.1.4 Unit- und Integrationstests

Die Unit- und Integrationstests des Admin-Dashboards werden mit dem Jasmine-Testing-Framework implementiert, das standardmässig in Angular-Projekten enthalten ist. Die Ausführung der Tests erfolgt über den Karma Test Runner, welcher als Laufzeitumgebung für automatisierte Tests dient und eine Integration in den Entwicklungsprozess ermöglicht. ([Angular-Contributors, 2025a](#))

7.1.5 Usability-Tests

Um die Benutzerfreundlichkeit und Funktionalität des Admin-Dashboard zu prüfen, werden Usability-Tests mit mindestens Drei Testpersonen durchgeführt. Die Tester werden aufgefordert, verschiedene Funktionen und Abläufe durchzuführen. Die Tests werden mithilfe eines Formulars durchgeführt, welches den Testern die Aufgaben beschreibt. Diese können sie nach der Durchführung bewerten. Bei der Bewertung wird die Klarheit der Durchführung sowie auch der Nutzen der Funktion bewertet.

7.2 Ablaufplan

Der Ablaufplan beschreibt die verschiedenen Testphasen und deren Durchführung. Die Ergebnisse der Tests liefern wertvolle Informationen zur Stabilität, Funktionalität und Benutzerfreundlichkeit des Systems.

ID	T1
Name	Unit-Tests
Beschreibung	Funktionen im Backend und die Logik einzelner Angular-Komponenten werden mit Unit-Tests getestet. Die Tests werden nach jedem Commit automatisch durch GitHub Actions durchgeführt.
Methode	Automatisch
Relevant für	Use-Cases, Functional Requirements
Testumgebung	
Intervall	Bei jedem Commit

Tab. 37: Ablauf Unit-Tests

ID	T2
Name	Integration-Tests
Beschreibung	Bei den Integration-Tests wird überprüft, ob die Verbindungen der verschiedenen Komponenten (Angular, Backend, Datenbank) richtig funktionieren. Die Tests werden nach jedem Commit automatisch durch GitHub Actions durchgeführt.
Methode	Automatisch
Relevant für	Use-Cases, Functional Requirements
Testumgebung	
Intervall	Bei jedem Commit

Tab. 38: Ablauf Integration-Tests

ID	T3
Name	Usability-Tests
Beschreibung	Das Admin-Dashboard wird, wie im Abschnitt <u>Usability-Test</u> genauer beschrieben, von mehreren Testern getestet.
Methode	Manuell
Relevant für	Usability, Security, Requirements, Reliability
Testumgebung	
Intervall	Gegen Ende der Konstruktionsphase

Tab. 39: Ablauf Usability-Test

ID	T4
Name	System-Tests
Beschreibung	Am Ende wird für das gesamte System überprüft, ob alle <u>Use Cases</u> und <u>Non-Functional Requirements</u> erfolgreich umgesetzt worden sind.
Methode	Manuell
Relevant für	Alle Testziele
Testumgebung	
Intervall	Am Ende

Tab. 40: Ablauf System-Tests

7.3 Code-Qualität

Um die Code-Qualität sicherzustellen wird das Vier-Augen-Prinzip angewendet. Bei jedem Meeting wird der geschriebene Code von den Teammitgliedern überprüft bevor er in den Main-Branch gemerged wird. Weitere Informationen zu Code-Qualität und GitHub können im [Abschnitt 12](#) gefunden werden.

Um die Einheitlichkeit des Codes sicherzustellen wird ein Linter verwendet. Mehr Informationen dazu können im [Code Setup](#) gefunden werden.

7.4 Definition of Done

Damit die Use-Cases als abgeschlossen gelten, müssen folgende Kriterien erfüllt sein:

- Die im Use Case beschriebenen Funktionen wurden vollständig umgesetzt und verhalten sich wie spezifiziert.
- Für den jeweiligen Anwendungsfall wurden geeignete Tests implementiert und erfolgreich ausgeführt.
- Die Testergebnisse sind vollständig dokumentiert
- Die [Code Guidelines](#) wurden eingehalten.

8 Implementation

8.1 Datenbank

Die Datenbank von ClansCore basiert auf einer im Rahmen des Vorprojektes entwickelten Datenbank, die für das aktuelle Projekt erweitert und angepasst wurde, um die neuen funktionalen und technischen Anforderungen zu erfüllen (vgl. [Vorprojekt, Kap. 7.1](#)).

8.1.1 Anpassungen der Entitäten

Bei der Entität Person wurde das Attribut „WebPW“ hinzugefügt, um die Authentifizierung auf dem Dashboard zu ermöglichen (vgl. [UC1](#)). Das Passwort wird gehasht und gesalzen gespeichert. Die Entität Person ist mit der neu eingeführten Entität BeitragsZahlung verknüpft, welche den Zeitpunkt speichert, wann eine Person ihren Mitgliederbeitrag bezahlt hat. Dieser wird in der Entität MitgliederBeitrag verwaltet. Zudem besteht eine Verbindung zur Entität Erinnerung, in der festgelegt wird, an welchem Datum und für welche Events oder Mitgliederbeiträge eine Erinnerung versendet werden soll (vgl. [UC8](#)). Bei der Entität Person wurde das Attribut WebPW hinzugefügt, um die Authentifizierung auf dem Dashboard zu ermöglichen (vgl. [UC1](#)). Das Passwort wird gehasht und gesalzen gespeichert. Die Entität Person ist mit der neu eingeführten Entität BeitragsZahlung verknüpft, welche speichert, wann eine Person ihren Mitgliederbeitrag bezahlt hat. Der jeweilige Mitgliederbeitrag wird in der Entität MitgliederBeitrag verwaltet. Zudem besteht eine Verbindung zur Entität Erinnerung, in der festgelegt wird, an welchem Datum und für welche Events oder Mitgliederbeiträge eine Erinnerung versendet werden soll (vgl. [UC8](#), [UC9](#)).

Ein zentraler Bestandteil des Gamification-Systems sind die in der Entität GamificationParameter definierten Attribute „PunkteProCHF“ und „PunkteProSpende“. Diese dienen als Grundlage für die Punkteberechnung und Punkteplanung. In der Entität Transaktion wurden die neuen Attribute „BetragVerbraucht“ und „PunkteLaufenabAm“ ergänzt, um die [geplante Gamification-Erweiterung E4](#) zu realisieren.

Die Entität Aufgabentyp, welche mit der Entität Aufgabe verbunden ist, beschreibt die verschiedenen Aufgabentypen und dient der Punkteberechnung (vgl. [UC4](#), [UC5](#)). Zur Differenzierung der Art der Punktevergabe verfügt die Entität Aufgabentyp über das Attribut „Entschädigung“. Dieses ist als Enumeration modelliert und kennzeichnet, ob eine Entschädigung stundenbasiert oder einmalig vergeben wird. Die Entität Jahresplanung steht in Beziehung zur Entität Aufgabentyp und dient der Unterstützung der vereinsinternen Jahresplanung und Kontrollpunkt des Punktesystem. Dabei wird definiert, welche Aufgabentypen in welchem Umfang und mit welcher Anzahl an Teilnehmenden geplant sind.



Da das Zielsystem aus mehreren logisch getrennten Anwendungen bestehen sollte (Backend / API, Discord-Bot, Admin-Dashboard, Shared-Package), war ein tiefgreifendes Refactoring notwendig. In diesem Kapitel wird die Ausgangslage des Vorprojekts beschrieben, die architektonischen Schwächen analysiert und die daraus motivierte Neugestaltung erläutert.

8.2.1 Analyse der Ausgangsarchitektur im Vorprojekt

Die Architektur des Vorprojekts befand sich in einer Übergangsphase. Es existierte eine Grundstruktur nach fachlichen Domänen (user, gamification, events) sowie erste technische Layer, jedoch fehlten zentrale Eigenschaften einer vollständig modularen Softwarearchitektur.

Merkmal	Beschreibung
Teilweise Schichtenarchitektur	Die Ordnerstruktur unterschied zwischen application/, infrastructure/, models/ und platform/discord/. Die Trennung war jedoch nicht durchgängig umgesetzt: • Commands griffen direkt auf Application-Models zu • Application-Models fungierten hauptsächlich als Wrapper um Datenbankservices • Präsentations- und Geschäftslogik waren nicht klar getrennt
Indirekte, aber enge Datenbankkopplung	Commands nutzten nicht direkt Mongoose, jedoch Application-Models wie UserModel oder GamificationModel. Diese stellten: • lediglich leichte Abstraktionen dar • keine eigenständige Businesslogik bereit • eine enge Kopplung an die Datenbankstruktur her Eine dedizierte Businesslogik-Schicht war nicht vorhanden.
Keine Wiederverwendbarkeit über Projektgrenzen	Obwohl Typen, Modelle und Validierungen vorhanden waren, fehlte eine Shared-Library zur gemeinsamen Nutzung in mehreren Applikationen (API, Bot, Dashboard). Dies führte zu: • mehrfach definierten Typen • fehlenden DTO-Strukturen • uneinheitlicher Validierungslogik • redundanten Fehlertypen
Fehlende API-Schicht	Der Bot kommunizierte nicht über HTTP mit dem Backend, sondern griff direkt über Application-Models und DB-Services auf Daten zu. Der einzige vorhandene Endpunkt diente dem Google-OAuth-Callback für die Kalenderanbindung. Dadurch fehlten: • eine Sicherheits- und Validierungsschicht • eine klare Trennung zwischen Bot und Backend • technische Voraussetzungen für ein Admin-Dashboard
Unvollständige Domänenabgrenzung	Obwohl Domänenordner existierten, waren diese nicht isoliert: • Commands importierten mehrere Domänen direkt • keine Aggregates im Sinne von Domain-Driven Design • keine Bounded Contexts • Geschäftsregeln waren auf Commands und DB-Services verteilt

Tab. 41: Beschreibung der architektonischen Merkmale des Vorprojekts

8.2.2 Neue Zielarchitektur (ClansCore)

Die neue Architektur folgt Domain-Driven Design (DDD) und Modularitätsprinzipien. Sie besteht aus vier vollständig getrennten, aber klar integriert arbeitenden Komponenten:

```
apps/
  clanscore-api/    > Zentrale Backend-API
  dashboard/        > Admin-Dashboard (Angular)
```

discord-bot/ > Entkoppelter Discord-Bot (API-Client)
 shared/ > DTOs, Validierungen, Fehlertypen, Utilities

Diese Struktur ermöglicht erstmals eine saubere Trennung aller fachlichen und technischen Verantwortlichkeiten.

Änderungs-Schritt	Nutzen
Entkopplung des Discord-Bots	Der Discord-Bot kommuniziert nun ausschliesslich über die Backend-API: - keine direkten Datenbankzugriffe - Commands enthalten keine Geschäftslogik - die Kommunikation erfolgt über einen dedizierten API-Client - gemeinsame DTOs gewährleisten Typensicherheit Die Entkopplung ermöglicht separate Deployments und verbessert Sicherheit, Validierung und Wartbarkeit erheblich.
Einführung einer vollständigen Schichtenarchitektur	Das Backend wurde in klar getrennte Schichten überführt: Domain Layer: Geschäftsobjekte und Regeln Application Layer: Orchestrierung von Use Cases Infrastructure Layer: Datenbankzugriffe, externe Dienste Presentation Layer: REST-Endpunkte, Routing, Validierung Diese Struktur schafft klare Verantwortlichkeiten und erlaubt testbare, erweiterbare Use Cases.
API-first Ansatz	Für alle Funktionen existieren nun REST-Endpunkte: - Mitgliederverwaltung - Rollenverwaltung - Events - Kalender - Gamification (Tasks, Rewards, Leaderboards) Dies ermöglicht: - vollständige Dashboard-Anbindung - stabile Bot-Integration - zentrale Validierung, Authentifizierung und Fehlerbehandlung
Einführung der Shared-Library	Die Shared-Library fasst sämtliche gemeinsam genutzten Strukturen zusammen: - DTOs - Validierungen - Fehlerdefinitionen - Domänenkonstanten - Utility-Funktionen Dadurch werden konsistente Typen und Schnittstellen für alle Apps bereitgestellt.

Tab. 42: Beschreibung der zentralen Änderungen der neuen Architektur

8.2.3 Umsetzung und Ergebnis des Refactorings

Die technische Umsetzung des Refactorings erfolgte in mehreren aufeinander abgestimmten Schritten. Zunächst wurde die alte Bot-Architektur vollständig dekonstruiert, indem direkte Modellzugriffe entfernt, eng gekoppelte Commands aufgelöst und über 25 Kommandodateien auf eine ausschliessliche Nutzung der neuen API migriert wurden. Parallel dazu wurde die gesamte API-Schicht neu aufgebaut. Es entstanden mehr als sechs dedizierte Controller, eine Validierungs- und Fehler-Middleware sowie eine konsistente REST-Struktur, die alle Domänen einheitlich abbildet.

Darauf aufbauend wurde der Application Layer neu gestaltet und die Geschäftslogik in klar definierte Use Cases überführt. Gleichzeitig wurden über fünfzehn Datenbank-Mappers für zentrale Entitäten wie Events, Tasks, Rewards, Leaderboards, Personen und Rollen entwickelt, um Datenflüsse sauber zwischen Datenbank-, Domain- und DTO-Ebene zu trennen. Auch die DB-Service-Schicht wurde umfassend überarbeitet. exemplarisch umfasst der neugestaltete `user.db.service.ts` inzwischen mehr als 470 Zeilen und kapselt sämtliche Benutzerinteraktionen zentral. Die Domain-Modelle wurden erweitert, unter anderem um `GamificationParameter`, `Reminder`, `MembershipFee`, wodurch eine modularere Fachlogik und eine klarere Domänenstruktur erreicht wurden.

Der Discord-Bot wurde daraufhin vollständig neu implementiert und kommuniziert nun ausschliesslich über den generierten API-Client mit dem Backend, wodurch Präsentationslogik und Geschäftslogik klar getrennt sind. Abschliessend wurde mit dem Admin-Dashboard erstmals eine eigenständige Webapplikation geschaffen, die sämtliche relevanten Backend-Funktionalitäten konsumiert und vollständig in die neue Architektur integriert ist.

Im Ergebnis entstand eine saubere, modulare Gesamtarchitektur mit klar abgegrenzten Domänen, einer vollständig entkoppelten Struktur zwischen Bot, Dashboard und Backend sowie einer API-first Plattform. Die neu gestalteten Use Cases sind testbar und erweiterbar, ein gemeinsames typisiertes Datenmodell stellt konsistente Datenflüsse sicher und die Systembasis bildet eine robuste Grundlage für Gamification, Rollen-Synchronisation, Event-Management sowie zukünftige Erweiterungen. Trotz des beträchtlichen Aufwands stellt die neue Architektur eine langfristig wartbare und skalierbare Plattform dar, die den Anforderungen der geplanten Weiterentwicklung entspricht.

8.3 Erstellung Dashboard

8.4 Erweiterung Discord-Bot

8.5 Limitierungen

8.5.1 Gamification-Erweiterungen

8.5.2 Rollenbenennungen

8.5.3 Logging

8.5.4 Event-Synchronisierung

9 Fazit

9.1 Resultate

9.2 Schlussfolgerung

Kapitel II

Projekt Dokumentation

10 Projekt Plan

10.1 Zusammenarbeit

Als Projektmanagement-Methode wird Kanban verwendet welches es erlaubt das Projekt agil durchzuführen. Aufgrund der Teamgrösse von zwei Personen wurde sich gegen Scrum entschieden. Der zusätzliche organisatorische Mehraufwand durch Daily Meetings, Sprint Reviews und weitere Scrum Meetings würde keinen grossen Nutzen bringen. Dank der geringen Teamgrösse ist es kein Problem den Überblick über das Projekt zu behalten, sodass eine schlanke Methode wie Kanban eine passende Wahl ist.

Die Zusammenarbeit und Aufgabenaufteilung wird mittels Jira organisiert.

10.2 Meetings

Jede Woche findet ein Meeting unter den Teammitgliedern und ein Meeting mit dem Referenten statt.

Im Team-Meeting wird besprochen, welche Aufgaben fertiggestellt wurden und diese gegebenenfalls zusammen anschaut. Zudem werden die Aufgaben für kommende Woche verteilt.

Beim Meeting mit dem Referenten wird der aktuelle Stand besprochen. Ausserdem kann dieses genutzt werden, um allfällige Fragen zu stellen.

10.3 Minimum Viable Product (MVP)

Das Minimum Viable Product (MVP) beinhaltet die Kernfunktionen des Projektes. Diese sind folgendermassen definiert:

- **Erweiterung Discord-Bot:** Ziel ist es, den bestehenden Discord-Bot basierend auf dem während des Einführungsprozesses für den Verein EGSwiss erhaltenen Feedback gezielt weiterzuentwickeln. Dabei sollen Funktionen neu erstellt, ergänzt und verbessert werden.
- **Erweiterung Gamification:** Ziel ist es, das bestehende Gamification-System auszubauen und zu untersuchen, welche Erweiterungen die Motivation und das Engagement im Verein steigern können.
- **Plattform-Unabhängigkeit:** Ein Admin-Dashboard soll erstellt werden, welches dem Vorstand Anpassungen an bereits erfassten Daten ermöglicht, ohne direkten Zugriff auf die Datenbank. Das Admin-Dashboard soll unabhängig von Discord nutzbar sein und somit die Applikation plattformunabhängig machen.

10.4 Long-Term Plan (Version 1.0 - Ursprüngliche Planung)

Der ursprüngliche Long-Term Plan wurde in 5 verschiedenen Phasen aufgeteilt. Diese werden in der folgenden Tabelle genauer beschrieben. Optionale Aufgaben stehen in Klammern.

Achtung:

Im Verlauf des Projektes entstand eine neue Version des Long-Term-Plans. (Siehe [Abschnitt 10.6](#))

Phase	Woche	Datum	Ziele	Meilenstein
Inception	1	15.09 - 21.09.	Einrichtung, MVP definieren	
	2	22.09. - 28.09.	Projektplan, Risikomanagement, Long-Term-Plan	M1
Elaboration	3	29.09. - 05.10.	Requirements, Einleitung, Stand der Technik	
	4	06.10. - 12.10.	Interviews andere Vereine, Sicherheitskonzept	
	5	13.10. - 19.10.	Gamificationkonzept, Architektur, Wireframes	
	6	20.10. - 26.10.	Testkonzept, Prototyp Admin-Dashboard (Proof of Concept)	M2
Construction (Version 1.0)	7	27.10. - 02.11.	2.1, 2.2, 2.3, 7.1	
	8	03.11. - 09.11.	Refactoring, 1.1, (1.2)	
	9	10.11. - 16.11.	3.1, 3.2, 4.1, 4.3	
	10	17.11. - 23.11.	4.2, 4.4, 4.5, (5.1), (5.2)	M3
	11	24.11. - 30.11.	3.3, 4.6, 6.1, 6.2, 6.3, (6.4), Reminder-System, Vereins-Analytics	
	12	01.12. - 07.12.	7.2, 7.3, 7.4, (7.5), (8.1), (8.2), (8.3), Implementation beenden	M4
Transition	13	08.12. - 14.12.	Simulation vorbereiten, Dokumentation, Abschluss	
	14	15.12. - 19.12.	Fertigstellung, Abgabe Abstract und Dokumentation	M5
Presentation	15	20.12. - 28.12.	Simulation durchführen	
	16	29.12.2025 - 04.01.2026		
	17	05.01. - 09.01.	Ergebnisse auswerten, Präsentation und A0-Poster	M6

Tab. 43: Long-Term Plan mit ursprünglicher Construction-Phase (Version 1.0)

Die einzelnen Komponenten und Funktionen sind wie folgt definiert:

1. Backend

- **1.1:** Features ergänzen
- **1.2:** Auto. Zahlungsbestätigung über Bank

2. Datenbank

- **2.1:** Schema anpassen
- **2.2:** DB-Login verbessern (2FA)
- **2.3:** DB-Transactions / Rollbacks

3. Discord-Bot

- **3.1:** Features ergänzen
- **3.2:** Darstellung Events verbessern
- **3.3:** Logs einsehen

4. Admin-Dashboard

- 4.1: Login erstellen
- 4.2: Darstellung / Bearbeitung DB-Daten (Mitglieder, Rollen, Events)
- 4.3: Darstellung / Bearbeitung / Anmeldung von Aufgaben
- 4.4: Darstellung / Bearbeitung von Ranglisten
- 4.5: Erstellung des Punktesystems (Punkte, Belohnungen, Jahresplanung)
- 4.6: Logs einsehen

5. Neue Plattform (optional)

- 5.1: Wahl der Plattform
- 5.2: Implementierung gleicher Features wie Discord-Bot

6. Unit-Tests

- 6.1: Backend
- 6.2: Discord-Bot
- 6.3: Dashboard
- 6.4: Neue Plattform

7. Pipelines

- 7.1: Dokumentation
- 7.2: Backend
- 7.3: Discord-Bot
- 7.4: Dashboard
- 7.5: Neue Plattform

8. Usability Tests (optional)

- 8.1: Discord-Bot
- 8.2: Dashboard
- 8.3: Neue Plattform

10.4.1 Inception (Initiierung)

In der Inception-Phase werden die Anforderungen für das Projekt und der Projektplan erstellt, sowie die Risiken identifiziert. In dieser Phase wird zudem die generelle Herangehensweise und die Machbarkeit des Projektes bestimmt.

10.4.2 Elaboration (Ausarbeitung)

In der Elaboration-Phase werden die Anforderungen des Projektes im Detail analysiert. Hier wird das Gamification- und Sicherheitskonzept ausgearbeitet. Die Architektur wird mittels des C4-Modells definiert und ein erster Softwareprototyp wird erstellt. Dieser wird verwendet, um erste Systeme zu testen, auf denen dann später aufgebaut wird.

10.4.3 Construction (Konstruktion)

Die Construction-Phase wird dazu verwendet das Projekt technisch umzusetzen. In dieser Phase werden die geplanten Features umgesetzt und systematisch getestet. Usability-Tests werden durchgeführt auf dessen Basis Anpassungen vorgenommen werden. Die umgesetzten Features und Tests werden dokumentiert.

10.4.4 Transition (Übergang)

In der Transition-Phase wird das Projekt abgeschlossen. Die Dokumentation wird fertiggestellt und der Abstract wird geschrieben. Die Dokumentation wird in dieser Phase abgegeben.

10.4.5 Presentation (Präsentation)

Da das Projekt neben einer Studienarbeit auch eine Bachelorarbeit ist, wird in der Presentation-Phase die Präsentation erstellt. In dieser Phase arbeitet Vanessa Alves, welche die Bachelorarbeit durchführt, an der Präsentation und am Poster alleine.

10.4.6 Meilensteine

Die folgenden Meilensteine dienen als Hilfe, um die definierten Ziele des Projektes zu erreichen. Sie gelten als erfüllt, sobald die einzelnen Komponenten durch das Team in [Jira](#) als erledigt gekennzeichnet und vom Referenten eingesehen wurden.

Achtung:

Im Verlauf des Projektes wurden Meilenstein M3 und M4 angepasst. (Siehe [Abschnitt 10.6](#))

M1: Projekt Initiierung

- Der Projektplan ist definiert.
- Die Risiken sind definiert.
- Das MVP ist definiert.
- Der Long-Term-Plan ist definiert.

M2: Prototyp

- Die Requirements sind definiert.
- Die Architektur ist definiert.
- Mockups für das Admin-Dashboard sind erstellt.
- Das Gamificationkonzept, Datenschutzkonzept und Testkonzept ist erstellt.
- Der erste Prototyp für das Admin-Dashboard ist erstellt.

M3: Erreichung des MVP

- Die Discord-Bot Erweiterungen sind umgesetzt.
- Das Admin-Dashboard ist elementarisch umgesetzt.
- Das [MVP](#) wird erreicht.

M4: Fertigstellung der Implementation

- Alle geplanten, nicht-optionalen Features sind umgesetzt.
- Alle Funktionen sind erfolgreich getestet.

M5: Abgabe

- Die Simulation ist vorbereitet.
- Die umfassende Dokumentation ist fertiggestellt.
- Das Abstract ist fertiggestellt.

M6: Präsentation

- Die Simulation wurde durchgeführt und ausgewertet.
- Die Präsentation ist fertiggestellt.
- Das Poster ist fertiggestellt.

10.5 Risikomanagement

Die folgenden Tabellen zeigen die Risiken des Projektes auf und die Strategien die verwendet werden, um diese zu vermindern.

Wahrscheinlichkeit / Schweregrad	1 - Sehr Unwahrscheinlich	2 - Unwahrscheinlich	3 - Gelegentlich	4 - Wahrscheinlich	5 - Oft
4 - Katastrophal	R2				
3 - Kritisch		R3, R7	R9, R11		
2 - Signifikant			R4, R8, R13	R1, R12	
1 - Gering		R5		R6, R14	

Tab. 44: Risiko Matrix

ID	R1
Risiko	Lernkurve
Kommentar	Der Einsatz neuer Technologien (z. B. Frameworks, CI/CD, Web-Technologien) führt zu einer erhöhten Einarbeitungszeit.
Prävention	Frühzeitige Prototypen reduzieren die Einarbeitungszeit. Wissen wird im Team dokumentiert und geteilt.
Korrektur	Komplexe Features werden in kleinere Arbeitspakete unterteilt. Bei Problemen erfolgt gegenseitige Unterstützung im Team.

Tab. 45: Beschreibung Risiko 1

ID	R2
Risiko	Probleme bei Discord oder der neuen Plattformen
Kommentar	Discord oder alternative Plattformen (z. B. MS Teams, Matrix) können temporär nicht verfügbar sein.
Prävention	Evaluierung alternativer Plattformen und Absicherung kritischer Funktionen über die Web-Applikation.
Korrektur	Risiko muss weitgehend akzeptiert werden. Bei längeren Ausfällen wird die Nutzung der Web-Applikation verstärkt.

Tab. 46: Beschreibung Risiko 2

ID	R3
Risiko	Ressourcen limitiert
Kommentar	Ein Teammitglied fällt temporär oder dauerhaft aus.
Prävention	Alle Arbeitsschritte und Entscheidungen werden systematisch dokumentiert, um Wissenstransfer sicherzustellen.
Korrektur	Aufgaben werden durch verbleibende Teammitglieder übernommen. Fokus liegt auf priorisierten Features, um Projektziele trotz reduzierter Kapazität zu erreichen.

Tab. 47: Beschreibung Risiko 3

ID	R4
Risiko	Scope Creep
Kommentar	Anforderungen werden unklar definiert oder nachträglich erweitert, was den Projektumfang vergrössert.
Prävention	Detaillierte Anforderungsdefinition und Scope-Management im Rahmen der Meetings.
Korrektur	Nachträglich eingebrachte oder optionale Anforderungen mit niedriger Priorität werden in spätere Phasen verschoben oder gestrichen.

Tab. 48: Beschreibung Risiko 4

ID	R5
Risiko	Kompatibilitätsprobleme
Kommentar	Unterschiedliche Versionen von Tools, Bibliotheken oder Frameworks können zu Integrationsproblemen führen.
Prävention	Einheitliche Dokumentation der eingesetzten Versionen und Verwendung von Lockfiles (z. B. package-lock.json).
Korrektur	Versionskonflikte werden durch Abstimmung im Team gelöst, eine einheitliche Version wird verbindlich definiert.

Tab. 49: Beschreibung Risiko 5

ID	R6
Risiko	Zeiteinschätzung
Kommentar	Arbeitsaufwände werden zu optimistisch oder pessimistisch eingeschätzt.
Prävention	Iterative Abschätzung in den wöchentlichen Meetings sowie Nutzung von Erfahrungswerten und Vergleich mit vorherigen Projekten.
Korrektur	Optional geplante Features werden verschoben oder gestrichen, um Kernziele nicht zu gefährden.

Tab. 50: Beschreibung Risiko 6

ID	R7
Risiko	Änderungen in der Discord API oder andere Technologien
Kommentar	Fundamentale Änderungen in der Discord API oder anderen zentralen Technologien (z. B. Datenbank, Framework).
Prävention	Keine Verwendung von Funktionen, die als „deprecated“ markiert sind und Monitoring von Release Notes.
Korrektur	Anpassung des Codes und Refactoring bei API-Änderungen, auch wenn zusätzlicher Aufwand entsteht.

Tab. 51: Beschreibung Risiko 7

ID	R8
Risiko	Der bestehende Code ist stark an Discord gekoppelt, was die Entwicklung einer Web-App erschwert.
Kommentar	Der bisherige Code ist nicht komplett von Discord entkoppelt.
Prävention	Frühzeitiges Refactoring und Einführung einer modularen Architektur.
Korrektur	Falls Aufwand höher als geplant, Anpassung des Zeitplans und Reduktion optionaler Features.

Tab. 52: Beschreibung Risiko 8

ID	R9
Risiko	Dashboard ist nicht benutzerfreundlich
Kommentar	Dashboard erfüllen nicht die Erwartungen der Mitglieder/Vorstände hinsichtlich Usability.
Prävention	Entwicklung mit Usability-Tests und Einbindung von Stakeholdern.
Korrektur	Anpassung der Benutzeroberfläche und Interaktionskonzepte anhand von Feedback.

Tab. 53: Beschreibung Risiko 9

ID	R10
Risiko	Manipulation Gamification-System
Kommentar	Mitglieder könnten das Punktesystem missbrauchen (z. B. durch Mehrfachaktionen).
Prävention	Definition klarer Regeln, technische Prüfungen gegen Mehrfacheinträge, Monitoring verdächtiger Aktivitäten durch den Vorstand.
Korrektur	Rücksetzen oder Anpassung von Punkten bei Missbrauch und Anpassung der Regeln.

Tab. 54: Beschreibung Risiko 10

ID	R11
Risiko	Datenschutz
Kommentar	Potenzielle Risiken im Umgang mit personenbezogenen Daten.
Prävention	Erstellung eines Datenschutzkonzepts in Anlehnung an DSGVO und Schweizer Datenschutzgesetz sowie die Minimierung der erhobenen Daten.
Korrektur	Einholung von Einverständniserklärungen.

Tab. 55: Beschreibung Risiko 11

ID	R12
Risiko	Stakeholder-Anforderungen nicht berücksichtigt
Kommentar	Feedback von Mitgliedern oder Vorstand wird nicht ausreichend eingebunden.
Prävention	Priorisierung der Anforderungen, Usability-Tests mit realen Mitgliedern.
Korrektur	Anpassungen anhand von Feedback, Verschiebung von Features niedriger Priorität.

Tab. 56: Beschreibung Risiko 12

ID	R13
Risiko	Plattform-Integration komplexer als erwartet
Kommentar	Die geplante Entkopplung auf Dashboard oder anderen Plattformen verursacht höheren Aufwand oder technische Probleme.
Prävention	Proof-of-Concept frühzeitig erstellen, modulare Architektur.
Korrektur	Fokus auf Admin-Dashboard, weitere Plattformintegration in spätere Phase verschieben.

Tab. 57: Beschreibung Risiko 13

ID	R14
Risiko	Hosting oder CI/CD Ausfall
Kommentar	Server oder CI/CD-Pipeline fallen aus und blockieren Entwicklung oder Betrieb.
Prävention	Monitoring und automatisierte Backups, klare Verantwortlichkeiten im Team.
Korrektur	Deployment auf alternative Cloud-Umgebung oder manuelles Deployment als Notfalllösung.

Tab. 58: Beschreibung Risiko 14

10.5.1 Ausweichplan

Der Ausweichplan dient dazu systematisch auf Risiken zu reagieren, die im Vorfeld nicht bestimmt werden konnten. Der Ablauf ist wie folgt definiert:

1. Das Problem innerhalb einer Aufgabe wird identifiziert und mit dem Teammitglied besprochen.
2. Es wird versucht das Problem so schnell wie möglich zu lösen.
3. Kann das Problem nicht gelöst werden, wird es im nächsten Meeting mit dem Referenten besprochen.

Ziel dieses Ausweichplans ist es, bei auftretenden Problemen, die den Projektverlauf beeinträchtigen, eine klare und strukturierte Vorgehensweise zu gewährleisten. Dadurch kann effizient reagiert und ein reibungsloser Projektfortschritt sichergestellt werden.

10.6 Auswirkungen des Refactorings auf die Planung

Der ursprüngliche Long-Term-Plan ([Abschnitt 10.4](#)) ging davon aus, dass das Vorprojekt eine weitgehend tragfähige Grundlage für die Weiterentwicklung bilden würde. Insbesondere wurde angenommen, dass die bestehenden Komponenten Mitgliederverwaltung, Kalender-Synchronisation, Discord-Bot sowie Gamification nur erweitert und nicht grundlegend neu strukturiert werden müssten. Auf dieser Basis wurden die Meilensteine [M3 \(Erreichung des MVP\)](#) und [M4 \(Fertigstellung der Implementation\)](#) zeitlich ambitioniert geplant, da nur ein moderater technischer Ausbau erwartet wurde ([Abschnitt 10.4.6](#)).

Wie im Kapitel Architekturelles Refactoring ([Abschnitt 8.2](#)) ausführlich erläutert, zeigte sich zu Beginn der Construction-Phase, dass das Vorprojekt trotz erster Strukturierungsansätze wesentliche architektonische Defizite aufwies. Besonders problematisch war das Fehlen einer eigenständigen API-Schicht, was dazu führte, dass der Discord-Bot direkt mit Datenbankmodellen interagierte und keinerlei saubere Trennung zwischen Präsentations-, Anwendungs- und Infrastrukturschicht bestand. Zudem fehlte eine gemeinsame Shared-Library, wodurch Typen, Validierungen und Fehlermeldungen mehrfach und inkonsistent implementiert waren. Die Kopplung zwischen Bot, Backend und Datenbank war so eng, dass eine flexible Erweiterung oder die Einführung einer zweiten Plattform, wie beispielsweise das Admin-Dashboard, nicht möglich war.

Aufgrund dieser strukturellen Einschränkungen war die ursprünglich geplante Multi-App-Architektur aus Backend, Discord-Bot, Dashboard und Shared-Package auf Basis des bestehenden Codes nicht realisierbar. Ein umfassendes Refactoring war daher unvermeidlich, um eine saubere technische Grundlage zu schaffen und die geplanten Funktionen nachhaltig und wartbar implementieren zu können.

10.6.1 Teamverteilung und Einfluss auf den Projektverlauf

Die Aufgaben wurden im Team strategisch verteilt. Das Teammitglied Vanessa übernahm das Refactoring des Backends und des Bots, während Joel gleichzeitig das Admin-Dashboard entwickelte. Diese Aufteilung war bewusst gewählt, da Vanessa das Vorprojekt erstellt hatte und sich somit besonders gut mit den Bestandsstrukturen auskannte. Es wurde erwartet, dass sie das Refactoring durch ihr Vorwissen effizient durchführen könnte, während Joel unabhängig davon das Dashboard neu aufbauen kann.

In der Praxis zeigte sich jedoch, dass der tatsächliche Aufwand deutlich höher war als prognostiziert. Die Entkopplung des Bots, der vollständige Neubau des API-Layers, die Migration aller Commands und der Neuaufbau der Domain-Logik waren komplexer und zeitintensiver als erwartet. Dennoch war die Aufgabenaufteilung effizient. Während Vanessa die technische Basis modernisierte, konnte Joel das Dashboard parallel vorantreiben, sodass trotz der Herausforderungen kontinuierlicher Fortschritt erzielt wurde.

Dieser Verlauf entspricht den im Risikomanagement identifizierten Risiken [R6 \(Zeiteinschätzung\)](#) , [R8 \(Kopplung des alten Systems\)](#) und [R13 \(komplexere Plattform-Integration\)](#) , die zwar als moderat eingeschätzt wurden, sich jedoch als planungsrelevant herausstellten.

10.6.2 Auswirkungen auf Zeitplan und Meilensteine

Der hohe technische Aufwand führte dazu, dass ein Grossteil der Wochen 7-10 für strukturelle Arbeiten verwendet werden musste. Die Funktionsentwicklung, wie sie im ursprünglichen Long-Term-Plan ([Abschnitt 10.4](#)) für diese Phase vorgesehen war, konnte daher nicht parallel erfolgen. Erst gegen Ende von Woche 10 erreichte das System wieder einen stabilen Zustand, der die weitere Implementierung erlaubte.

Trotz dieser Verzögerung konnten wesentliche Kernkomponenten wie Bot-Basiskommandos, Mitglieder- und Rollenverwaltung, Dashboard-Login und erste Gamification-Ansichten fertiggestellt werden. Die Verschiebung betraf allerdings direkt die Meilensteine M3 und M4, da der ursprünglich geplante Funktionsumfang in diesem Zeitraum nicht vollständig erreicht werden konnte.

Die nachfolgende Tabelle zeigt den Stand der in Woche 10 umgesetzten und noch nicht erledigten Funktionen. Diese Abweichung machte eine umfassende Anpassung des Long-Term-Plans notwendig.

Umgesetzt	• Discord-Bot: /join, /leave, /events, /linkcalendar, /synccalendar • Dashboard: Login, Mitgliederverwaltung, Rollenverwaltung, Basisansichten für Aufgaben, Events, Belohnungen und Ranglisten • Backend: vollständige neue Architektur mit Calendar-Sync, Domain-Modellen, DB-Services und API-Layer
Nicht umgesetzt	• Gamification-Logik • Rollen- und Benutzersynchronisation • Leaderboards und Rewards • erweiterte Dashboard-Verwaltung • CI/CD, Tests, E-Mail-Funktionalität

Tab. 59: Beschreibung der umgesetzten bzw. noch nicht umgesetzten Funktionen in Woche 10

Um die Meilensteine trotz der architektonischen Notwendigkeiten zu erreichen, wurde eine realistische Priorisierung vorgenommen. Der Fokus liegt nun ausschliesslich auf [MVP-relevanten Funktionen](#) :

- **Discord-Bot:** Punktevergabe, Aufgabenabschluss, Sync-Befehle, Leaderboard-Erstellung und das Speichern von Spenden
- **Admin-Dashboard:** Anzeige aller Transaktionen, Leaderboards sowie das Erstellen des Punktesystems
- **Backend API:** Gamification-Minimalversion sowie Rollen- / User-Sync
- **Weiteres:** einfache CI und grundlegende Tests

Die nachfolgenden Funktionen werden bewusst verschoben oder gestrichen, da sie nicht MVP-relevant sind:

- Event-, Belohnungs- und Ranglistenverwaltung im Dashboard
- Vereins-Analytics
- Logging der Aktionen
- E-Mail Passwort-Reset
- Gamification Elemente (Punkteablauf, Jahresbeitrags-Reminder)
- Bidirektionaler Event-Sync
- Automatisierte Zahlungsbestätigung von Mitgliederbeiträgen
- Vollständige CI/CD-Pipeline
- Mitglieder-Frontend
- Erweiterte Testabdeckung

Diese Reduktion entspricht dem im Risikomanagement definierten Umgang mit Scope Creep (R4) und stellt sicher, dass das Projekt weiterhin zielgerichtet in Richtung MVP voranschreiten kann.

Da der ursprüngliche Zeitplan durch das notwendige Refactoring erheblich beeinflusst wurde, betrifft die Neuplanung ausschliesslich die Construction-Phase. Die übrigen Phasen (Inception, Elaboration, Transition, Presentation) können unverändert bestehen bleiben. Die folgende Tabelle zeigt den neuen Long-Term-Plan mit der überarbeiteten Construction-Phase (Version 2.0).

Phase	Woche	Datum	Ziele	Meilenstein
Inception	1	15.09. - 21.09.	Einrichtung, MVP definieren	
	2	22.09. - 28.09.	Projektplan, Risikomanagement, Long-Term-Plan	M1
Elaboration	3	29.09. - 05.10.	Requirements, Einleitung, Stand der Technik	
	4	06.10. - 12.10.	Interviews andere Vereine, Sicherheitskonzept	
	5	13.10. - 19.10.	Gamificationkonzept, Architektur, Wireframes	
	6	20.10. - 26.10.	Testkonzept, Prototyp Admin-Dashboard (Proof of Concept)	M2
Construction (Version 2.0)	7	27.10. - 02.11.	Analyse Architektur Vorprojekt, Vollständiges Refactoring starten, 1.1, 1.2, 5.1	
	8	03.11. - 09.11.	API Grundgerüst, Dashboard aufsetzen, 3.1, 3.2	
	9	10.11. - 16.11.	API-Schicht, Shared-Library, erste Controller, 3.3, 3.4, 3.5	
	10	17.11. - 23.11.	Discord-Bot auf API umstellen, 2.1, 2.2, 2.3, 2.4, 3.6, 3.7, 3.8	M3
	11	24.11. - 30.11.	2.5, 2.6, 3.9, 3.10, 3.11	
	12	01.12. - 07.12.	2.7, 4, 5.2, 5.3, 5.4, Implementation beenden	M4
Transition	13	08.12. - 14.12.	Simulation vorbereiten, Dokumentation, Abschluss	
	14	15.12. - 19.12.	Fertigstellung, Abgabe Abstract und Dokumentation	M5
Presentation	15	20.12. - 28.12.	Simulation durchführen	
	16	29.12.2025 - 04.01.2026		
	17	05.01. - 09.01.	Ergebnisse auswerten, Präsentation und A0-Poster	M6

Tab. 60: Long-Term Plan mit angepasster Construction-Phase (Version 2.0)

Die angepassten Komponenten und Funktionen:

1. Datenbank

- **1.1:** Schema anpassen
- **1.2:** DB-Transactions / Rollbacks (Theorie angesehen + Beispiel-Umsetzung)

2. Discord-Bot

- **2.1:** Mitglied-Bewerbung (/join, /leave)
- **2.2:** Event-Ansicht (/events)
- **2.3:** Darstellung Events verbessern
- **2.4:** Kalender-Anbindung (/linkcalendar, /synccalendar)
- **2.5:** Gamification-Elemente (/score, /createtask, /completetask, /createleaderboard, /rewards, /donation)
- **2.6:** Daten-Ansicht (/getdata)
- **2.7:** Daten-Synchronisation (/syncroles, /syncusers)

3. Admin-Dashboard

- **3.1:** Login erstellen
- **3.2:** Mitgliederverwaltung
- **3.3:** Rollenverwaltung
- **3.4:** Transaktions-Historie (nur eigenes)
- **3.5:** Aufgabenverwaltung (ohne claim)
- **3.6:** Event-Ansicht (ohne Verwaltung)
- **3.7:** Belohnungs-Ansicht (ohne Verwaltung)
- **3.8:** Ranglisten-Ansicht (nur All-Time)
- **3.9:** Transaktionen anzeigen (Auswahl aller User, read-only)
- **3.10:** Ranglisten ansehen (Auswahl aller Ranglisten, read-only)
- **3.11:** Erstellung Punkte-System

4. Unit-Tests (Nur Mitgliedsbewerbung)

5. Pipelines

- **5.1:** Dokumentation
- **5.2:** Backend
- **5.3:** Discord-Bot
- **5.4:** Dashboard

Optionale Features, falls Zeit vorhanden:

- **Backend:**
 - Bidirektionales Event-Sync
 - Gamification-Logik (Punkteablauf, Erinnerung Jahresbeitrag / Aufgaben, Hervorhebungen aktiver Mitglieder)
 - Logging
- **Discord-Bot:** Logs einsehen (/logs)
- **Admin-Dashboard:**
 - Vollständige Event-Verwaltung
 - Vollständige Belohnungs-Verwaltung
 - Vollständige Ranglisten-Verwaltung
 - Vereins-Analytics
 - E-Mail-Funktion (bei Vergessen des Passwortes)
 - Logs einsehen
 - Login und Darstellung der Daten für Mitglieder
- **Weitere Tests**
- **Vollständiges CI/CD**

Angepasste Meilensteine:

- **M3: Architektur stabilisiert**
 - Refactoring abgeschlossen
 - Backend API, Discord-Bot und Admin-Dashboard laufen
- **M4: MVP Implementation abgeschlossen**
 - Gamification-Basis funktioniert mit der neuen Architektur
 - Bot-Kommandos und Dashboard-Views sind umgesetzt
 - Das MVP wird erreicht.

Durch die gezielte Fokussierung auf MVP-relevante Funktionen, die klare Priorisierung und die effiziente Teamaufteilung bleibt der Projektverlauf trotz der Herausforderungen planbar. Die überarbeiteten Planungsentscheidungen stellen sicher, dass die zentralen Meilensteine termingerecht erreicht werden können und die Simulation in der Transition-Phase erfolgreich vorbereitet ist.

11 Arbeitszeitznachweis

12 Tooling und Technologie-Entscheidungen

Dieses Kapitel beschreibt die im Rahmen der Bachelorarbeit eingesetzten Werkzeuge, Technologien und Entwicklungsprozesse. Es begründet die wesentlichen Entscheidungen hinsichtlich Projektorganisation, Dokumentation, Code-Qualität und Deployment. Grundlage bilden die Erfahrungen aus dem Vorprojekt, welche in dieser Arbeit systematisch weiterentwickelt werden.

12.1 Jira

Die Zusammenarbeit und Aufgabenverteilung wird mit Jira² verwaltet. Für jede Aufgabe wird im Kanban Board ein Issue erstellt mit den folgenden Merkmalen:

- **Epic:** Die Phase zu dem das Issue gehört.
- **Typ:** Der Typ des Issues (Meeting, Aufgabe).
- **Geschätzte Zeit:** Die Soll-Zeit des Issues.
- **Verwendete Zeit:** Die tatsächlich aufgewendete Zeit um, den Issue zu beenden.
- **Zugewiesene Person:** Die Person, die das Issue durchführt.

Die Issues werden alle zuerst im Backlog definiert. Im wöchentlichen Team-Meeting werden die Issues, welche in dieser Woche geplant sind in die „Todo“-Spalte geschoben. Wenn die Person, der das Issue zugeteilt wurde, mit der Arbeit beginnt, wird dieser in die „In Progress“-Spalte geschoben. Wurde das Issue umgesetzt, kommt er in die „Review“-Spalte. Alle Issues in der Review Spalte werden im wöchentlichen Meeting besprochen und bei einer erfolgreichen Umsetzung in die „Done“-Spalte verschoben. Falls es noch Verbesserungspotenzial gibt, kommt das Issue zurück in die „Todo“-Spalte.

12.2 Teams

Für die interne Kommunikation, spontane Absprachen und die Planung von ausserordentlichen Meetings wird Microsoft Teams³ verwendet. Es dient als zentraler Kommunikationskanal und Speicherort für geteilte Dokumente. Damit wird ein kontinuierlicher Informationsfluss sichergestellt, was insbesondere bei verteilter Arbeit im Rahmen der Bachelorarbeit entscheidend ist.

12.3 GitHub

Das Vorprojekt wurde zu Beginn der Arbeit von GitLab⁴ auf GitHub⁵ migriert, um die Entwicklungs- und Veröffentlichungsprozesse langfristig unabhängig von der OST-internen Infrastruktur zu gestalten. Die zuvor auf der OST-GitLab gehostete Umgebung war nur innerhalb der Hochschule zugänglich, wodurch externe Zusammenarbeit und eine spätere Öffnung des Projekts erschwert wurden. GitHub bietet im Gegensatz dazu eine öffentlich zugängliche, weit verbreitete Plattform, die sich in der Open-Source-Community etabliert hat. Durch die Nutzung von GitHub Organisations und integrierten CI/CD-Workflows (GitHub Actions) kann die Projektstruktur klar gegliedert und zukünftige Erweiterungen effizient verwaltet werden. Neben diesen Vorteilen diente der Wechsel ausserdem dem Erkenntnisgewinn für das Team.

Für die Projektstruktur wurde eine eigene Organisation namens „ClansCore“⁶ erstellt. Innerhalb dieser Organisation befinden sich die nachfolgenden Repositories:

Repository	Beschreibung
clanscore ⁷	Der Quellcode der Applikation.
clanscore-dok ⁸	Die Projek-Dokumentation.
clanscore-dok-pub ⁹	Die automatisch generierten PDF-Versionen der Dokumentation und Sitzungs-Protokolle, welche über GitHub Pages veröffentlicht werden (siehe <u>Abschnitt 12.4.2</u>).

²<https://www.atlassian.com/software/jira>

³<https://www.microsoft.com/de-ch/microsoft-teams/log-in?market=ch>

⁴<https://gitlab.ost.ch/>

⁵<https://github.com/>

⁶<https://github.com/ClansCore>

⁷<https://github.com/ClansCore/clanscore>

⁸<https://github.com/ClansCore/clanscore-doc>

Tab. 61: Übersicht der Repositories innerhalb der GitHub-Organisation ClansCore

Während der Bearbeitung dieser Arbeit sind die Haupt-Repositories privat gestellt, um den Entwicklungsprozess geschützt zu halten. Die Organisation selbst ist jedoch öffentlich, wodurch Transparenz, Auffindbarkeit und spätere Zusammenarbeit gewährleistet bleiben.

12.3.1 GitHub Guidelines

1. Auf dem Dokumentations-Repository hat jedes Teammitglied seinen eigenen Branch im Format „dev-Initialen“ (z.B. dev-jt)
2. Auf den Code-Repositories (Discord-Bot, Admin-Dashboard) soll für jedes Feature oder Fehler ein jeweiliger Branch im Format „feature-FeatureName“ oder „bug-BugName“ erstellt werden.
3. Das Deployment der Repositories wird je auf einem eigenen Branch namens „deploy“ umgesetzt. Die Sammlung der Commits werden beim mergen des Main-Banches mithilfe der Funktion „Squash“ zusammengefasst, um die Anzahl ähnlicher Commit-Nachrichten zu reduzieren.
4. Bevor Änderungen beim Code in den Develop-Branch gemerged werden, müssen diese von einem anderen Teammitglied überprüft werden.
5. Schreibe Commit-Nachrichten auf Englisch.
6. Beim wöchentlichen Meeting werden die Änderungen vom Develop-Branch in den Main-Branch gemerged.

12.4 Dokumentation

Die Projektdokumentation wird vollständig mit Typst¹⁰ erstellt. Die Entscheidung für Typst erfolgte aufgrund der einfacheren Syntax im Vergleich zu Latex¹¹, der nativen Git-Integration und der bereits im Vorprojekt begonnenen Typst-Vorlage.

12.4.1 Dokumentation Guidelines

1. Dokumentiere auf Deutsch.
2. Dokumentiere mit Typst.
3. Verwende die während diesem Projekt entwickelte Vorlage.
4. Unterteile den Text in eine Typst-Datei pro grösseres Kapitel.
5. Benenne die einzelnen Typst Dateien in Englisch und verwende hierfür kebab-case.
6. Verwende aussagekräftige Titel, welche den Text gut unterteilen.
7. Verseehe alle nötigen Titel mit einer kurzen und klaren Referenz-Variable im snake_case.
8. Verwende eine saubere Formatierung, die dabei, hilft den Text gut lesen zu können.
9. Kennzeichne Referenzen aus externen Quellen als solche.
10. Schreibe für jede Abbildung und Tabelle eine Beschriftung.
11. Schreibe Referenz-Namen der Beschriftungen für Abbildungen und Tabellen im kebab-case.

12.4.2 Dokumentation Deployment

Im Rahmen dieser Arbeit wurde eine automatisierte Dokumentations-Pipeline implementiert, welche die kontinuierliche Erstellung und Veröffentlichung der Projektdokumentation gewährleistet. Ziel war es, den zuvor im Vorprojekt auf GitLab basierenden CI/CD-Prozess auf die GitHub-Infrastruktur zu migrieren und an die Anforderungen des neuen Projektsetups anzupassen (siehe Abschnitt 12.3).

Die Pipeline dient der automatischen Kompilierung und Veröffentlichung der Typst-Dokumentation, bestehend aus dem Hauptbericht und den Sitzungsprotokollen. Dabei werden sämtliche Schritte, vom Erstellen der PDF-Dateien bis zur Bereitstellung auf GitHub Pages, automatisiert ausgeführt. Da GitHub Pages für private Repositories in der kostenfreien Version nicht verfügbar ist, wurde eine Split-Repository-Strategie umgesetzt. Das private Repository (clanscore-doc) enthält den vollständigen Typst-Quellcode, während ein separates öffentliches Ziel-Repository (clanscore-doc-pub) ausschliesslich die generierten Ausgabedateien beherbergt.

⁹<https://github.com/ClansCore/clanscore-doc-pub>

¹⁰<https://typst.app/>

¹¹<https://www.latex-project.org/>

Die Pipeline wurde mittels GitHub Actions realisiert und befindet sich im privaten Repository. Der Workflow wird bei jedem Push auf den Main-Branch ausgelöst. Für die Authentifizierung und das Deployment wurde eine eigene GitHub App innerhalb der Organisation ClansCore erstellt. Diese App besitzt ausschliesslich die für das Deployment benötigten Berechtigungen auf das Ziel-Repository. Das App-Zugriffstoken wird zur Laufzeit über eine Action generiert und anschliessend verwendet, um den Inhalt des public/-Ordners in den Main-Branch des öffentlichen Repositories zu pushen.

Die GitHub Pages des Ziel-Repositories wurden so konfiguriert, dass der Main-Branch automatisch als statische Website veröffentlicht wird. Damit wird bei jeder Änderung im privaten Repository die Dokumentation neu erzeugt und unmittelbar online verfügbar gemacht.

Dieser Ansatz stellt eine klare und sichere CI/CD-Lösung dar. Der Quellcode und interne Inhalte bleiben privat, während die erzeugten Enddokumente öffentlich zugänglich sind. Durch die Verwendung einer GitHub App wird zudem auf persönliche Zugriffstokens verzichtet, was eine nachhaltige und wartungsarme Authentifizierung gewährleistet.

12.5 Code und Entwicklung

Der Code für dieses Projekt wird in der hier beschriebenen Form erstellt.

12.5.1 Code Guidelines

1. Schreibe TypeScript Variablen- und Funktionsnamen im camelCase.
2. Schreibe TypeScript Konstanten in Grossbuchstaben, wobei einzelne Wörter mit Unterstrichen aufgeteilt werden.
3. Schreibe HTML-Element-Bezeichner im kebab-case.
4. Gib Arrays einen Namen im Plural.
5. Schreibe kurze Funktionen mit wenigen Argumenten. Eine Funktion soll nur einen Zweck erfüllen.
6. Verwende let oder const anstelle von var.
7. Schreibe Vergleiche mit === oder !==.
8. Verwende bei Variablen-Zuweisungen Leerzeichen zwischen dem Gleichheitszeichen.
9. Schreibe bei Funktionsheadern die geschweifte Klammer auf die gleiche Zeile.
10. Schreibe Kommentare nur wo es nötig ist, der Code sollte selbsterklärend geschrieben werden.
11. Verwende für gleiche Konzepte die gleichen Wörter.
12. Überprüfe Änderungen vor dem pushen mit Prettier und ESLint.

12.5.2 Setup

Um den Coding-Style einzuhalten werden die Standardregeln von Prettier und ESLint für TypeScript verwendet. Zudem sind bei den Coding Projekten bestimmte Prozesse automatisiert, um die Entwicklung zu erleichtern.

12.5.3 Hosting

Um [ClansCore](#) dauerhaft online zu halten und beispielsweise auf Discord-Ereignisse zu reagieren oder zuverlässigen Zugriff auf das Admin-Dashboard zu gewährleisten, ist ein geeignetes Hosting erforderlich. Während dem [Vorprojekt](#) wurde der Discord-Bot auf einem Linux-Server des Vereins [EGSwiss](#) gehostet, durch manuelle Deployments über den Vereinsinternen Serveradmin. Da dieser Serveradmin zum Zeitpunkt der Arbeit anderweitig beschäftigt war sowie die Teammitglieder keinen direkten Zugriff auf den Server hatten, fiel die Entscheidung auf einen temporären virtuellen Server der Fachhochschule OST. Dieser läuft in einer stabilen Umgebung, ist kostenfrei und garantiert dem Team vollen Zugriff für die Einführung eines automatischen Deployments (siehe [Abschnitt 12.5.4](#)).

12.5.4 Code Deployment

Im [Vorprojekt](#) wurde das Deployment mit Docker manuell durch eine Drittperson auf einem dedizierten Linux-Server durchgeführt. Da ein manuelles Deployment fehleranfällig, langsam und ineffizient ist, wird ein automatisiertes Deployment umzusetzen ([AutoMQ-Contributors, 2025](#)).

Mit GitHub Actions kann das Deployment automatisiert und sichergestellt werden, dass bei einem Push auf den Main-Branch die Änderungen in die Produktivumgebung übernommen werden. Docker wird für das Containermanagement verwendet,

da es eine zuverlässige und plattformunabhängige Bereitstellung der Anwendung ermöglicht ([Docker-Contributors, 2025](#)) und das Team bereits Erfahrung mit Docker hat.

12.6 Datenbank

Im Vorprojekt wurde eine MongoDB-Datenbank verwendet. MongoDB ist eine dokumentenorientierte NoSQL-Datenbank, die Daten in JSON-ähnlichen Dokumenten speichert ([MongoDB-Contributors, 2025b](#)). Eine zentrale Funktion von MongoDB ist die horizontale Skalierbarkeit, was insbesondere bei einer steigenden Nutzung des Bots einen entscheidenden Vorteil darstellt. Die Erweiterungen, welche für ClansCore umgesetzt wurden, erforderten keine fundamentalen Änderungen an der Datenbank, sodass kein Anlass bestand, eine alternative Datenbanklösung in Erwägung zu ziehen.

12.7 Programmiersprachen und Frameworks

ClansCore baut auf einem bereits im Vorprojekt entwickelten Discord-Bot auf. Dieser wurde in TypeScript programmiert und verwendet die Bibliothek Discord.js, welche als die am weitesten verbreitete und aktiv gepflegte Schnittstelle zur Discord API gilt ([Discord.js-Contributors, 2025a](#)). Die Bibliothek abstrahiert die Kommunikation über REST- und WebSocket-Schnittstellen und ermöglicht dadurch eine effiziente und stabile Implementierung der Bot-Funktionalitäten.

Discord.js wird fortlaufend weiterentwickelt und erhält regelmässig sicherheits- und funktionsbezogene Updates ([Discord.js-Contributors, 2025b](#)). Durch die positiven Erfahrungen aus dem Vorprojekt, den aktiven Community-Support und die hohe Kompatibilität mit TypeScript, wird Discord.js weiterhin als technologische Basis verwendet. Diese Entscheidung stellt sicher, dass bestehende Codebestandteile übernommen und nachhaltig weiterentwickelt werden können, ohne eine kostspielige Migration auf eine alternative Bibliothek durchführen zu müssen.

Für die Entwicklung des Admin-Dashboards standen verschiedene moderne Web-Technologien zur Auswahl. Da ein Framework den Entwicklungsprozess strukturiert, die Wiederverwendbarkeit erhöht sowie die Wartung erleichtert, wurde ein komponentenbasiertes Framework eingesetzt. Die Auswahl erfolgte anhand von Kriterien wie Verbreitung, Funktionsumfang, Zukunftssicherheit und vorhandene Teamerfahrung.

Web-Technologie	Beschreibung
React	Eine weit verbreitete JavaScript-Bibliothek für User Interfaces mit Millionen von Anwendern weltweit. React ist komponentenbasiert, kombiniert Logik und Markup in derselben Datei und profitiert von einer grossen Entwickler-Community. Die Flexibilität der Bibliothek ermöglicht unterschiedliche Architekturansätze, erfordert jedoch zusätzliche Tools für Routing, State-Management und Build-Prozesse. (React-Contributors, 2025)
Angular	Ein umfassendes Web-Framework, das eine vollständige Entwicklungsumgebung für skalierbare Web-Applikationen bietet. Angular wird von Google gepflegt, unterstützt TypeScript nativ und beinhaltet bereits Funktionen wie Routing, Dependency Injection und Formularverwaltung. Es ermöglicht eine klare Strukturierung und eine nachhaltige Code-Architektur. (Angular-Contributors, 2025b)
Vue.js	Ein progressives JavaScript-Framework zur Entwicklung von User Interfaces. Vue kombiniert einfache Syntax mit hoher Flexibilität und eignet sich sowohl für kleine als auch komplexe Anwendungen. Es besitzt eine engagierte Community, wird jedoch primär von Einzelentwicklern und kleineren Teams gepflegt. (Vue.js-Contributors, 2025)

Tab. 62: Beschreibung der evaluierten Web-Technologien

Nach einer systematischen Bewertung wurde entschieden, das Admin-Dashboard mit Angular zu entwickeln. Diese Entscheidung beruht auf mehreren Faktoren:

- **TypeScript-Integration:** Angular ist vollständig in TypeScript implementiert, was eine einheitliche Codebasis zwischen Frontend und Backend ermöglicht. Dadurch können Typdefinitionen und Schnittstellen gemeinsam genutzt werden, was die Konsistenz und Fehlersicherheit erhöht.
- **Vollständige Framework-Struktur:** Im Gegensatz zu React, das primär eine UI-Bibliothek darstellt, bietet Angular ein integriertes Framework mit klaren Architekturvorgaben, standardisierten Projektstrukturen und eingebautem Tooling (z. B. Testing, Routing, Dependency Injection). Dies reduziert den Integrationsaufwand externer Bibliotheken und erhöht die Wartbarkeit.

- **Skalierbarkeit und Nachhaltigkeit:** Angular ist auf gross angelegte, modular aufgebaute Anwendungen ausgelegt und gewährleistet damit eine langfristig stabile Codebasis.
- **Teamkompetenz:** Beide Teammitglieder verfügten bereits über Erfahrung im Umgang mit Angular, was den Einarbeitungsaufwand erheblich reduzierte und einen produktiven Entwicklungsstart ermöglichte. Mit React und Vue.js liegen hingegen keine oder nur geringe praktische Erfahrungen vor.

Zusammenfassend wurde Angular aufgrund der klaren Struktur, nativen TypeScript-Unterstützung, hohen Skalierbarkeit und der bestehenden Teamexpertise als geeignetstes Framework für die Umsetzung des Admin-Dashboards bewertet. Diese Wahl stellt sicher, dass das Dashboard langfristig wartbar bleibt und sich konsistent in die bestehende ClansCore-Systemarchitektur integriert.

12.8 UI-Bibliothek

Bibliothekename	Beschreibung
Angular Material	Angular Material ist eine von der Angular-Entwicklergemeinschaft bereitgestellte Komponentenbibliothek, die auf den Richtlinien des Material Designs basiert. Sie bietet qualitativ hochwertige, internationalisierte und barrierefreie UI-Komponenten, die durch umfangreiche Tests auf Leistung und Zuverlässigkeit geprüft sind. (Material-Contributors, 2025)
PrimeNG	PrimeNG ist eine umfassende UI-Komponentenbibliothek für Angular, die eine Vielzahl an individuell anpassbaren und funktionsreichen Komponenten bereitstellt. Mit über 80 verfügbaren UI-Elementen unterstützt sie Entwicklerinnen und Entwickler bei der effizienten Umsetzung moderner Webanwendungen. (PrimeNg-Contributors, 2025)
NG-Zorro	NG-Zorro ist eine auf dem Ant Design basierende UI-Komponentenbibliothek für Angular, die speziell für den Unternehmenseinsatz entwickelt wurde. Sie stellt über 70 hochwertige, in TypeScript implementierte Komponenten bereit. (NG-ZORRO-Contributors, 2025)

Tab. 63: Bibliotheken für das User Interface

Im Unterschied zu PrimeNG und NG-Zorro fiel die Wahl auf Angular Material als UI-Komponentenbibliothek, da es eine besonders enge Integration in das Angular-Framework bietet und direkt vom Angular-Entwicklungsteam gepflegt wird. Dadurch ist eine hohe Kompatibilität mit aktuellen Angular-Versionen sowie eine langfristige Wartung und Stabilität gewährleistet.

Zudem überzeugt Angular Material durch seine klare Ausrichtung auf die Material-Design-Richtlinien, wodurch ein konsistentes und modernes Benutzererlebnis sichergestellt wird. Die Komponenten sind qualitativ hochwertig, barrierefrei und umfassend getestet, was insbesondere für produktive und langlebige Anwendungen von Vorteil ist.

12.9 KI-Tools

KI-Tools wurden eingesetzt, um die Effizienz zu steigern und die Qualität der Ergebnisse zu verbessern. Die Einsatzbereiche umfassen:

- **Technische Entscheidung:** Unterstützung bei der Bewertung von Frameworks, Bibliotheken und Architekturansätzen.
- **Programmierung:** Hilfe bei der Fehlersuche, Optimierung und Codegenerierung.
- **Dokumentation:** Unterstützung beim Strukturieren und Formulieren von Textabschnitten, insbesondere für technische Beschreibungen, Kapitelüberschriften und sprachliche Korrektheit.

Die KI wurde als Assistenzwerkzeug eingesetzt und diente nicht zur automatisierten Generierung vollständiger Inhalte. Alle durch die KI erzeugten Vorschläge wurden kritisch geprüft und bei Bedarf angepasst.

Folgende KI-Tools wurden verwendet:

- OpenAI ChatGPT¹²
- Google Gemini¹³
- GitHub Copilot¹⁴

¹²<https://chatgpt.com/>

¹³<https://gemini.google.com>

¹⁴<https://github.com/features/copilot>

13 Sitzungsprotokolle

Besprechung vom 19.09.2025

Anwesenheitsliste:	Frieder Loch, Joel Thoma, Vanessa Alves
Autor:	Joel Thoma

Inhalt der Sitzung:

- Aufgabenstellung
 - Was kommt als nächstes
 - Abgabetermine
 - Feedback SA FS25
 - Sonstiges
-

Aufgabenstellung

Ein konkretes Projekt finden. Bei anderen onlinebasierten Vereinen Umfragen und Recherche durchführen um Konzept zu verbessern.

Eine Idee ist ein Admin-Dashboard zu machen um von Discord zu entkoppeln und keine Datenbankzugriffe mehr nötig sind. Eine Anbindung an Microsoft Teams oder die Open Source Plattform Matrix wäre denkbar.

Was kommt als nächstes

Problembeschreibung, Zeitplan und Meilensteine definieren.

Abgabetermine

Wegen der SA von Joel ist der Abgabetermin für die Arbeit am 19. Dezember und Vanessa hat noch für die Präsentation und Poster bis am 9. Januar Zeit.

Feedback SA FS25

Wird noch bereitgestellt.

Sonstiges

Änderungen zur Aufgabenstellung:

Gegenleser: Severin Dellsperger

Koreferent: Julia Czerniak-Wilmes

Besprechung vom 29.09.2025

Anwesenheitsliste:	Frieder Loch, Joel Thoma, Vanessa Alves
Autor:	Joel Thoma

Inhalt der Sitzung:

- Besprechung aktueller Stand
 - ToDo
-

Besprechung aktueller Stand

- Meilensteine könnte man messbar machen (klar definieren wann diese erreicht sind)
- Im MVP sollte klar definiert sein welche Teile neu sind (Nur die Plattform-Unabhängigkeit in das MVP?)

ToDo

- Self-Determination-Theory: über dieses oder ähnliche Frameworks Research betreiben und in das Interview einbauen. (vorallem in Verbindung mit Competence, Autonomy, Relatedness)
- Problemstellung am Anfang sollte folgendes enthalten:
 - die zentralen technischen Problem finden und beschreiben. Wo könnte es Probleme geben und wieso ist es schwierig. (Kernprobleme, da unterschiedliche Lösungen finden und analysieren) (vlt Synchronisierung, Modularisierung?)
 - Alles Begründen. Alle Handlungsmöglichkeiten anschauen, analysieren und die Entscheidungen dokumentieren.
 - Das Produkt gut verkaufen. Was ist das endgültige Produkt. Gamification und Produkt gut zusammenpacken.

Besprechung vom 03.10.2025

Anwesenheitsliste:	Frieder Loch, Joel Thoma, Vanessa Alves
Autor:	Joel Thoma

Inhalt der Sitzung:

- Besprechung aktueller Stand
-

Besprechung aktueller Stand

- Es wurde der aktuelle Stand geschildert.
- Verlängerung der Elaboration Phase und Verschiebung vom Gamification Konzept

Besprechung vom 10.10.2025

Anwesenheitsliste:	Frieder Loch, Joel Thoma, Vanessa Alves
Autor:	Joel Thoma

Inhalt der Sitzung:

- GitHub Orga und CI/CD Doku
 - Einleitung / Stand der Technik
 - Anforderungen
 - Interview andere Vereine
 - Zwischenpräsentation
-

GitHub Orga und CI/CD Doku

Frieder wurde über die Organisation in GitHub informiert und die Möglichkeit Zugriff auf den Code und vor allem die online Version der aktuell im Main Branch deployte Dokumentation der Arbeit.

Organisation mit allen Repos: <https://github.com/ClansCore>

Einleitung / Stand der Technik

Wir haben zusammen die Einleitung und den Stand der Technik angeschaut und besprochen. Es ist nichts aufgefallen.

Anforderungen

Die Anforderungen (User Stories, FR, Use Cases und NFR) wurden angeschaut und besprochen.

Forlgende Anpassungen müssen noch gemacht werden:

- NFR3: Ist das ein Security Thema oder wie genau soll das gemessen werden? Evtl. Skript?
- NFR4: Welche UI-Regeln / Guidelines werden genau befolgt?
- NFR9: Messung muss genauer beschrieben werden

Interview andere Vereine

Die Interviews wurden durchgeführt und die unbearbeiteten Resultate vorgestellt. Diese werden diese Woche ausgewertet für das neue Gamification-Konzept.

Zwischenpräsentation

Vanessa wird voraussichtlich alleine die Präsentation Mitte Semester (Woche 7) durchführen. Joel darf freiwillig mitmachen oder zusehen.

Mögliche Daten für die Zwischenpräsentation, welche noch bestimmt wird.

- 28.10 - 15:00-17:00
- 29.10 - 13:00-15:00
- 30.10 - 13:00-15:00

Besprechung vom 17.10.2025

Anwesenheitsliste:	Frieder Loch, Joel Thoma, Vanessa Alves
Autor:	Joel Thoma

Inhalt der Sitzung:

- Anforderungen
 - Gamification
 - Sicherheitskonzept
 - Wireframes
-

Anforderungen

Die Anforderungen (User Stories, FR, Use Cases und NFR) wurden angeschaut und besprochen.

- Es wurde entschieden, dass die Zahlungsbestätigung, Zahlungserinnerung und die Umsetzung der neuen Plattform eine tiefe Priorität bekommen und wahrscheinlich nicht umgesetzt werden. Folgende Anpassungen müssen noch gemacht werden:
- Die User Stories lösungsneutral halten (Kein Admin-Dashboard in der Beschreibung)
- FR12, FR13, FR14, FR15: Verschiebung in den Teil Implementation.
- FR16: Erklären wieso eine Implementation von einer weiteren Plattform möglich wäre, falls diese nicht umgesetzt wird. z.B durch gute Architektur/Schnittstellen Dokumentation

Gamification

Der aktuelle Stand wurde geschildert. Noch zu erledigen sind: Self-Determination Theory, Interview Auswertung, Analyse

Sicherheitskonzept

Der aktuelle Stand wurde angeschaut und besprochen. Es ist nichts aufgefallen.

- Severin darf kontaktiert werden und wegen dem Sicherheitskonzept gefragt werden.

Wireframes

Der aktuelle Stand wurde geschildert. Folgende Anpassungen müssen noch gemacht werden:

- Schildern wie man zum Design gekommen ist. (Beispiele, Forschung)

Besprechung vom 24.10.2025

Anwesenheitsliste:	Frieder Loch, Joel Thoma, Vanessa Alves
Autor:	Joel Thoma

Inhalt der Sitzung:

- Feedback Anforderungen
 - Gamification
 - Domain Analyse
 - Architektur
 - Wireframes
 - Long-Term-Plan
 - Zwischenpräsentation
-

Anforderungen

Die Anforderungen mit implementiertem Feedback wurden angeschaut und besprochen

- Es könnten NFR entfernt werden welche nicht projektspezifisch sind

Gamification

Die verschiedenen Abschnitte von der Gamification (SDT, Auswertung Umfragen, Octalysis, Analyse) wurden angeschaut und besprochen.

Domain Analyse

Die Domain Analyse wurde angeschaut und besprochen. Das Domain Model ist auf den ersten Blick noch etwas unübersichtlich. Neue Sachen farblich hervorzuheben

Architektur

Der aktuelle Stand des Kapitels über die Architektur wurde geschildert.

- Das Component Diagramm muss noch erledigt werden.

Wireframes

Die Wireframes wurden angeschaut.

Long-Term-Plan

Der Meilenstein 3 wird eine Woche nach hinten geschoben.

Zwischenpräsentation

Informationen zur Zwischenpräsentation:

- Die Präsentation findet Online statt und sollte ca. 15 Min. dauern.
- Prototyp kann, aber muss nicht gezeigt werden.
- Nicht alle Anwesenden kennen die Arbeit bereits, sie muss erklärt werden.
- Den aktuellen Stand + Pläne zeigen
- Feedback einholen

Besprechung vom 31.10.2025

Anwesenheitsliste:	Frieder Loch, Joel Thoma, Vanessa Alves
Autor:	Joel Thoma

Inhalt der Sitzung:

- Besprechung aktueller Stand
 - Feedback Präsentation
 - ToDo Kommende Woche
-

Besprechung aktueller Stand

- Es wurde der aktuelle Stand vom Testkonzept, DB-Schema und dem Prototyp geschildert.

Feedback Präsentation

Folgende Punkte wurden als Feedback bei der Zwischenpräsentation gegeben:

- Mehr Konzentration auf CI und weniger auf CD
- Anstatt Usability Tests könnte nach dem 19. Dezember 2 Wochen ClansCore getestet/benutzt werden.

ToDo Kommende Woche

- Refactoring
- DB-Modell Beschreibung
- neue Backend Funktionen implementieren

Besprechung vom 07.11.2025

Anwesenheitsliste:	Frieder Loch, Joel Thoma, Vanessa Alves
Autor:	Joel Thoma

Inhalt der Sitzung:

- Besprechung aktueller Stand
 - Long-Term-Plan und Simulation
 - ToDo Kommende Woche
-

Besprechung aktueller Stand

- Es wurde der aktuelle Stand geschildert.

Long-Term-Plan und Simulation

- Der neue Long-Term-Plan wurde gezeigt.
- Nach dem 19. Dezember, welches die Abgabe der schriftlichen Arbeit ist, gibt es ein 2 Wochen Pilot-Projekt welches von Vanessa durchgeführt wird.

ToDo Kommende Woche

- Angular Dashboard Entwicklung
 - Login
 - Funktionen Vorstandsmitglied
 - Funktionen Vereinsmitglied
- Refactoring Backend
- Neue Backend Funktionen
- Transaction im neuen Backend einbauen

Besprechung vom 14.11.2025

Anwesenheitsliste:	Frieder Loch, Joel Thoma, Vanessa Alves
Autor:	Joel Thoma

Inhalt der Sitzung:

- Refactoring
 - Admin-Dashboard
 - ToDo Kommende Woche
-

Refactoring

Der aktuelle Stand vom Umbau des Backends wurde geschildert.

Admin-Dashboard

Der aktuelle Stand wurde geschildert.

ToDo Kommende Woche

- Angular Dashboard Entwicklung
 - Alle Funktionen von Vorstandmitgliedern
- Fertigstellung Refactoring Backend
- Feedback Discord-Bot implementieren
 - Darstellung Events
- MS3: MVP Fertigstellen

14 Persönliche Erfahrungsberichte

14.1 Bericht Joel Thoma

14.2 Bericht Vanessa Alves

Kapitel III

Glossar und Abkürzungsverzeichnis

ClansCore	ClansCore ist das Gesamt-System und der Produktname für die Erweiterung des Discord-Bots aus dem <u>Vorprojekt</u> . Es ist das End-Produkt dieser Arbeit und verbindet sämtliche Teil-Komponenten wie Backend, Datenbank, Discord-Bot, Admin-Dashboard und weitere Plattformen.
EGSwiss	Ein Schweizer Gaming Verein, wobei das Teammitglied Vanessa Alves Präsidentin ist und daher die nötige Expertise für die Vorarbeit und aktuelle Weiterentwicklung mitbringt sowie Erfahrungen aus dem Vereinsalltag einfließen lässt.
CI/CD	Continuous Integration (CI) und Continuous Deployment/Delivery (CD). Ein automatisierter Prozess, bei dem Codeänderungen kontinuierlich integriert, getestet und automatisch oder mit manueller Freigabe in die Produktionsumgebung publiziert werden.
Representational State Transfer (REST)	Representational State Transfer (REST) ist eine Architekturform für webbasierte Schnittstellen, bei der Daten über standardisierte HTTP-Methoden (z. B. GET, POST, PATCH, DELETE) ausgetauscht werden. REST-APIs sind zustandslos, d. h. jede Anfrage enthält alle notwendigen Informationen, um sie unabhängig von früheren Interaktionen zu verarbeiten.
WebSocket-Schnittstelle	WebSockets sind bidirektionales Kommunikationsprotokolle (RFC 6455), welches eine dauerhafte Verbindung zwischen Client und Server ermöglicht. Im Gegensatz zu REST, das auf einzelnen HTTP-Anfragen basiert, erlaubt WebSocket eine kontinuierliche Übertragung von Daten in Echtzeit.
Role-Based Access Control (RBAC)	Role-Based Access Control (RBAC) ist ein Sicherheitskonzept, bei dem der Zugriff auf Ressourcen anhand von Rollen vergeben wird. Benutzer erhalten dabei bestimmte Rollen, die jeweils unterschiedliche Berechtigungen für Funktionen und Daten gewähren. So können Administratoren granular steuern, wer auf welche Teile eines Systems zugreifen darf.
Zwei- / Multi-Faktor-Authentifizierung (2FA/MFA)	Zwei- oder Multi-Faktor-Authentifizierung (2FA/MFA) ist ein Sicherheitsmechanismus, bei dem zur Anmeldung an einem System zwei verschiedene Authentifizierungsfaktoren abgefragt werden. Dies können z. B. ein Passwort und ein einmaliger Code (z. B. aus einer App oder per SMS) sein. 2FA beziehungsweise MFA erhöht die Sicherheit erheblich, da ein Angreifer beide Faktoren kennen oder besitzen müsste, um sich erfolgreich anzumelden.

SCRAM-SHA-256

Authentifizierungsverfahren gemäss RFC 7677¹⁵, das nach dem Prinzip des Salted Challenge Response Authentication Mechanism (SCRAM) arbeitet. Passwörter werden dabei nicht im Klartext gespeichert oder übertragen, sondern mit einem zufälligen Salt und mehreren Iterationen nach dem Hash-Algorithmus SHA-256 gesichert. Während des Logins prüft der Server eine kryptografische Antwort (Challenge-Response), wodurch das Verfahren gegenüber Replay- und Wörterbuchangriffen geschützt ist. MongoDB verwendet SCRAM-SHA-256 als Standardmechanismus für Benutzeranmeldungen, um die Vertraulichkeit und Integrität der Authentifizierung zu gewährleisten.

Reverse Proxy

Vermittlungsinstanz zwischen Clients und internen Serverdiensten, die eingehende Anfragen entgegennimmt und an den entsprechenden Backend-Service weiterleitet. Dadurch werden interne Systeme nach aussen abgeschirmt, Lasten verteilt und zusätzliche Sicherheitsfunktionen wie TLS-Verschlüsselung, Zugriffskontrolle und Caching bereitgestellt. Reverse Proxys sind ein zentrales Element moderner Webarchitekturen und dienen der Trennung von öffentlicher und privater Netzwerkebene.

Virtual Private Network (VPN)

Ein Virtual Private Network (VPN) ist eine Technologie, die eine verschlüsselte Verbindung über ein öffentliches oder unsicheres Netzwerk ermöglicht. Durch die Nutzung eines VPNs wird der Datenverkehr zwischen Client und Server gesichert, wodurch Vertraulichkeit und Integrität der übertragenen Daten gewährleistet werden.

Systemhärtung (Hardening)

Massnahmen zur Reduktion der Angriffsfläche eines Systems durch Entfernen unnötiger Dienste, Einschränkung von Benutzerrechten und konsequente Sicherheitskonfiguration. Ziel ist es, die Widerstandsfähigkeit gegen Angriffe zu erhöhen und unautorisierte Zugriffe zu verhindern.

Bastion Host

Ein gehärteter Server, der als sicherer Zugangspunkt zu einem internen Netzwerk dient. Er vermittelt administrative Zugriffe über einen kontrollierten, MFA-geschützten Kanal und verhindert direkten Zugriff auf interne Systeme.

Time-based One-Time Password (TOTP)

Ein zeitbasiertes Einmalpasswort ist ein Verfahren zur Zwei-Faktor-Authentifizierung (2FA), bei dem temporäre, nur wenige Sekunden gültige Passwörter generiert werden. Diese werden auf einem Authenticator-Gerät oder in einer App erstellt. Die Synchronisation erfolgt über eine gemeinsame geheime Basis und die aktuelle Zeit, was eine zusätzliche Sicherheitsebene beim Login-Prozess bietet.

Phishing

Phishing bezeichnet den Versuch, über gefälschte E-Mails, Websites oder Nachrichten vertrauliche Informationen wie Passwörter, Kreditkartendaten oder Zugangsdaten zu erlangen. Angreifer nutzen hierbei Social-Engineering-Methoden, um das Vertrauen von Nutzern zu erschleichen. Ziel ist meist der unbefugte Zugriff auf Konten oder Systeme. Schulungen und Sensibilisierung helfen, Phishing-Angriffe zu erkennen und zu vermeiden.

CLOUD Act

Der Clarifying Lawful Overseas Use of Data Act (CLOUD Act) ist ein US-amerikanisches Gesetz von 2018, das US-Behörden den Zugriff auf Daten ermöglicht, die von US-Unternehmen gespeichert werden, auch wenn sich diese Daten physisch ausserhalb der USA befinden. Für in der Schweiz gehostete Systeme bedeutet dies, dass bei Nutzung von US-Cloud-Anbietern theoretisch ein ausländischer Datenzugriff möglich ist. Daher wird häufig auf Hosting in der Schweiz gesetzt, um die nationale Datenhoheit zu gewährleisten.

¹⁵<https://www.rfc-editor.org/rfc/rfc7677.html>

Kapitel IV

Literaturverzeichnis

- Alves, V., & Schnider, M. (2025). *Discord-Bot für Vereine* [Studien- und Bachelorarbeit].
- Angular-Contributors. (2025a,). *Testing*. <https://angular.dev/guide/testing>
- Angular-Contributors. (2025b,). *What is Angular?*. <https://angular.dev/overview>
- Arcane-Contributors. (2025,). *Arcane Discord Bot*. <https://arcane.bot/>
- AutoMQ-Contributors. (2025,). *Common Issues with Manual Deployment*. <https://www.automq.com/blog/fully-automated-deployment-vs-manual-deployment-comparison#common-issues-with-manual-deployment>
- Chou, Y.-k. (2015). *Actionable Gamification: Beyond Points, Badges and Leaderboards* (, Hrsg.). CreateSpace Independent Publishing Platform.
- CISA. (2022). *Implementing Multi-Factor Authentication (MFA) for Remote Access and VPNs* [Technical report]. https://www.cisa.gov/sites/default/files/publications/CISA_MFA_Guidelines_Remote_Access_2022.pdf
- ClubDesk-Contributors. (2025,). *Vereinssoftware für die einfache Vereinsverwaltung*. <https://www.clubdesk.de/de/startseite>
- Discord.js-Contributors. (2025b,). *Commits*. <https://github.com/discordjs/discord.js/commits/main/>
- Discord.js-Contributors. (2025a,). *The most popular way to build Discord bots*. <https://discord.js.org/>
- Docker-Contributors. (2025,). *Developers bring their ideas to life with Docker*. <https://www.docker.com/why-docker/>
- Europäische-Union. (2016,). *VERORDNUNG (EU) 2016/679 DES EUROPÄISCHEN PARLAMENTS UND DES RATES*. <https://eur-lex.europa.eu/legal-content/DE/TXT/?uri=CELEX:32016R0679>
- Jakob, N. (1993,). *Response Times: The 3 Important Limits*. Nielsen Norman Group. <https://www.nngroup.com/articles/response-times-3-important-limits/>
- madewithangular-Contributors. (2025,). *A showcase of web apps made with Angular*. <https://www.madewithangular.com/sites>
- Martin, F. (2003,). *Optimistic Offline Lock*. <https://martinfowler.com/eaCatalog/optimisticOfflineLock.html>
- Material-Contributors, A. (2025,). *Angular Material - Material Design components for Angular*. <https://material.angular.dev/>
- MEE6-Contributors. (2025,). *Statistics Channels Plugin*. <https://wiki.mee6.xyz/en/plugins/statistics-channels>
- Microsoft-Contributors. (2025b,). *Planning for mandatory multifactor authentication for Azure and other admin portals*. <https://learn.microsoft.com/en-us/entra/identity/authentication/concept-mandatory-multifactor-authentication?tabs=dotnet>
- Microsoft-Contributors. (2025a,). *Security at your organization: Multifactor authentication statistics*. <https://learn.microsoft.com/en-us/partner-center/security/security-at-your-organization>
- MongoDB-Contributors. (2025d,). *Authentication and Authorization with OIDC/OAuth 2.0*. <https://www.mongodb.com/docs/manual/core/oidc/security-oidc/>
- MongoDB-Contributors. (2025b,). *Introduction to MongoDB*. <https://www.mongodb.com/docs/manual/introduction/>
- MongoDB-Contributors. (2025f,). *Role-Based Access Control in Self-Managed Deployments*. <https://www.mongodb.com/docs/manual/core/authorization/>
- MongoDB-Contributors. (2025e,). *SCRAM*. <https://www.mongodb.com/docs/manual/core/security-scam/>
- MongoDB-Contributors. (2025a,). *Transactions*. <https://www.mongodb.com/docs/manual/core/transactions/>
- MongoDB-Contributors. (2025c,). *Use X.509 Certificates to Authenticate Clients on Self-Managed Deployments*. <https://www.mongodb.com/docs/manual/tutorial/configure-x509-client-authentication/>

- Murugiah, S., & Karen, S. (2017). *Application Container Security Guide (NIST SP 800-190)* [Technical report]. <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-190.pdf>
- NG-ZORRO-Contributors. (2025,). *Angular UI Component Library based on Ant Design*. <https://github.com/NG-ZORRO/ng-zorro-antd>
- NIST. (2020). *Security and Privacy Controls for Information Systems and Organizations (SP 800-53 Rev. 5)* [Technical report]. <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-53r5.pdf>
- OWASP-Cheat-Sheets-Series-Contributors. (2025,). *Session Management Cheat Sheet*. https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html
- OWASP-Contributors. (2021,). *OWASP Top 10*. <https://owasp.org/Top10/>
- Peters Dorian, R. R. M., Calvo Rafael A. (2018). Designing for Motivation, Engagement and Wellbeing in Digital Experience. *Frontiers in Psychology*, 9. <https://www.frontiersin.org/journals/psychology/articles/10.3389/fpsyg.2018.00797>
- PrimeNg-Contributors. (2025,). *The Next-Gen UI Suite for Angular*. <https://primeng.org/>
- React-Contributors. (2025,). *React*. <https://react.dev/>
- Schweizerische-Eidgenossenschaft. (2020,). *Bundesgesetz über den Datenschutz (Datenschutzgesetz, DSG)*. <https://www.fedlex.admin.ch/eli/cc/2022/491/de>
- U.S.-Department-of-Justice. (2019,). *The Purpose and Impact of the CLOUD Act - FAQs*. <https://www.justice.gov/criminal/media/999616/dl?inline>
- Vue.js-Contributors. (2025,). *What is Vue?*. <https://vuejs.org/guide/introduction>
- Weir, K. (2025,). *Self-determination theory: A quarter century of human motivation research*. <https://www.apa.org/research-practice/conduct-research/self-determination-theory>
- WildApricot-Community. (2015,). *Gamify Member's Site Participation*. <https://forums.wildapricot.com/forums/308932-wishlist/suggestions/10321731-gamify-member-s-site-participation>
- WildApricot-Contributors. (2025,). *Membership Management Software*. <https://www.wildapricot.com/>

Kapitel V

Abbildungsverzeichnis

Abb. 1	Die Bewertung des Vorprojektes im Octalysis-Modell	17
Abb. 2	Domain Model	27
Abb. 3	System Context Diagramm	28
Abb. 4	Container Diagramm	29
Abb. 5	Container Diagramm	31
Abb. 6	Login	32
Abb. 7	Bewerbungsformular	32
Abb. 8	Ranglisten-Seite	32
Abb. 9	Aufgaben-Seite	33
Abb. 10	Mitglieder-Seite	33
Abb. 11	Punktdefinitions-Seite	34
Abb. 12	Jahresplanung-Seite	34
Abb. 13	Belohnung-Seite	34
Abb. 14	Punktebeispiel-Seite	35
Abb. 15	Statistiken-Seite	35
Abb. 16	Logisches. Orange markiert sind die im Vergleich zum Vorprojekt modifizierten Entitäten.	40

Kapitel VI

Tabellenverzeichnis

Tab. 1	Beschreibung der Prioritäts-Kategorien	4
Tab. 2	Beschreibung der Resultats-Kategorien	5
Tab. 3	Beschreibung Functional Requirement 1	6
Tab. 4	Beschreibung Functional Requirement 2	6
Tab. 5	Beschreibung Functional Requirement 3	6
Tab. 6	Beschreibung Functional Requirement 4	6
Tab. 7	Beschreibung Functional Requirement 5	6
Tab. 8	Beschreibung Functional Requirement 6	7
Tab. 9	Beschreibung Functional Requirement 7	7
Tab. 10	Beschreibung Functional Requirement 8	7
Tab. 11	Beschreibung Functional Requirement 10	7
Tab. 12	Beschreibung Functional Requirement 9	7
Tab. 13	Beschreibung Functional Requirement 11	8
Tab. 14	Beschreibung Fully dressed Use Case 1	9
Tab. 15	Beschreibung Fully dressed Use Case 2	9
Tab. 16	Beschreibung Fully dressed Use Case 3	10
Tab. 17	Beschreibung Fully dressed Use Case 4	10
Tab. 18	Beschreibung Fully dressed Use Case 5	10
Tab. 19	Beschreibung Fully dressed Use Case 6	11
Tab. 20	Beschreibung Fully dressed Use Case 7	11
Tab. 21	Beschreibung Fully dressed Use Case 8	12
Tab. 22	Beschreibung Fully dressed Use Case 9	12
Tab. 23	Beschreibung Fully dressed Use Case 10	13
Tab. 24	Beschreibung Non-Functional Requirement 1	13
Tab. 25	Beschreibung Non-Functional Requirement 2	13

Tab. 26 Beschreibung Non-Functional Requirement 3	14
Tab. 27 Beschreibung Non-Functional Requirement 4	14
Tab. 28 Beschreibung Non-Functional Requirement 5	14
Tab. 29 Beschreibung Non-Functional Requirement 6	14
Tab. 30 Mapping der Interviewergebnisse zu den SDT-Grundbedürfnissen	16
Tab. 31 Die Bewertung des Vorprojektes im Octalysis-Modell	18
Tab. 32 Leitprinzipien für ClansCore	19
Tab. 33 Erweiterungen und Zielwirkung	19
Tab. 34 Beschreibung der vier grundlegenden Prinzipien	22
Tab. 35 Beschreibung der abgesicherten Kanäle in ClansCore	24
Tab. 36 Einschätzung der Risiken und deren Massnahmen	25
Tab. 37 Ablauf Unit-Tests	37
Tab. 38 Ablauf Integration-Tests	37
Tab. 39 Ablauf Usability-Test	37
Tab. 40 Ablauf System-Tests	37
Tab. 41 Beschreibung der architektonischen Merkmale des Vorprojekts	41
Tab. 42 Beschreibung der zentralen Änderungen der neuen Architektur	42
Tab. 43 Long-Term Plan mit ursprünglicher Construction-Phase (Version 1.0)	46
Tab. 44 Risiko Matrix	49
Tab. 45 Beschreibung Risiko 1	49
Tab. 46 Beschreibung Risiko 2	49
Tab. 47 Beschreibung Risiko 3	49
Tab. 48 Beschreibung Risiko 4	50
Tab. 49 Beschreibung Risiko 5	50
Tab. 50 Beschreibung Risiko 6	50
Tab. 51 Beschreibung Risiko 7	50
Tab. 52 Beschreibung Risiko 8	50
Tab. 53 Beschreibung Risiko 9	51
Tab. 54 Beschreibung Risiko 10	51

Tab. 55 Beschreibung Risiko 11	51
Tab. 56 Beschreibung Risiko 12	51
Tab. 57 Beschreibung Risiko 13	51
Tab. 58 Beschreibung Risiko 14	51
Tab. 59 Beschreibung der umgesetzten bzw. noch nicht umgesetzten Funktionen in Woche 10 ...	53
Tab. 60 Long-Term Plan mit angepasster Construction-Phase (Version 2.0)	54
Tab. 61 Übersicht der Repositories innerhalb der GitHub-Organisation ClansCore	57
Tab. 62 Beschreibung der evaluierten Web-Technologien	60
Tab. 63 Bilbiotheken für das User Interface	61
Tab. 64 Varianten zur Absicherung des DB-Zugangs	81

Kapitel VII

Code-Listings

Kapitel VIII

Anhang

15 Interviews zur Vereins-Motivation

Die Interviews wurden als halbstrukturierte, qualitative Gespräche mit vier anonymisierten Präsidenten Schweizer Gaming-Vereine geführt (durchschnittlich ungefähr 40 Minuten, leitfadengestützt). Die Auswahl erfolgte gezielt, um organisatorische und strategische Perspektiven abzubilden.

Der Leitfaden war entlang der drei Grundbedürfnisse der Self-Determination Theory (SDT: Autonomie, Kompetenz, soziale Eingebundenheit) strukturiert und wurde um praxisbezogene Blöcke (Kommunikation, digitale Tools, Fairness / Regeln, Kennzahlen) ergänzt. Die Auswertung erfolgte qualitativ-interpretativ in Form einer abgeleiteten Kodierung nach SDT, ergänzt um darauf aufbauende Subcodes (z. B. „Kommunikationsengpass“, „Anerkennung sichtbar machen“, „Saisonalität / Resets“, „Datensparsamkeit“).

Zur Sicherstellung eines klaren Aufbaus wurde der Leitfaden in sieben thematische Blöcke gegliedert:

1. **Allgemeine Fragen:** Kontext zu Rolle, Vereinsgrösse und Gründungsjahr zur Vergleichbarkeit der Antworten.
2. **Vereinsmotivation:** Aktuelle Beitrittsgründe, Bindungsfaktoren und Herausforderungen.
3. **Aktivitäten und Engagement:** Motivierende Angebote, Unterschiede zwischen neuen und erfahrenen Mitgliedern, Umgang mit Passivität.
4. **Digitale Tools und Gamification:** Nutzung digitaler Plattformen, Erfahrungen mit spielerischen Elementen, Chancen und Risiken.
5. **Zukunft und Anpassungen:** Wünsche nach Belohnungen und nachhaltigen Fördermassnahmen.
6. **Digitale Brücke:** Hypothetische Fragen zu einem digitalen Tool, Kennzahlen und gewünschten Features.
7. **Abschluss:** Offene Ergänzungen und Reflexionen.

Um unbeeinflusste Antworten zu gewährleisten, wurden digitale Lösungsansätze erst in den abschliessenden Blöcken thematisiert. Die vollständigen Leitfadenfragen sowie die anonymisierten Rohantworten und die Kodiermatrix sind als separate Dateien im Anhang abgelegt (vgl. „Gamification Interviews - Plan.pdf“ und „Gamification Interviews - Antworten.xlsx“).

Eine zusammenfassende Analyse der Ergebnisse erfolgt in [Abschnitt 3.2](#) des erweiterten Gamification-Konzeptes.

16 Technische Ergänzungen zum Sicherheitskonzept

Dieses Kapitel enthält technische Detailinformationen zum erweiterten Sicherheitskonzept ([Abschnitt 5](#)). Die hier dargestellten Inhalte dienen der Nachvollziehbarkeit der sicherheitsrelevanten Implementierungsaspekte und unterstützen die im Hauptteil beschriebenen konzeptionellen Massnahmen.

16.1 Vergleich der DB-MFA-Varianten

In der folgenden Tabelle sind die untersuchten MFA-Varianten für die in diesem Projekt verwendete MongoDB dargestellt:

Variante	Beschreibung	Stärken (+) und Schwächen (-)
Netzwerk-MFA (VPN / SSH-Bastion)	Zugriff auf die Datenbank ist nur über ein privates, nicht öffentlich erreichbares Netzwerk möglich. Der Zugang zu diesem Netzwerk wird über eine Bastion oder einen VPN-Gateway geregelt, der eine Multi-Faktor-Authentifizierung (z. B. TOTP oder Hardware-Token) verlangt. Damit wird der gesamte Zugriffspfad geschützt, ohne dass Backend oder Bot angepasst werden müssen. (Bezug: NFR4 , NFR5)	+ geringer Implementierungsaufwand, hohe Kompatibilität und transparente Integration in bestehende Infrastruktur. - VPN oder Bastion werden zu zentralen Kontrollpunkten, die gezieltes Monitoring und Backup-Strategien erfordern.
X.509-Clientzertifikate (mutual TLS)	Die Authentifizierung erfolgt über ein digitales Zertifikat als Besitzfaktor (MongoDB-Contributors, 2025c) (Bezug: NFR4). Diese Methode bietet einen hohen Schutz vor Phishing und erlaubt eine präzise Nachverfolgbarkeit einzelner Zugriffe.	+ stark gegen Phishing und gute Auditierbarkeit. - aufwändige Zertifikatsverwaltung, zusätzlicher Administrationsaufwand.
SSO (Single-Sign-On) / Identity-Provider-basierte MFA	Authentifizierung erfolgt über einen externen Identity Provider (z. B. Microsoft Entra ID ¹⁶) mittels OpenID Connect (nachfolgend OIDC genannt). MongoDB unterstützt seit Version 7.0 die direkte Integration solcher Anbieter, wodurch sich SSO und MFA zentral verwalten lassen (MongoDB-Contributors, 2025d) (Bezug: NFR4).	+ zentrale Benutzer- und Richtlinienverwaltung über Systeme hinweg. - erfordert zusätzliche Infrastruktur, Lizenzkosten und laufende Wartung.

Tab. 64: Varianten zur Absicherung des DB-Zugangs

Die Alternative über X.509-Zertifikate als besonders sicher eingestuft, ist aber aufgrund des erhöhten Verwaltungsaufwands und der nötigen Zertifikatsinfrastruktur für den Anwendungsfall eines Vereins-Tools nur bedingt sinnvoll.

Eine SSO- / IdP-basierte Lösung bietet theoretisch die beste Richtliniensteuerung und zentrale Benutzerverwaltung, würde aber zusätzliche Infrastruktur, Lizenzen und Wartungskosten verursachen. Da ClansCore bewusst als Open-Source-System für Vereine konzipiert ist, stehen Kosteneffizienz und einfache Wartbarkeit im Vordergrund. Die Einführung eines kommerziellen Identity-Providers würde diesen Grundsatz unterlaufen und die Einstiegshürde für kleinere Vereine erheblich erhöhen.

Aus diesen Gründen wird für ClansCore die Lösung Netzwerk-MFA über VPN beziehungsweise SSH-Bastion gewählt. Diese Variante bietet eine robuste Absicherung gegen unautorisierte Zugriffe, ist wirtschaftlich umsetzbar und unterstützt das Prinzip der klaren Trennung zwischen menschlichen und maschinellen Zugängen. Menschliche Admins authentifizieren sich über MFA auf der Netzwerkebene und systeminterne Dienste (z. B. Bot, Backend) kommunizieren ausschliesslich innerhalb des privaten Docker-Netzwerks über dedizierte Dienstkonten mit minimalen Rechten.

Die Wirksamkeit dieser Architektur wird durch externe Quellen untermauert. Laut Microsoft verhindern aktivierte MFA-Mechanismen ca. 99.2% bis 99.9% der üblichen Kontoübernahmen und werden deshalb in Zukunft potenziell zwingend eingeführt für VPN / Bastion ([Microsoft-Contributors, 2025a; 2025b](#)) . Ausserdem empfiehlt die Cybersecurity and Infrastructure Security Agency (CISA) den MFA-Einsatz, insbesondere für VPN- und Bastion-Zugänge ([CISA, 2022](#)) . Damit erfüllt die gewählte Lösung die anerkannten Best Practices moderner IT-Sicherheitsarchitekturen und bleibt zugleich ressourcenschonend, was den spezifischen Anforderungen einer Vereinssoftware gerecht wird.

Zusätzlich nutzt die Datenbank das standardisierte Authentifizierungsverfahren SCRAM-SHA-256 ([MongoDB-Contributors, 2025e](#)) in Kombination mit rollenbasierter Zugriffskontrolle ([MongoDB-Contributors, 2025f](#)) . Für zukünftige Ausbaustufen kann eine ergänzende Zertifikatsauthentifizierung für besonders privilegierte Konten in Betracht gezogen werden.

16.2 Dashboard-Sicherheit und Passwortmanagement

Nach erfolgreicher Anmeldung werden kurzlebige Zugriffstokens und rotierende Auffrischungstokens ausgestellt. Tokens werden als HttpOnly- und Secure-Cookies gespeichert und zustandsverändernde Anfragen erfordern ein CSRF-Token. Token-Lebenszyklen und Session-Invalidierung entsprechen den Empfehlungen des OWASP zur sicheren Sitzungsverwaltung ([OWASP-Cheat-Sheets-Series-Contributors, 2025](#)) .

¹⁶<https://www.microsoft.com/de-ch/security/business/identity-access/microsoft-entra-id>

Sämtliche Kommunikation zwischen Frontend und Backend erfolgt über TLS-verschlüsselte HTTPS-Verbindungen. Rollen- und Berechtigungsprüfungen erfolgen serverseitig über RBAC.

Vergessene Passwörter können über einen zeitlich begrenzten Reset-Link (Gültigkeit ≤ 30 Minuten) zurückgesetzt werden. Reset-Links werden ausschliesslich an die hinterlegte E-Mail-Adresse gesendet, ohne offenzulegen, ob ein Konto existiert. Die Passwort-Policy fordert mindestens 12 Zeichen, Kombination aus Gross- / Kleinbuchstaben, Ziffern und Sonderzeichen. Passwörter werden mit bcrypt (≥ 12 Rounds) oder einem gleichwertigen KDF sicher gehasht gespeichert. Fehlgeschlagene Anmeldeversuche führen zu progressiven Delays (Rate-Limiting). Erfolgreiche und fehlerhafte Änderungen werden im Audit-Log protokolliert.

Diese technischen Massnahmen dienen der Umsetzung der Anforderungen [NFR4](#) und [NFR5](#) und stellen sicher, dass Authentifizierungsvorgänge den Prinzipien von Privacy by Design und Defense in Depth entsprechen.

16.3 Nebenläufigkeitskontrolle und Transaktionen

Jeder Datensatz in der Datenbank enthält eine Versionskennung sowie einen Zeitstempel. Bei einem Update-Vorgang überprüft das Backend, ob die vom Client übermittelte Version noch mit der aktuellen Version in der Datenbank übereinstimmt. Wurde der Datensatz in der Zwischenzeit verändert, wird der Schreibvorgang abgewiesen und als Konflikt im Audit-Log vermerkt. Das Dashboard fordert daraufhin den aktuellen Datensatz an und kann die Änderungen manuell oder automatisch zusammenführen.

Für besonders sensible Vorgänge, wie Punktebuchungen, Rollenänderungen oder Event-Updates, werden zusätzlich atomare Transaktionen eingesetzt. Dadurch ist gewährleistet, dass zusammenhängende Operationen entweder vollständig oder gar nicht ausgeführt werden, was Inkonsistenzen in der Datenbank verhindert.

Alle Konflikte sowie deren Auflösung werden im Audit-Log dokumentiert.

16.4 Container- und Deployment-Details

Die Architektur besteht aus separaten Containern für Discord-Bot, Backend-Service, Admin-Dashboard, Reverse-Proxy und MongoDB. Jeder Container besitzt ein eigenes Dateisystem, eigene Prozessräume und dedizierte Netzwerk-Namespaces. Die Kommunikation erfolgt ausschliesslich über interne Docker-Netzwerke. Externe Zugriffe laufen ausschliesslich über den Reverse-Proxy mit TLS 1.3-Verschlüsselung. Die Datenbank ist nicht öffentlich erreichbar (siehe [Abschnitt 5.3.2](#)).

Container-Isolation basiert auf Linux-Namespaces, Cgroups und Capability-Drops, die laut NIST als Kernmechanismen moderner Container-Sicherheit gelten sowie die Angriffsfläche erheblich reduzieren. ([Murugiah & Karen, 2017](#))

Zugriff auf den Host erfolgt ausschliesslich über SSH-Key-Authentifizierung mit MFA über VPN / Bastion-Ebene ([CISA, 2022](#)). Das Deployment erfolgt automatisiert über GitHub Actions, wobei sämtliche Secrets (z. B. Tokens, Zugangsdaten) in verschlüsselten GitHub-Vaults verwaltet werden. Backups sind versioniert, verschlüsselt und nur für Admins zugänglich. (Bezug: [NFR4](#), [NFR5](#))

17 Test-Ergebnisse

18 Deployment

19 Screenshots Discord-Bot

20 Screenshots Dashboard